



第三章

数据链路层

陆有军 tjlyj@tongji.edu.cn



数据链路层

✍ 数据链路层设计问题

✍ 数据链路层协议

✍ 数据链路层协议举例



一些术语

✍ **节点**: 主机、路由器

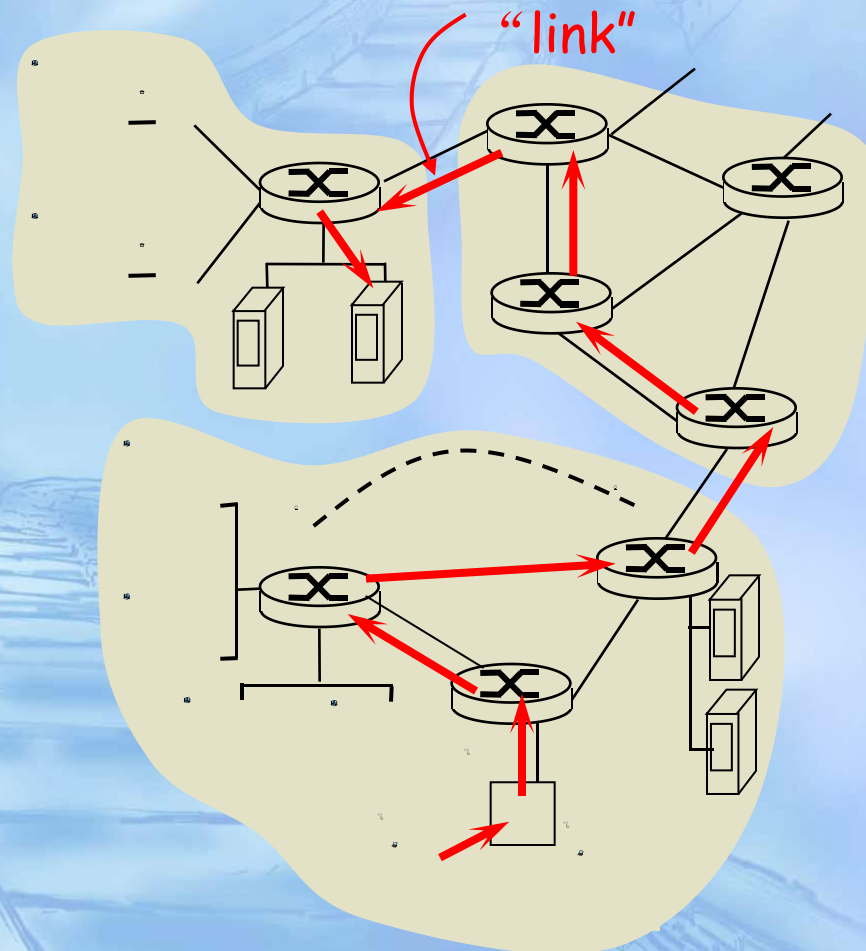
✍ **链路**: 通信路径中的相邻节点的路径:

✍ 有线链路

✍ 无线链路

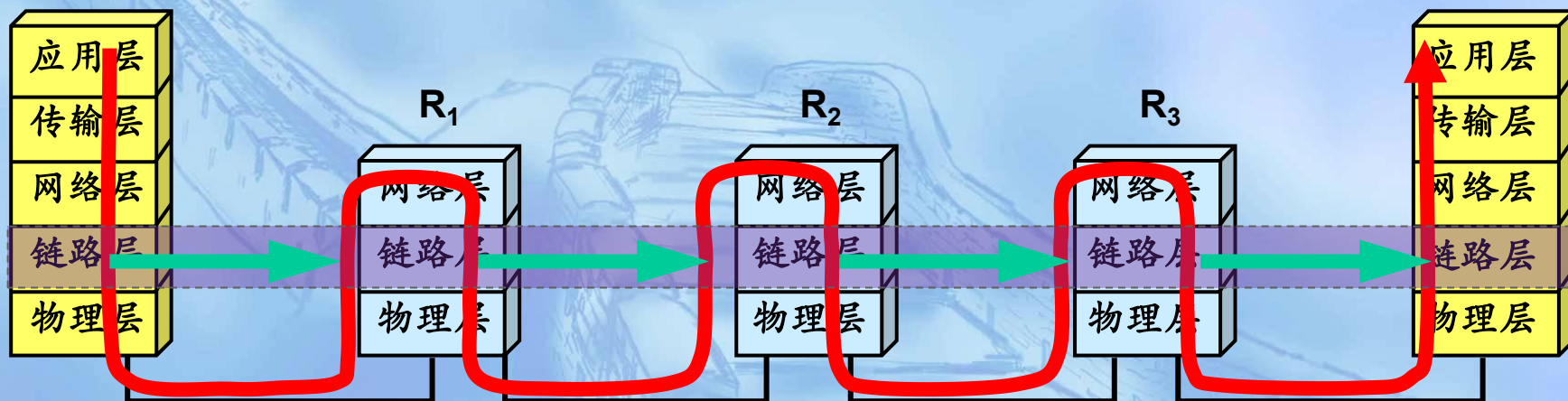
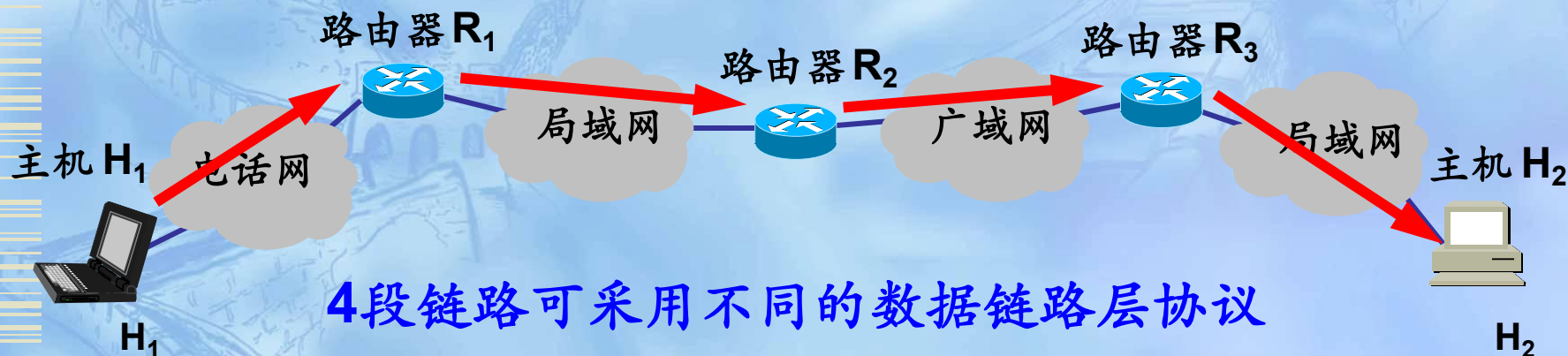
✍ 局域网

✍ **帧**: 第二层的数据传输单元, 用于封装高层数据。





数据传输过程



仅从数据链路层观察帧的流动



数据链路层中的帧





数据链路层目的和功能

目的

- 数据链路层主要研究的是两个直接相连的机器（点到点，链路）进行可靠、有效通信的算法。

功能

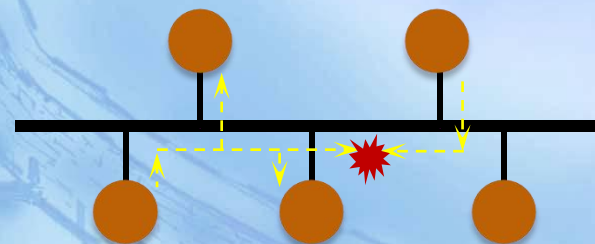
- 为网络层提供服务
- 数据成帧
- 差错控制
- 流量控制

链路类型



(a) 点对点链路信道

使用一对一的点对点通信方式。



(b) 广播信道

使用一对多的广播通信方式。

必须使用专用的MAC协议来协调这些主机的数据发送。

同济大学.计算机科学与技术学院



数据链路层功能

✍ 为网络层提供服务

✍ 数据成帧

✍ 差错控制

✍ 流量控制



服务类型一

(无确认的无连接服务)

工作原理

- 无需建立和释放连接;
- 目的节点正确收到帧后不确认;
- 丢失帧由上层恢复。

应用场合

- 低误码率场合;
- 实时系统, 例如, 声音传输;
- 大多数LAN, 例如: 以太网。



服务类型二

(有确认的无连接服务)

工作原理

- 无需建立和释放连接;
- 目的节点正确收到帧后发送确认帧;
- 源节点需配置存放待确认帧的缓冲区和计时器;
- 引入帧序号。

应用场合

- 不可靠信道, 例如: 无线系统 (WiFi) 。



服务类型三

(有确认的面向连接服务)

工作原理

-  步骤1：建立连接，初始化计时器和所需变量；
-  步骤2：帧传输；
-  步骤3：断开连接，释放资源。

应用场合

-  长距离且不可靠的链路，例如：卫星信道或者长途电话电路。



数据成帧

目的

- 将物理层来的比特流组合成数据链路层的数据帧。

方法

- 字节计数法
- 使用字符填充的首尾定界符法
- 使用比特填充的首尾定界符法
- 违法编码法



字节计数法

✍ **思想：** 采用一个特定的字符（例如：**SOH**）来表示一帧的开始，并以一个专门的字段（**Count**）来表示帧内的字节数。

✍ **例子：** DEC公司的DDCMP（数字数据通信报文协议）

bit	8	14	2	8	8	8	16	8~131064	8
	SOH	Count	Flag	Ack	Seg	Addr	CRC1	Data	CRC2

✍ **缺点**

- ✦ **Count**字段发生差错，将失去帧边界划分的依据；
- ✦ 所用的特定字符依赖于所采用的字符编码集，不利于兼容性。



使用字符填充的首尾定界符法

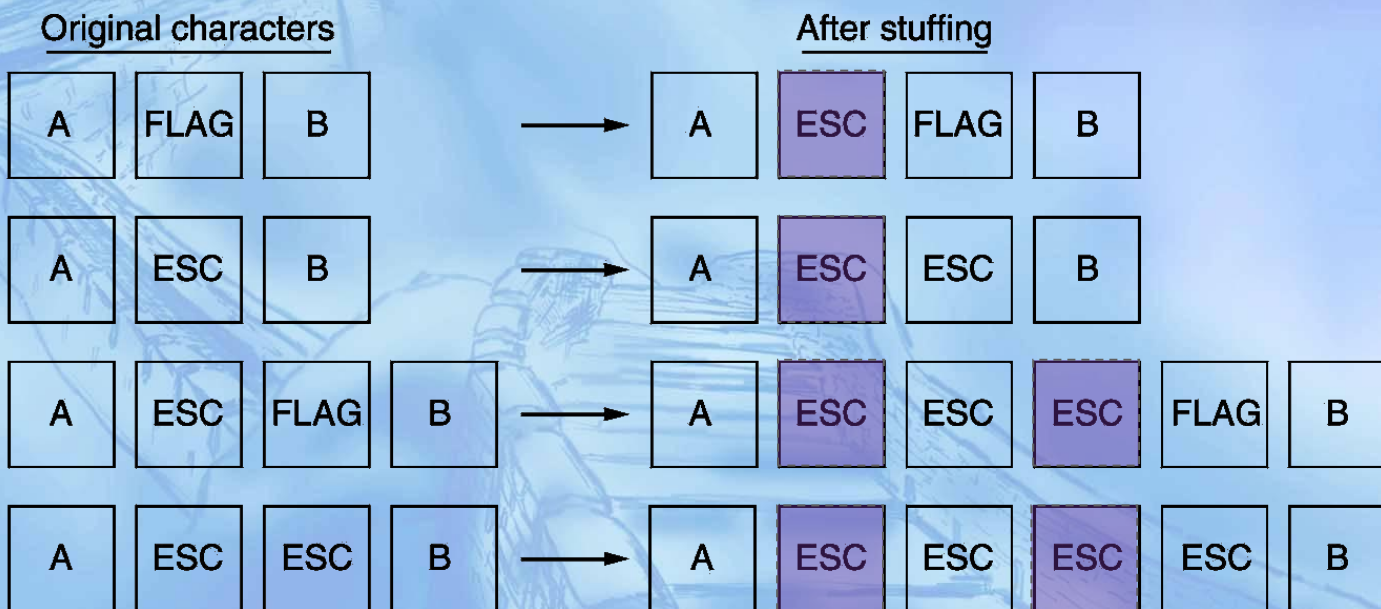
- ✍ **思想：**采用一些特定字符来表示一帧的开始和结束。
- ✍ **例子：**PPP协议（点对点协议）
- ✍ **问题：**如果传输的数据是字符串，则可实现透明传输；
如果传输的数据是二进制数据（例：多媒体数据），则数据中可能会出现用于定界的特定字符，干扰传输。
- ✍ **解决：**“字符插入/删除法”
- ✍ **缺点：**所用的特定字符依赖于所采用的字符编码集，不利于兼容性。



举 例



(a)



(b)



使用比特填充的首尾定界符法

- ✍ **思想：**采用一特定的比特组合（例如：**01111110**）来表示一帧的开始和结束。
- ✍ **例子：**HDLC（高级数据链路控制）协议
- ✍ **透明性传输：**如果传输的数据中出现用于定界的比特组合，会干扰传输，此时可采用“**0比特插入/删除法**”实现数据的透明性传输。
- ✍ **特点：**
 - ✦ 协议不依赖于任何一种字符编码集；
 - ✦ 用于实现数据的透明性传输的“**0比特插入/删除法**”易于硬件实现。



举 例

(a) 发送方的原始数据

(a) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

(b) 信道上“0比特插入”后的数据

(b) 0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0 0 1 0

Stuffed bits

(c) 接收方“0比特删除”后的数据

(c) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0



违法编码法

✍ 思想

- ✦ 采用违法的编码来表示一帧的开始和结束。

✍ 例子

- ✦ 局域网IEEE802标准（曼彻斯特编码：“高-低”电平对表示“1”，“低-高”电平对表示“0”，“高-高”和“低-低”电平对是非法的，可用来定界帧的开始和结束）。

✍ 特点

- ✦ 易实现数据的透明性传输，但只适用于采用冗余编码的特殊编码环境中。



差错控制

✍ 目的

- 在数据通信过程中能发现或纠正差错，将差错限制在尽可能小的允许范围内。

✍ 内容

- 传输差错类型
- 差错控制编码
- 差错控制方法



传输差错类型

✍ 随机错

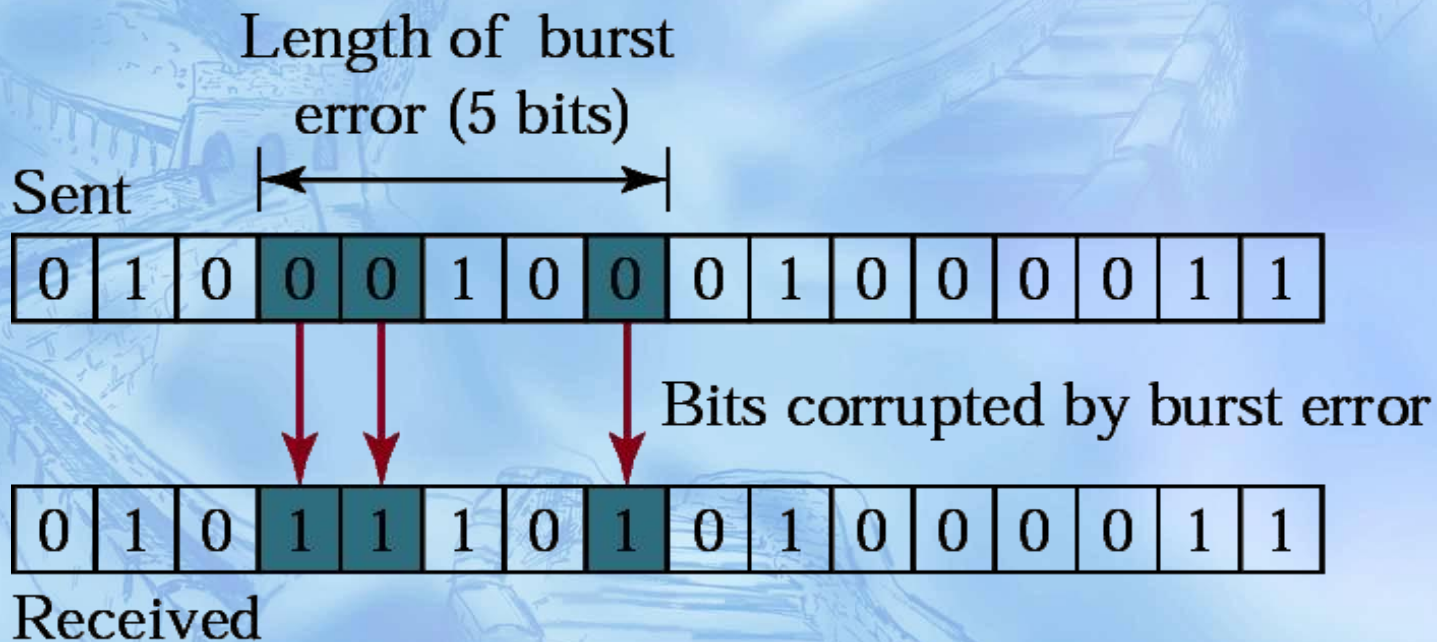
- 定义：由随机噪声引起的；
- 特点：差错是孤立的。此类错误在计算机网络中是个别的。

✍ 突发错

- 定义：由冲击噪声引起的；
- 特点：差错呈突发状，影响一批连续的bit（突发长度）。此类错误在计算机网络中是主要的。



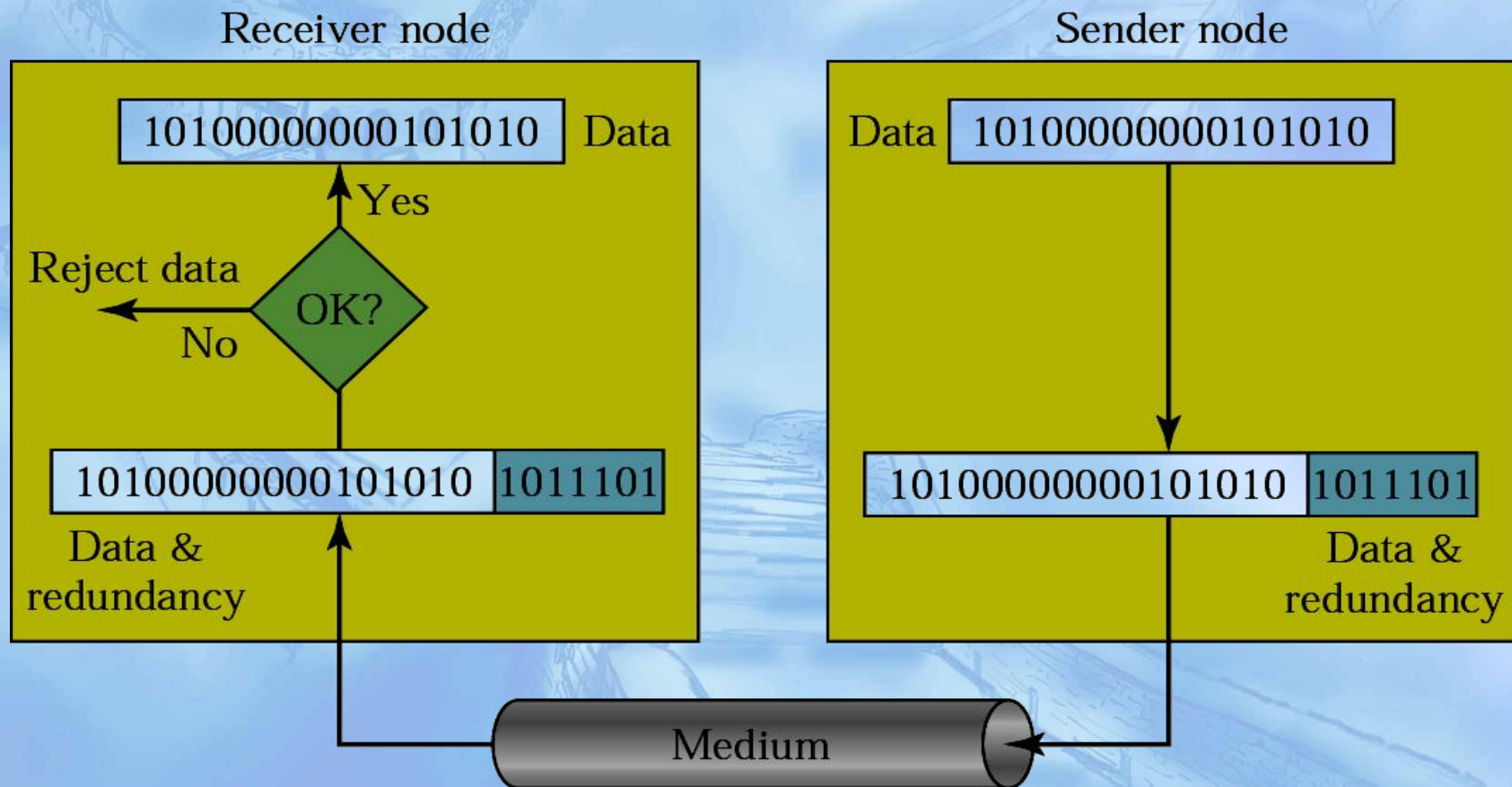
突发长度



✎ 例如：一次电火花（冲击噪声）持续时间为100ms，但对于4800bps的信道，会造成连续480bit的数据差错。



差错控制编码





编码分类

✍ 检错码

- 能自动发现差错的编码
- 例：奇偶校验码、校验和、循环冗余码等

✍ 纠错码

- 不仅能发现差错，而且能自动纠正差错的编码
- 例：海明码、二进制卷积码等



编码效率

编码效率 R 是编码性能好坏衡量的一个重要参数，若**码字**长 n 位，其中**信息位**长 k 位，**冗余位**长 r 位，则编码效率 R 为：

$$R = \frac{k}{n} = \frac{k}{k + r}$$



常用差错控制编码

 **检错码**

 奇偶校验码

 校验和

 循环冗余码

 **纠错码**

 海明码

 二进制卷积码

 里德所罗门码

 低密度奇偶校验码



奇偶校验码

✍ 编码原理

- 通过增加冗余位来使得码字中“1”的个数保持为奇数（奇校验）或偶数（偶校验）。例如，偶校验：110101000、011011011。

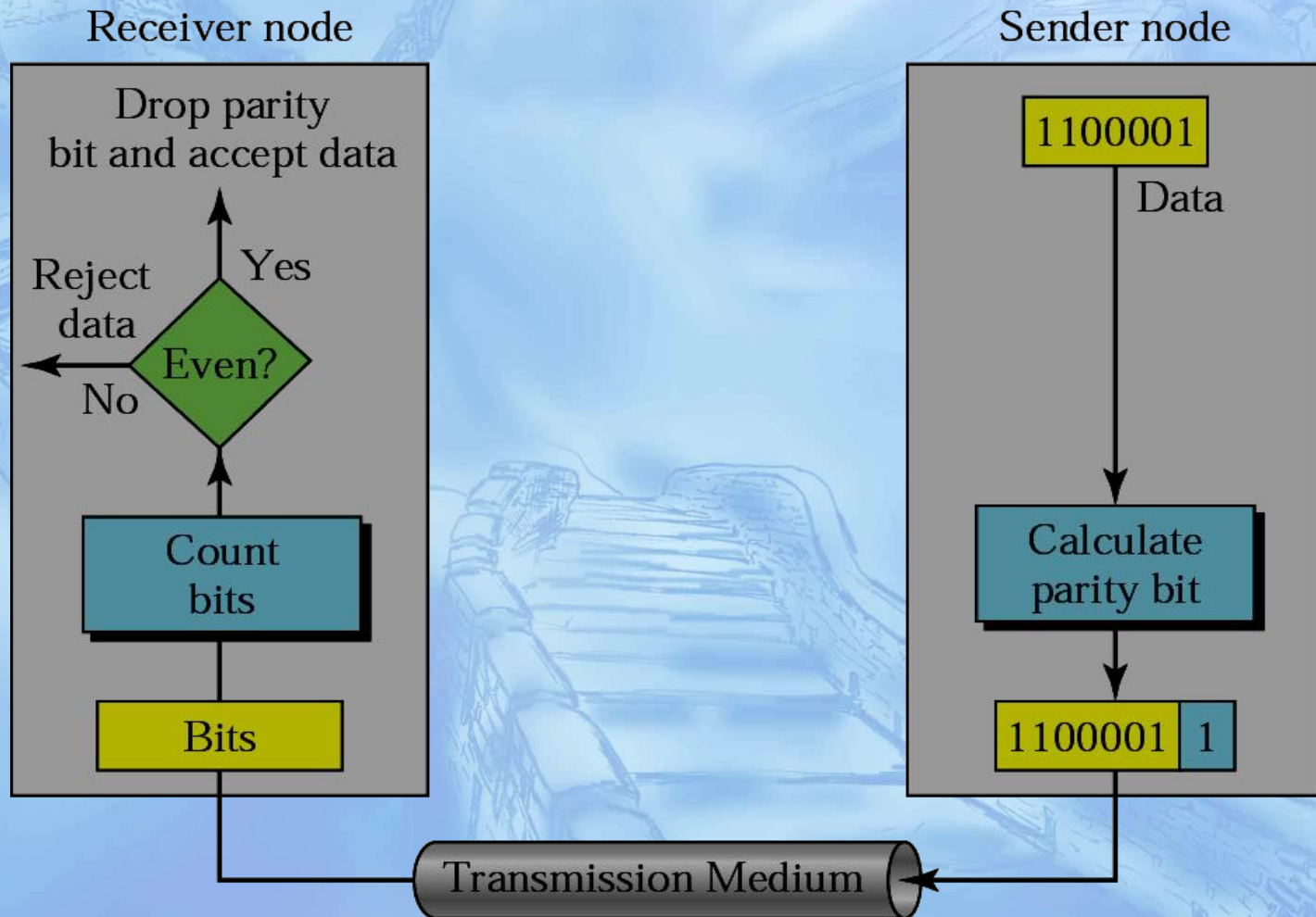
✍ 检错能力

- 能检测出所有单比特错，但突发错漏检率约为50%（只能检测出所有奇数个错）。



奇偶校验码

偶校验





二维奇偶校验码 偶校验

Original data

1100111 1011101 0111001 0101001





二维奇偶校验码

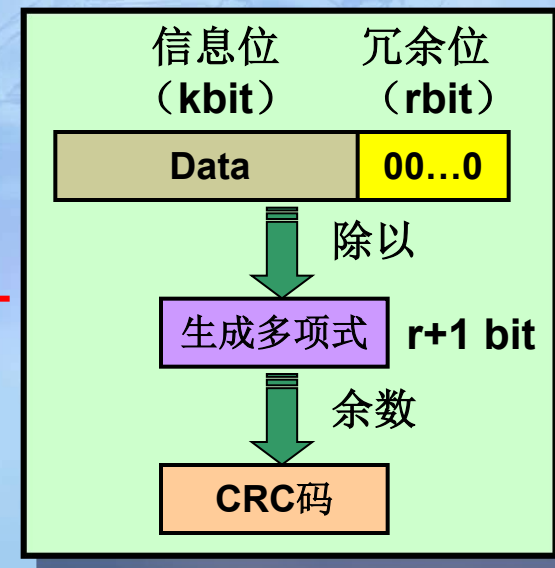
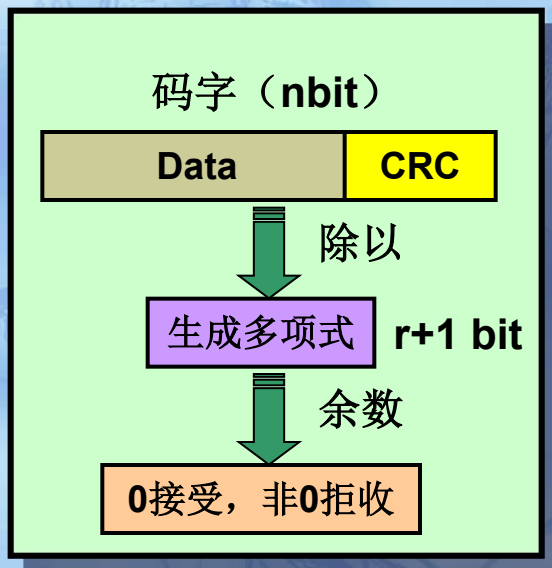
检错能力

假设有P个Qbit信息形成二维奇偶校验码，则：

- ✘ 编码效率 $R = PQ / (P+1)(Q+1)$;
- ✘ 能检出所有3位或3位以下的错误;
- ✘ 能检出所有奇数个错和很大一部分偶数个错;
- ✘ 对突发长度 $\leq Q+1$ 的突发错都能检出。



循环冗余码 (CRC)



✂ 接收端

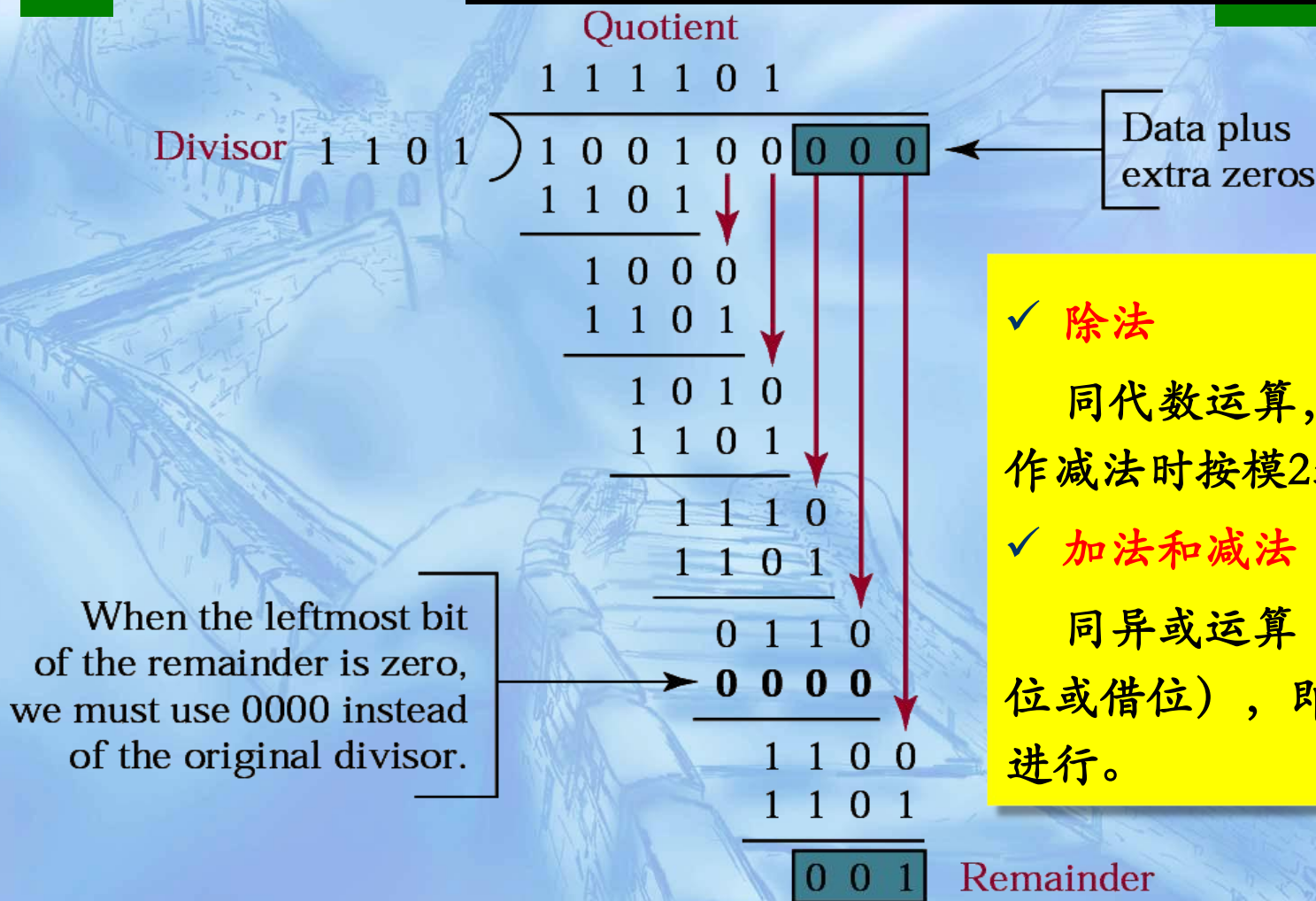
✂ 将收到的码字除以生成多项式, 如果余数为0, 则正确接收, 否则拒绝接收。

✂ 发送端

✂ 先在信息位后添加 r 个0, 然后除以生成多项式, 最后将得到的余数替代 r 个0形成码字进行传输。



例：发送端的编码



✓ 除法

同代数运算，只是在作减法时按模2进行。

✓ 加法和减法

同异或运算（不带进位或借位），即按模2进行。



例：接收端的校验

Quotient
1 1 1 1 0 1

Divisor 1 1 0 1) 1 0 0 1 0 0 0 0 1 ← Data plus CRC received

1 1 0 1

1 0 0 0

1 1 0 1

1 0 1 0

1 1 0 1

1 1 1 0

1 1 0 1

0 1 1 0

0 0 0 0

1 1 0 1

1 1 0 1

0 0 0

Result

When the leftmost bit of the remainder is zero, we must use 0000 instead of the original divisor.

✓ 除法

同代数运算，只是在作减法时按模2进行。

✓ 加法和减法

同异或运算（不带进位或借位），即按模2进行。



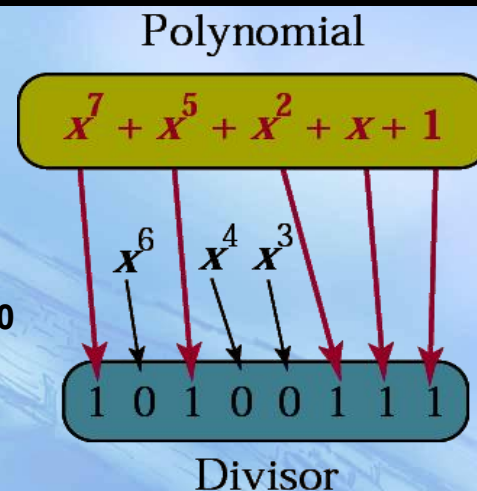
标准生成多项式

Name	Polynomial	Application
CRC-8	$x^8 + x^2 + x + 1$	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$	ATM AAL
ITU-16	$x^{16} + x^{12} + x^5 + 1$	HDLC
ITU-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	LANs

✍ 多项式和比特串的对应关系为：

比特串： $b_{n-1}b_{n-2} \dots b_2b_1b_0$

多项式： $b_{n-1}x^{n-1} + b_{n-2}x^{n-2} + \dots + b_2x^2 + b_1x^1 + b_0x^0$





CRC性质

- ✗ 若生成多项式 $G(x)$ 中含有 $(x+1)$ 的因子，则检出**所有奇数位错**。
- ✗ 若生成多项式 $G(x)$ 中不含有 x 的因子，即 $G(x)$ 中含有常数项1，则可检测所有**突发长度 $\leq r$** 的突发错。
- ✗ 若生成多项式 $G(x)$ 中不含有 x 的因子，且对任何 $0 < e \leq n-1$ 的 e ，除不尽 x^e+1 ，则能检测出**所有双位错**。
- ✗ 若生成多项式 $G(x)$ 中不含有 x 的因子，则对**突发长度为 $r+1$** 的突发错的漏检率为 **$2^{-(r-1)}$** 。
- ✗ 若生成多项式 $G(x)$ 中不含有 x 的因子，则对**突发长度大于 $r+1$** 的突发错的漏检率为 **2^{-r}** 。



CRC检错能力和应用

✎ CRC检错能力

若适当选取生成多项式 $G(x)$ ，使其含有 $(x+1)$ 因子，常数项不为0，则相应的CRC码可检测出：

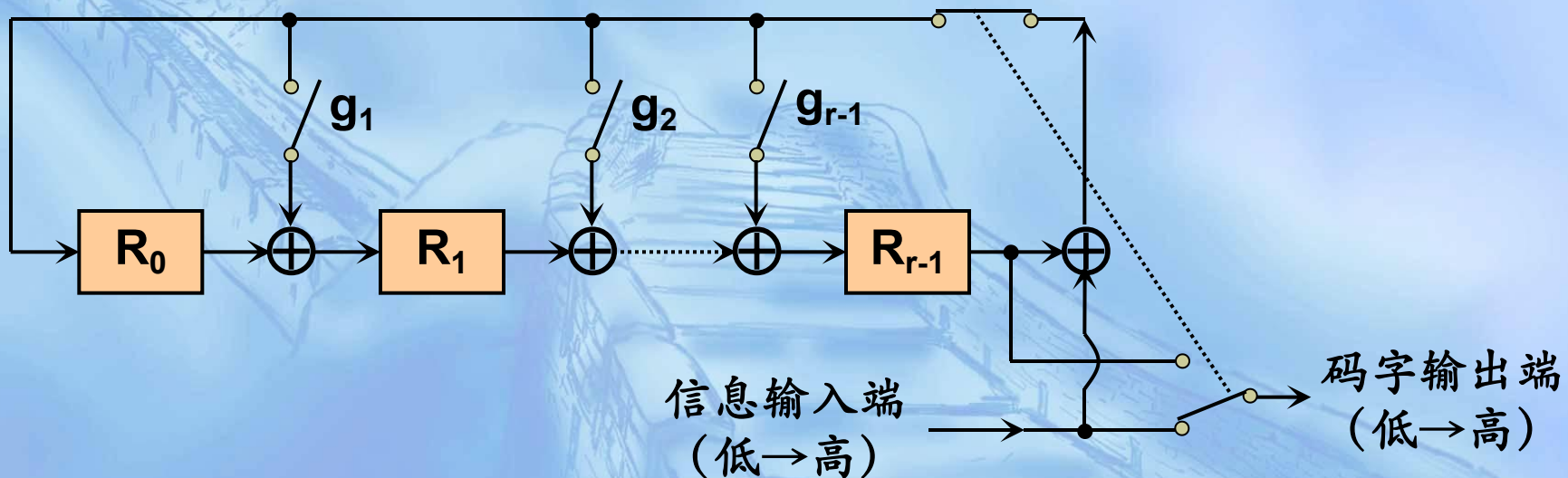
- ✦ 所有双位错；
- ✦ 所有奇数位错；
- ✦ 所有突发长度小于等于 r 的突发错；
- ✦ $(1-2^{-(r-1)})$ 的突发长度等于 $r+1$ 的突发错；
- ✦ $(1-2^{-r})$ 的突发长度大于 $r+1$ 的突发错。

✎ **应用：**数十种网络标准（以太网、802.11、SONET上的数据包等）采用了不同的CRC

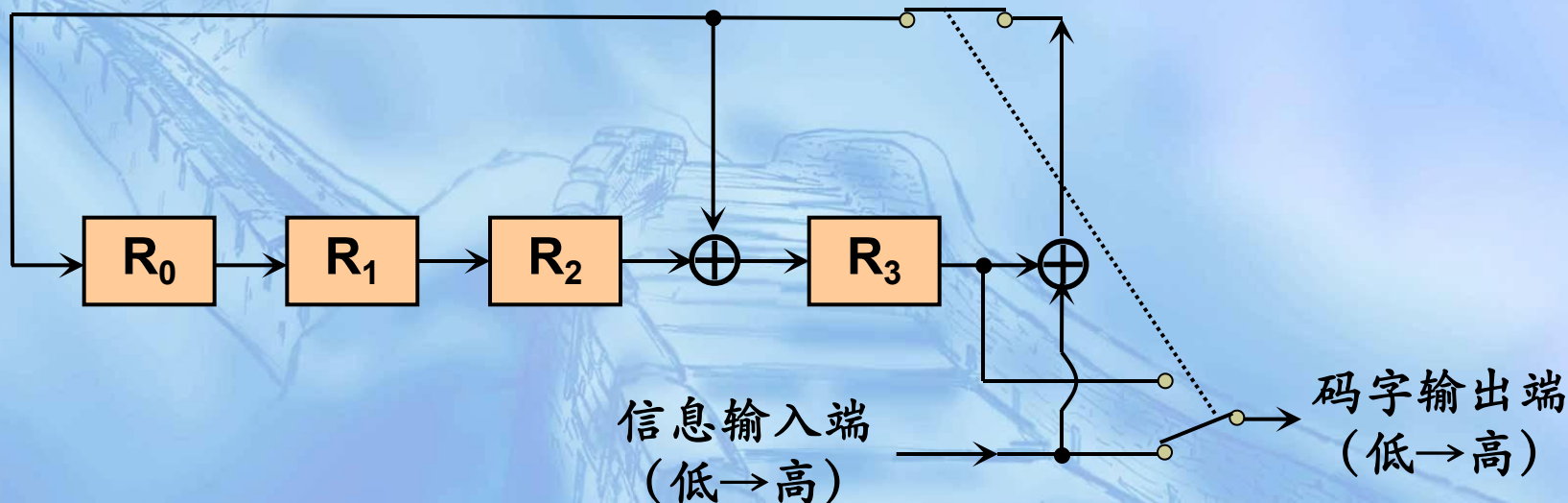


硬件编码电路

设 $G(x) = x^r + g_{r-1}x^{r-1} + \cdots + g_2x^2 + g_1x^1 + 1$ 则对应的硬件编码电路为：（由移位寄存器、半加器和开关组成）



生成多项式 $G(x) = x^4 + x^3 + 1$ 对应的硬件编码电路为：



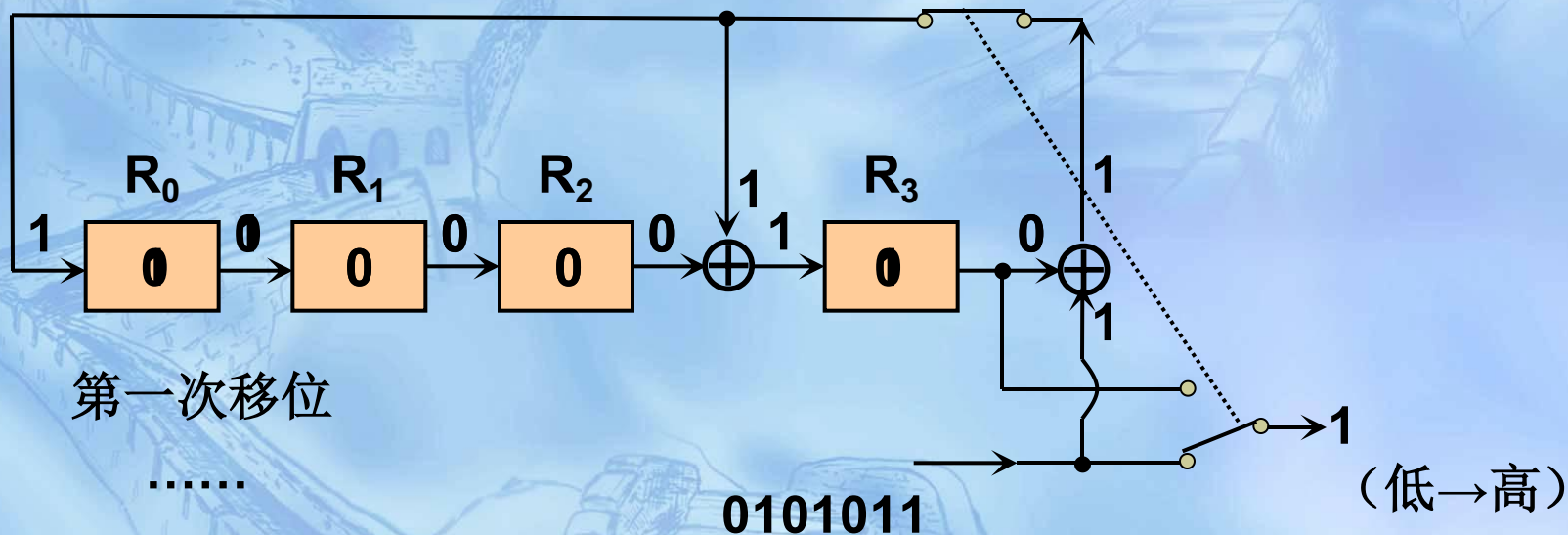


例：工作过程

(数据:1101010)

序号	输入 (低→高)	R_0	R_1	R_2	R_3	输出 (低→高)	备注
1		0	0	0	0		初始化
2	010101	1	0	0	1	1	将输出开关打到信息输入端，经过k(7)次移位，得到r位冗余位($R_{r-1} \dots R_1 R_0$)。
3	01010	0	1	0	0	11	
4	0101	0	0	1	0	011	
5	010	1	0	0	0	1011	
6	01	0	1	0	0	01011	
7	0	1	0	1	1	101011	
8		1	1	0	0	0101011	
9		×	1	1	0	00101011	将输出开关打到 R_{r-1} 寄存器，经过r(4)次移位，得到传送位 $n=k+r$ 。
10		×	×	1	1	000101011	
11		×	×	×	1	1000101011	
12		×	×	×	×	11000101011	

(数据:1101010)



✎ 将输出开关打到信息输入端，经过k (7) 次移位，得到r位冗余位 ($R_{r-1} \dots R_1 R_0$)。



海明码

 纠正一位错

 纠正突发错



海明距离

✍ 定义

- 两个码字中对应位比较，不同值的位的个数称为**海明距离**。对于码字长 n 位，数据 k 位，一般有 $n \geq k$ ，可以计算全部码字的海明距离，距离最小的就是**整个编码方案的海明距离**。

✍ 检验和纠错能力

✍ 检验码

欲检测出 d 比特错，需使用海明距离为 $d+1$ 的编码。

✍ 纠错码

欲纠正 d 比特错，需使用海明距离为 $2d+1$ 的编码。



校验位位数

✍ **定理：** 设数据位为 k 位，校验位为 r 位，码字位数为 n 位，若欲纠正单比特错，则海明码校验位的位数为 $(k+r+1) \leq 2^r$ 。

✍ **证明**

∵ 2^k 个有效数据每个均有 n 个距离为 1 的非法码字

∴ 2^k 个有效数据每个均需要 $n+1$ 个比特模式

又 ∵ 比特模式总数为 2^n

∴ $(n+1)2^k \leq 2^n$

考虑到 $n=k+r$ ，故 $(k+r+1) \leq 2^r$ <证毕>

✍ **例子**

$k=4$ ，则 $r \geq 3$ （取 $r=3$ ）； $k=7$ ，则 $r \geq 4$ （取 $r=4$ ）



发送方的编码

(纠错位的位置)

k 个数据位外加 r 个纠错位，在 2^i ($i=0,1,2,\dots$)的位置放的是纠错位， k 个数据位的次序不变。例如：字符“m”的7位二进制码为1101101，要加上4位纠错码0011（偶校验），共11个bit。

11	10	9	8	7	6	5	4	3	2	1
A7	A6	A5	P4	A4	A3	A2	P3	A1	P2	P1
1	1	0	0	1	1	0	0	1	1	1



发送方的编码

(信息位影响的纠错位)

信息位	位序号	位序号展开	影响的纠错位
A7	11	$=8+2+1$	对P4、P2、P1有影响
A6	10	$=8+2$	对P4、P2有影响
A5	9	$=8+1$	对P4、P1有影响
A4	7	$=4+2+1$	对P3、P2、P1有影响
A3	6	$=4+2$	对P3、P2有影响
A2	5	$=4+1$	对P3、P1有影响
A1	3	$=2+1$	对P2、P1有影响



发送方的编码

(纠错位的取值)

纠错位	纠错位的取值 (再取偶/奇校验)
P4	$=A7 \oplus A6 \oplus A5$
P3	$=A4 \oplus A3 \oplus A2$
P2	$=A7 \oplus A6 \oplus A4 \oplus A3 \oplus A1$
P1	$=A7 \oplus A5 \oplus A4 \oplus A2 \oplus A1$



发送方的编码

(举例)

纠错位	纠错位的取值 (再取偶校验)
P4	$=A7 \oplus A6 \oplus A5 = 0$
P3	$=A4 \oplus A3 \oplus A2 = 0$
P2	$=A7 \oplus A6 \oplus A4 \oplus A3 \oplus A1 = 1$
P1	$=A7 \oplus A5 \oplus A4 \oplus A2 \oplus A1 = 1$



接收方的编码 (方法)

校验位	校验位计算表达式
S4	$=A7 \oplus A6 \oplus A5 \oplus P4$
S3	$=A4 \oplus A3 \oplus A2 \oplus P3$
S2	$=A7 \oplus A6 \oplus A4 \oplus A3 \oplus A1 \oplus P2$
S1	$=A7 \oplus A5 \oplus A4 \oplus A2 \oplus A1 \oplus P1$

如计算结果**S4S3S2S1=0000**则表示接收正确，
如计算结果**S4S3S2S1=0101**则表示第5位出错。



接收方的编码

(举例:无错)

字符m采用海明码的发送序列为

11001100111

如接收的序列也为

11001100111

11	10	9	8	7	6	5	4	3	2	1
A7	A6	A5	P4	A4	A3	A2	P3	A1	P2	P1
1	1	0	0	1	1	0	0	1	1	1

校验位	校验位计算表达式
S4	$=A7 \oplus A6 \oplus A5 \oplus P4 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$
S3	$=A4 \oplus A3 \oplus A2 \oplus P3 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$
S2	$=A7 \oplus A6 \oplus A4 \oplus A3 \oplus A1 \oplus P2 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 = 0$
S1	$=A7 \oplus A5 \oplus A4 \oplus A2 \oplus A1 \oplus P1 = 1 \oplus 0 \oplus 1 \oplus 0 \oplus 1 \oplus 1 = 0$



接收方的编码

(举例:有错)

字符m采用海明码的发送序列为

11001100111

如接收的序列也为

110011**1**0111

11	10	9	8	7	6	5	4	3	2	1
A7	A6	A5	P4	A4	A3	A2	P3	A1	P2	P1
1	1	0	0	1	1	1	0	1	1	1

校验位	校验位计算表达式
S4	$=A7 \oplus A6 \oplus A5 \oplus P4 = 1 \oplus 1 \oplus 0 \oplus 0 = \mathbf{0}$
S3	$=A4 \oplus A3 \oplus A2 \oplus P3 = 1 \oplus 1 \oplus 1 \oplus 0 = \mathbf{1}$
S2	$=A7 \oplus A6 \oplus A4 \oplus A3 \oplus A1 \oplus P2 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 = \mathbf{0}$
S1	$=A7 \oplus A5 \oplus A4 \oplus A2 \oplus A1 \oplus P1 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 1 \oplus 1 = \mathbf{1}$

海明码只能纠正一位错，但采用右边方法可以纠正突发错：以码字为行，一行一个，构成 m 个码字的矩阵，按列发送，仅当突发长度 $\leq m$ 时，则可纠正该突发错。

《Computer Networks》 V6 (49/179)



差错控制方法

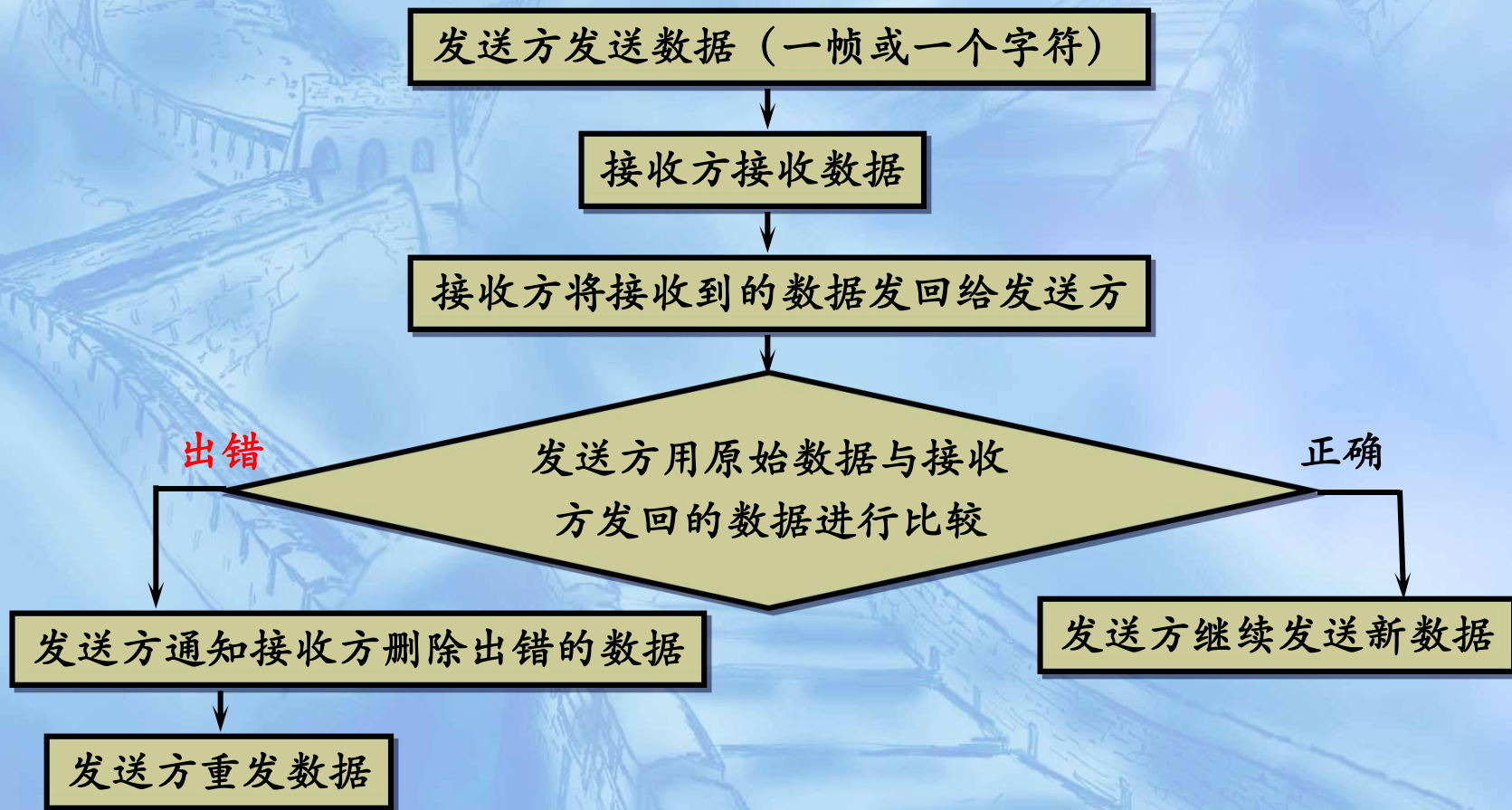
✍ 反馈检测法

✍ 自动请求重发

✍ 前向纠错



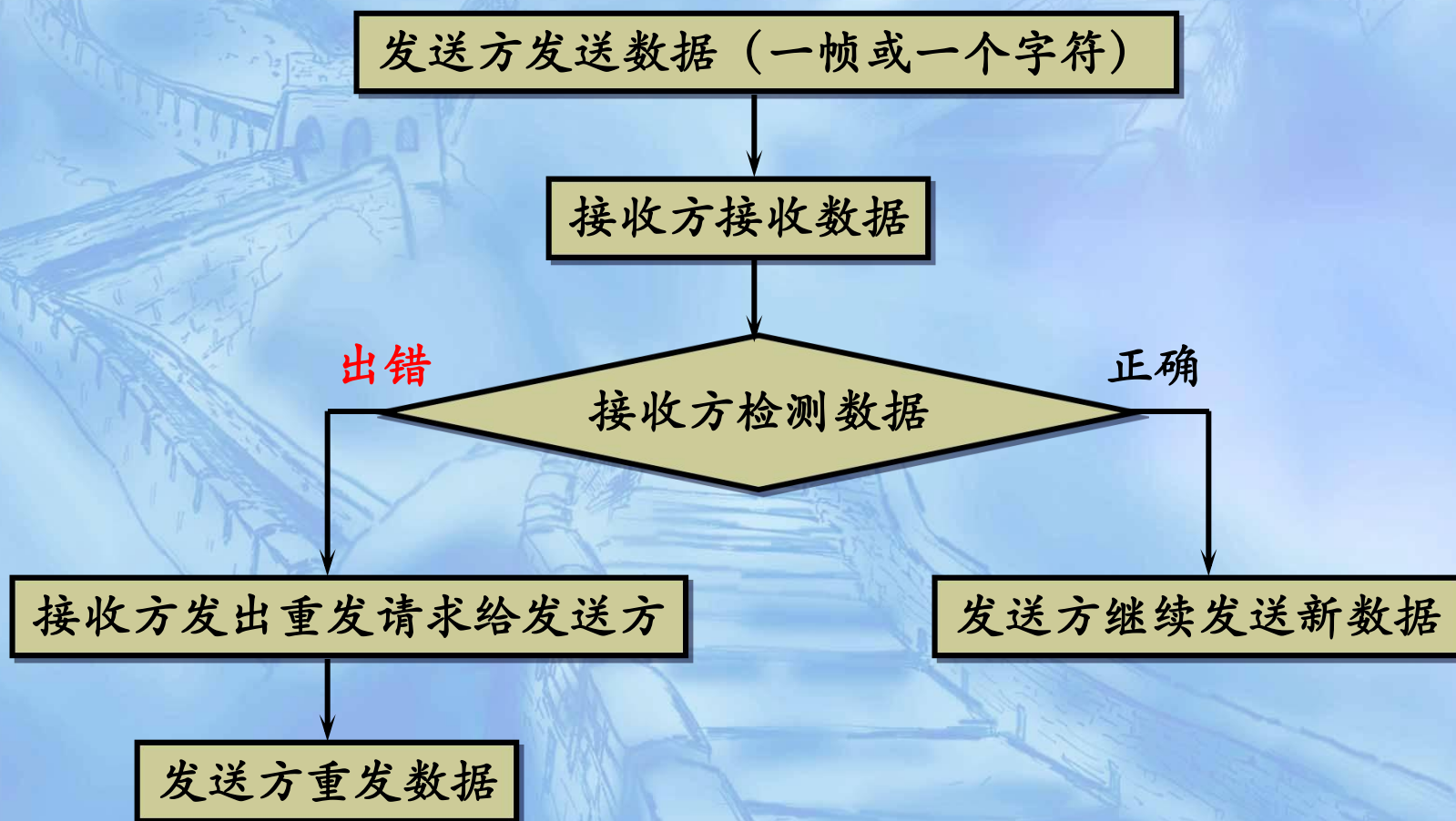
反馈检测法



✎ **特点：**原理简单、实现容易、可靠性强，但开销大，信道利用率低。



自动请求重发 (ARQ)





特点

✍ 为保证通信正常进行，还需引入：

- ✦ 定时器：防止整个数据帧或反馈信息丢失。
- ✦ 帧序号：防止接收方多次收到同一帧并递交网络层。

✍ 特点：

- ✦ 使用检错码；
- ✦ 必须是双向信道；
- ✦ 发送方需设置缓冲器。

✍ 应用：

- ✦ 高可靠性信道，例如光纤；
- ✦ 经常被用在数据链路层、网络层和传输层。



前向纠错 (FEC)

发送方发送数据（一帧或一个字符）

接收方接收数据

接收方检测数据，如发现差错，
由接收方进行纠正

发送方继续发送新数据

✍ **应用：**

- ✦ 有噪声的信道，例如无线信道；
- ✦ 经常被用在数据链路层和物理层。

✍ **特点：**

- ✦ 使用纠错码（纠错码编码效率低且设备复杂）；
- ✦ 单向信道；
- ✦ 发送方无需设置缓冲器。



流量控制

✎ **问题：**发送方发送能力大于接收方接收能力。例如，高速发送方的帧会“淹没”低速的接收方。

✎ **解决**

✦ **基于反馈的流量控制**（本章介绍）

接收方显式或隐式通知发送方发送速率是否可接受。

✦ **基于速率的流量控制**

由协议限制发送方可发送的速率，无需接收方反馈。

✎ **提示：**解决方法原理大致相同，都是限制发送方所发出的数据流量，使其发送速率不要超过接收方能处理的速率。



数据链路层协议

 基本数据链路协议

 滑动窗口协议

 协议描述和验证



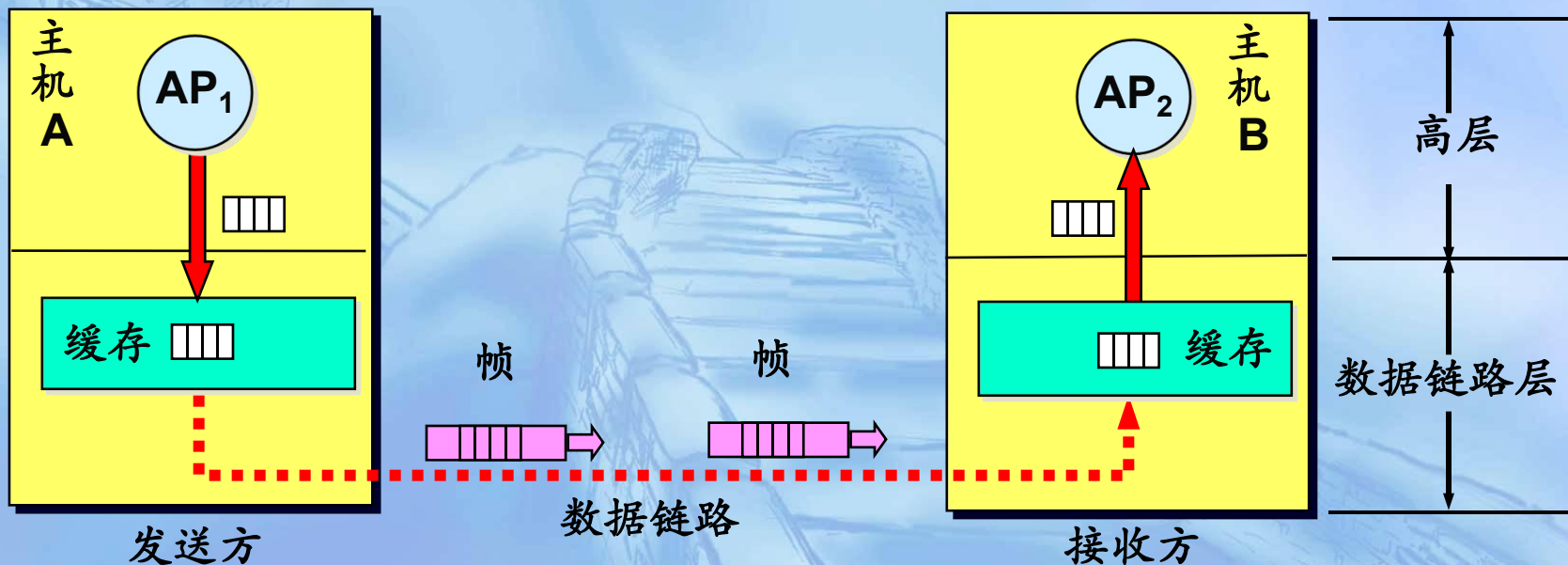
基本数据链路协议

- ✍ 一些约定
- ✍ 一个乌托邦式的单工协议
- ✍ 无错信道上的单工停-等式协议
- ✍ 有错信道上的单工停-等式协议
- ✍ 协议效率分析



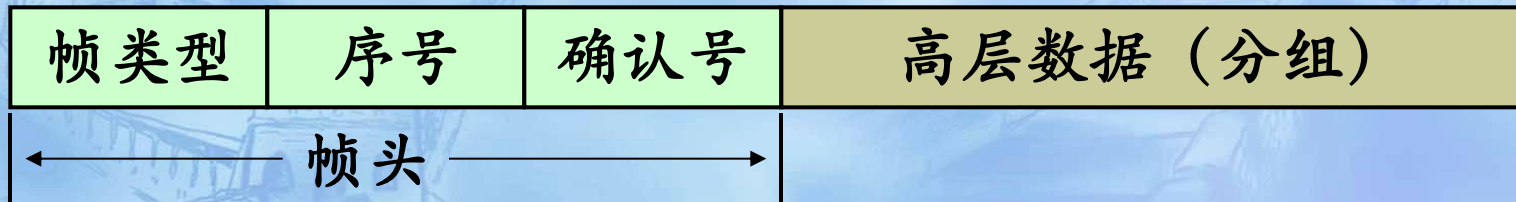
简化的通信模型

- ✘ 假设物理层、数据链路层和网络层都是**独立的进程**；
- ✘ 假设主机A用一个可靠的、面向连接的服务向主机B发送长数据流（**单工**）；且准备发送的数据是**无限**的；
- ✘ 假设主机不会崩溃；
- ✘ 硬件计算帧校验和。





帧格式



✍ 帧类型

- ✦ 数据帧data、肯定确认帧ack

✍ 序号

- ✦ 当前发送的数据帧序号

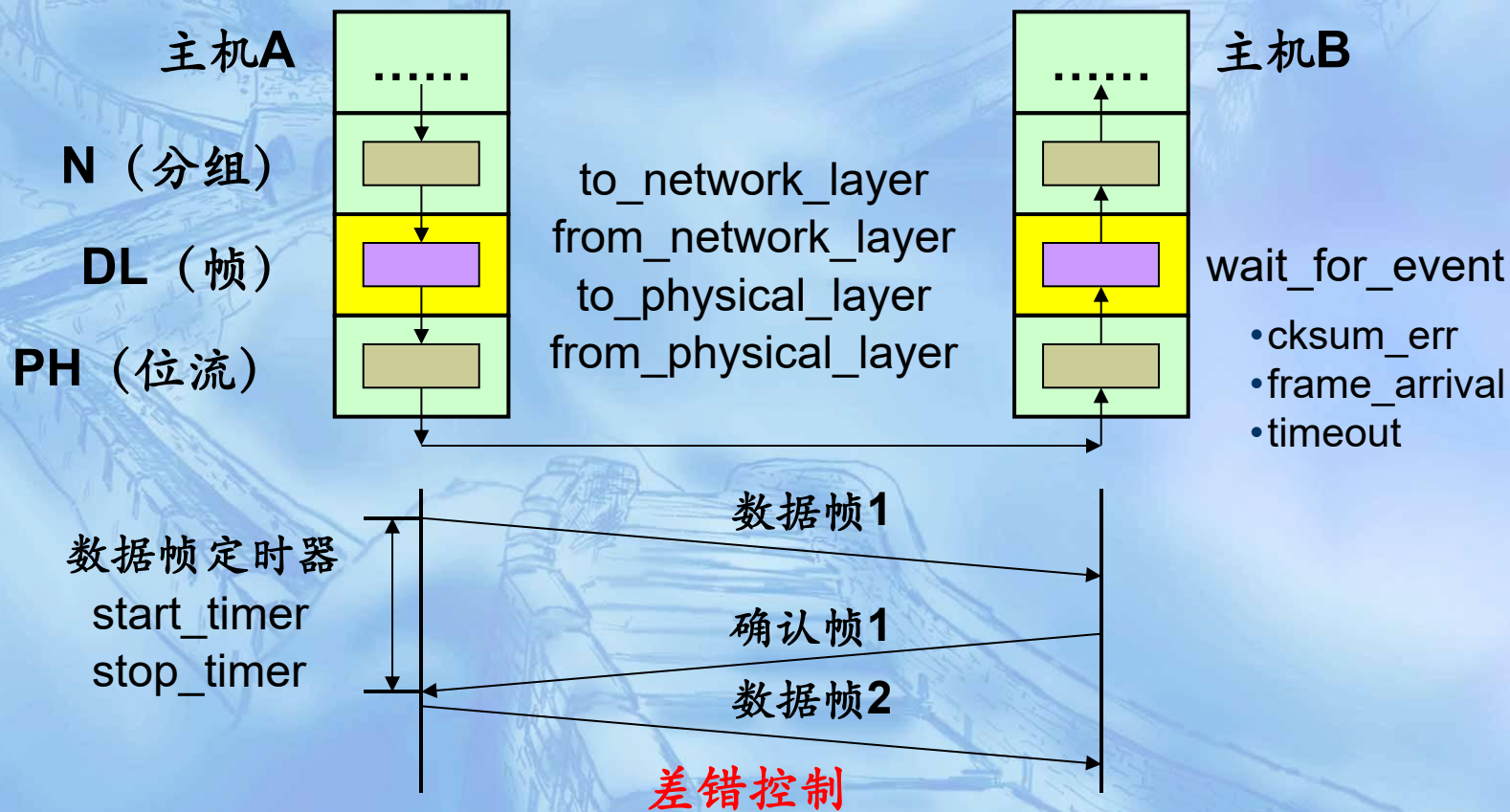
✍ 确认号

- ✦ 确认刚接收成功的数据帧序号

注：参见教材P174图3-11中的数据结构



函数



注： 参见教材P174图3-11中的函数



假 设

✍ 假设一：无差错控制

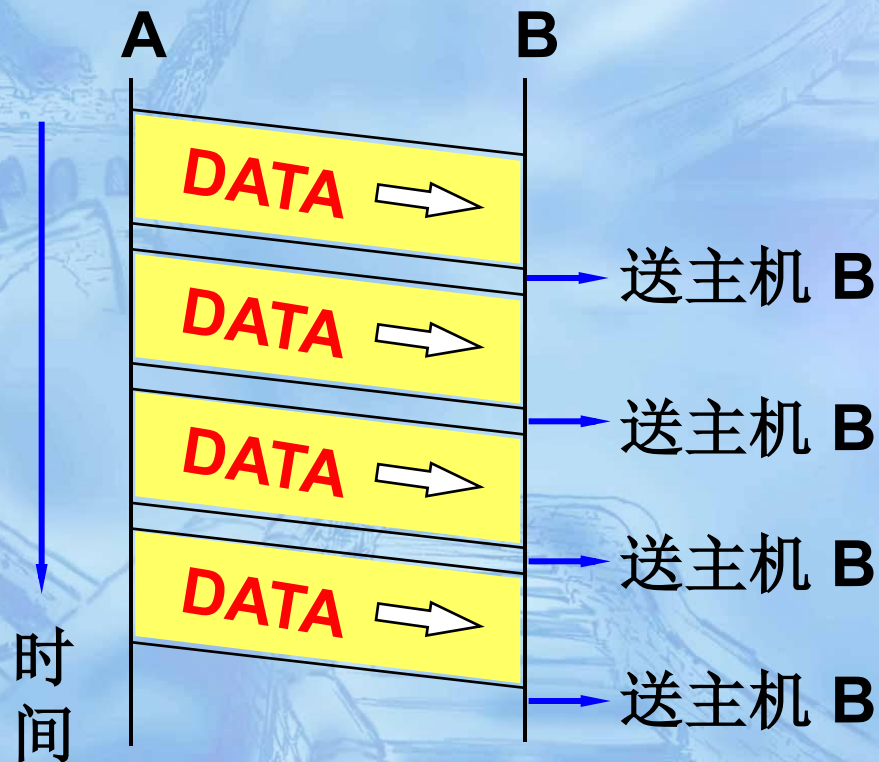
链路是理想的传输信道，所传送的任何数据既不会出差错也不会丢失。

✍ 假设二：无流量控制

不管发方以多快的速率发送数据，收方总是来得及收下，并及时上交主机。



图示数据传输



完全理想化的数据传输



协议1: sender

```
void sender1(void)
{
    frame s;
    packet buffer;

    while (true)
    {
        from_network_layer(&buffer);
        s.info = buffer;
        to_physical_layer(&s);
    }
}
```



协议1: receiver

```
typedef enum {frame_arrival} event_type;
void receiver1(void)
{   frame r;
    event_type event;

    while (true)
    {   wait_for_event(&event);
        from_physical_layer(&r);
        to_network_layer(&r.info);
    }
}
```



假 设

✍ 假设一：无差错控制

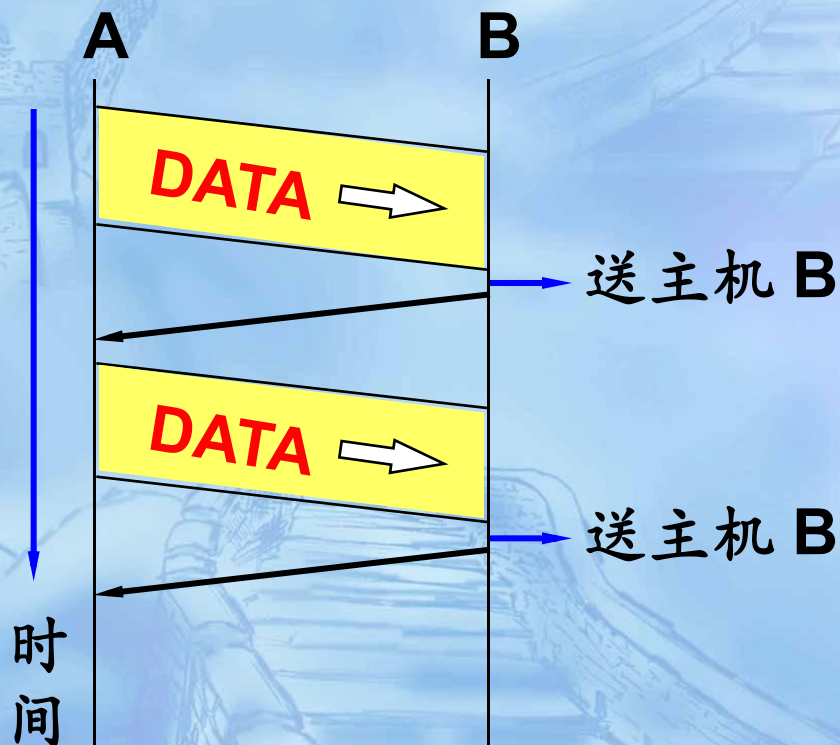
链路是理想的传输信道，所传送的任何数据既不会出差错也不会丢失。

✍ 假设二：无流量控制 (✖)

不管发送方以多快的速率发送数据，接收方总是来得及收下，并及时上交网络层。



图示流量控制



具有最简单流量控制（**停-等式协议**）的数据传输



协议2: sender

```
typedef enum {frame_arrival} event_type;
void sender2(void)
{   frame s;
    packet buffer;
    event_type event;

    while (true)
    {   from_network_layer(&buffer);
        s.info = buffer;
        to_physical_layer(&s);
        wait_for_event(&event);    }
}
```



协议2:receiver

```
void receiver2(void)
{
    frame r, s;
    event_type event;

    while (true)
    {
        wait_for_event(&event);
        from_physical_layer(&r);
        to_network_layer(&r.info);
        to_physical_layer(&s);
    }
}
```

确认帧是哑帧



假 设

假设一：无差错控制 (✕)

链路是理想的传输信道，所传送的任何数据既不会出差错也不会丢失。

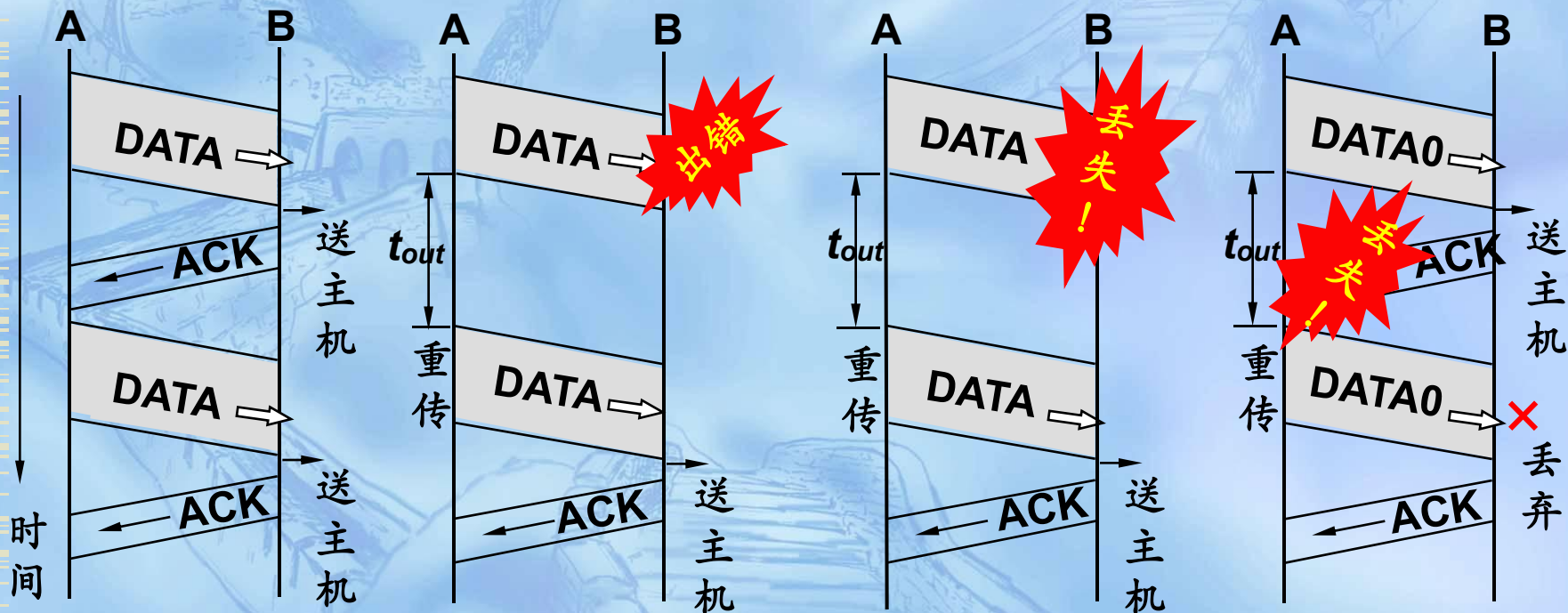
假设二：无流量控制 (✕)

不管发送方以多快的速率发送数据，接收方总是来得及收下，并及时上交网络层。

图示差错控制

(停-等式协议)

也是ARQ协议!



(a) 正常情况
ACK确认帧

(b) 数据帧出错
定时器

(c) 数据帧丢失
定时器

(d) 确认帧丢失
定时器+ 帧序号

停-等式协议: 肯定确认+定时器+帧序号



定时器和帧编号

✎ 超时定时器

- ✦ 主机每发送完一个数据帧都需启动该定时器，超时可重发。
- ✦ 重传时间 t_{out} 一般可设为略大于“从发完数据帧到收到确认帧所需的平均时间”。

✎ 帧编号

- ✦ 用于区别接收的数据帧是否重复，如重复则丢弃重复帧，但必须向对方补发最后一个有效确认帧**ACK**。
- ✦ 对于停-等式协议来说，用于编号的序号字段只需**1bit (0/1)** 即可。



协议3: sender

```
typedef enum {frame_arrival, cksum_err, timeout} event_type;
void sender3(void)
{
    seq_nr next_frame_to_send=0;
    frame s;      packet buffer;      event_type event;
    from_network_layer(&buffer);
    while (true)
    {
        s.info = buffer;      s.seq = next_frame_to_send;
        to_physical_layer(&s);      start_timer(s.seq);
        wait_for_event(&event);
        if (event == frame_arrival)
        {
            from_physical_layer(&s);
            if (s.ack == next_frame_to_send)
            {
                stop_timer(s.ack);
                from_network_layer(&buffer);
                inc(next_frame_to_send);
            }
        }
    }
}
```

ACK 帧



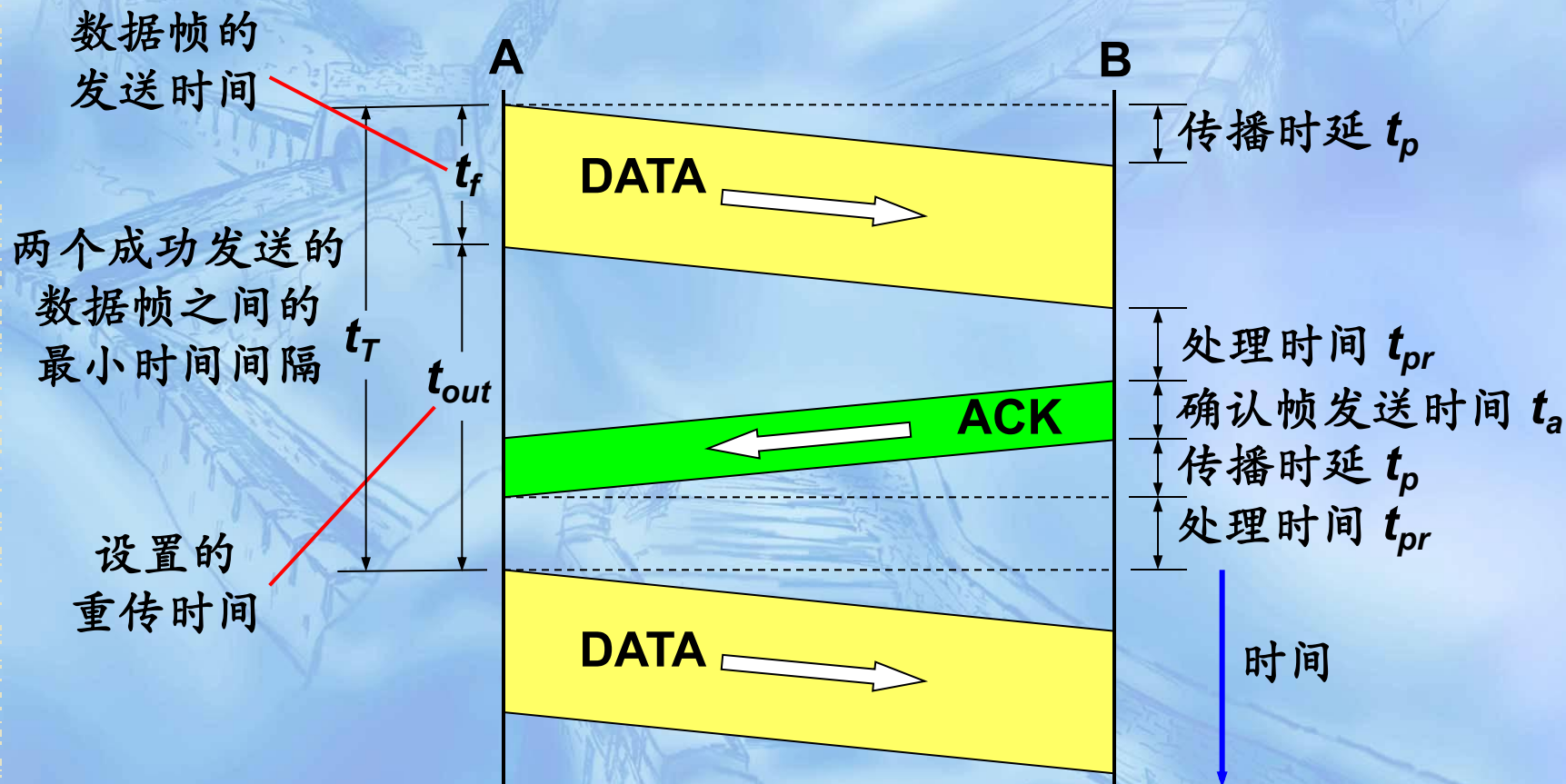
协议3: receiver

```
void receiver3(void)
{  seq_nr frame_expected=0;
   frame r, s;
   event_type event;
   while (true)
   {   wait_for_event(&event);
       if (event == frame_arrival)
       {   from_physical_layer(&r);
           if (r.seq == frame_expected)
           {   to_network_layer(&r.info);
               inc(frame_expected);
           }
           s.ack = 1 - frame_expected;
           to_physical_layer(&s);
       } } }
```

数据帧



停-等协议中的时间





重传时间 t_{out}

✍ 作用

数据帧发送完毕后若经过了这样长的时间还没有收到确认帧，就重传这个数据帧。

✍ 设置

$$t_{out} = t_p + t_{pr} + t_a + t_p + t_{pr}$$

设上式右端的处理时间 t_{pr} 和确认帧的发送时间 t_a 都远小于传播时延 t_p ，因此可将重传时间取为两倍的传播时延，即：

$$t_{out} = 2t_p$$



一个帧的平均重传次数

设数据帧出现差错(包括帧丢失)的概率为 p , 但假设确认帧不会出现差错。则一帧的平均重传次数:

$$\begin{aligned} &= \{1 \times P[\text{重传次数为1}] + 2 \times P[\text{重传次数为2}] + 3 \times \\ &\quad P[\text{重传次数为3}] + \dots\} \\ &= \{1 \times P[\text{第1次发送出错}] \times P[\text{第2次发送成功}] \\ &\quad + 2 \times P[\text{第1, 2次发送出错}] \times P[\text{第3次发送成功}] \\ &\quad + 3 \times P[\text{第1, 2, 3次发送出错}] \times P[\text{第4次发送成功}] \\ &\quad + \dots\} \\ &= p(1 - p) + 2p^2(1 - p) + 3p^3(1 - p) + \dots \end{aligned}$$

这里 $P[X]$ 是出现事件 X 的概率。



信道利用率

✎ 两个发送成功的数据帧之间的最小时间间隔 t_T 为：

$$t_T = t_f + t_{out} = t_f + 2t_p$$

✎ 正确传送一个数据帧所需的平均时间 t_{av} 为：

$$t_{av} = t_T (1 + \text{一个帧的平均重传次数})$$

$$t_{av} = t_T + (1-p) \sum_{i=1}^{\infty} i p^i t_T = t_T / (1-p)$$

当传输差错率增大时， t_{av} 也随之增大。当无差错时， $p = 0$ ， $t_{av} = t_T$ 。

✎ 信道利用率： t_f / t_{av}



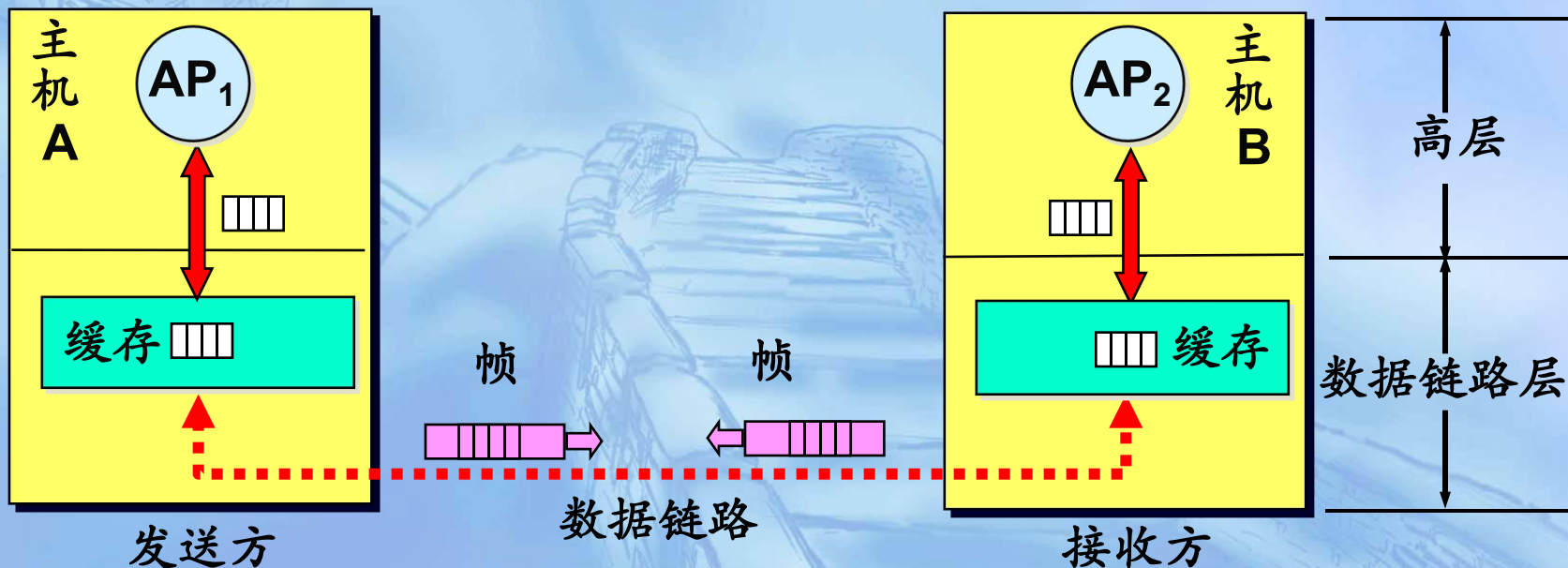
滑动窗口协议

- ✍ 一些约定
- ✍ 滑动窗口协议
- ✍ 一个1bit滑动窗口协议
- ✍ 使用后退n帧的滑动窗口协议
- ✍ 使用选择重发的滑动窗口协议
- ✍ 协议效率分析



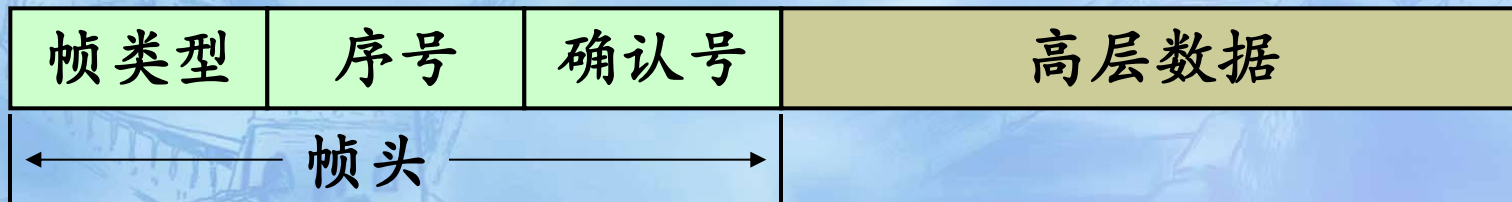
简化的通信模型

- ❌ 假设物理层、数据链路层和网络层都是独立的进程；
- ❌ 假设主机A和主机B通过一个可靠的、面向连接的服务互相发送数据流（**全双工**）；且准备发送的数据**不是无限的**；
- ❌ 假设主机不会崩溃；
- ❌ 硬件计算帧校验和。





帧格式



✍ 帧类型

- ✦ 数据帧data、肯定确认帧ack、否定确认帧nak

✍ 序号

- ✦ 当前发送的数据帧序号

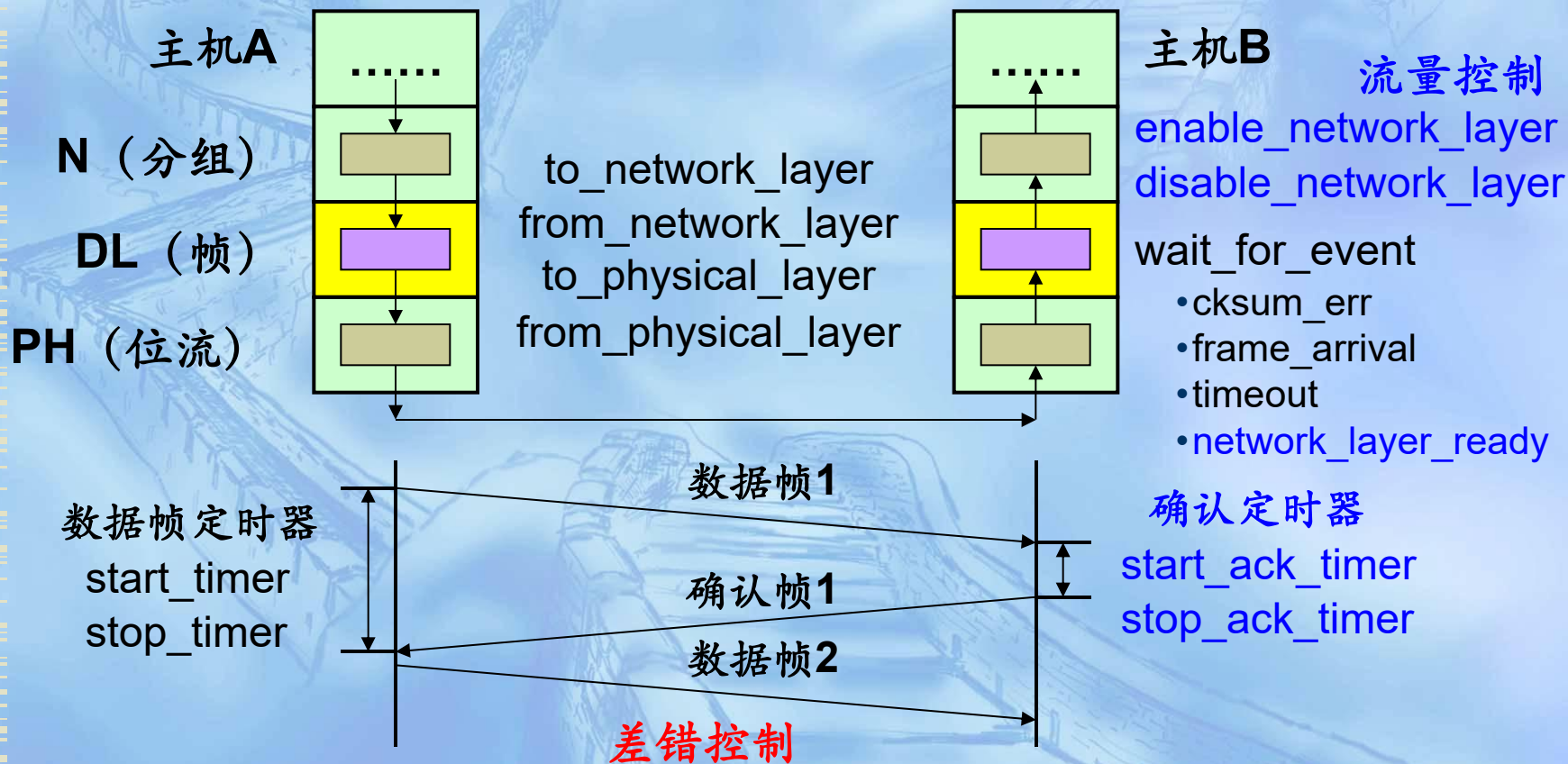
✍ 确认号

- ✦ 确认刚接收成功的数据帧序号（捎带确认）

注：参见教材P174图3-11中的数据结构



函数



注：参见教材P174图3-11中的函数

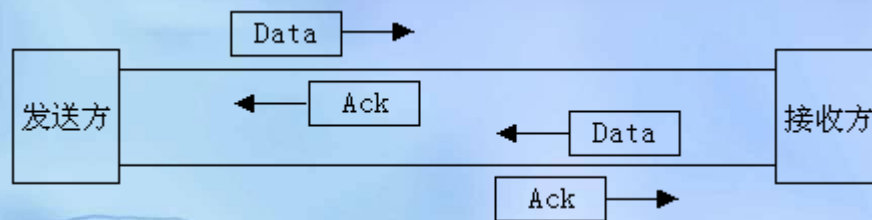


全双工的实现

方法一

- 思想：使用两条独立的信道，每一信道分别用作单工数据传输。

- 问题：浪费严重。



方法二

- 思想：使用同一条线路进行数据的双向传输，使用专门的确认帧。

- 问题：浪费严重。





双工的实现

✎ 方法三（多采用）

✦ **思想**：使用同一条线路进行数据的双向传输，但确认帧采用捎带技术（确认是放入下一帧头部的ack字段）。

✦ **优点**（与使用专门的确认帧相比较）：

✦ 能有效地利用信道带宽

✦ 开销少

✦ 接收方所需的缓冲区少



✦ **问题**：等待时间不可预知（即：不知等多长时间才能得到要捎带确认的数据帧）。

✦ **解决**：引入确认定时器（在计时范围内采用捎带，否则发送专门的确认帧）。



滑动窗口

✍ **作用：** 用于流量控制

✍ **思想：**

- ✧ 发送端设置发送窗口控制发送流量
- ✧ 接收端设置接收窗口控制接收流量
- ✧ 设置原则：发送流量 $\leq$ 接收流量
- ✧ 全双工通信则通信双方都应同时设置发送和接收窗口



发送窗口

✍ 概念

- ✦ 每个发送端设置一个，用于保存待确认帧的序号
(即：对应发送方已发送但未收到确认的数据帧序号，数据必须顺序发送)。初始为0，有最大限制，窗口大小 W_T 是动态改变的。

✍ 滑动方法

- ✦ 每发送一个数据帧，上限向前滑动一步
- ✦ 每接收一个确认帧，下限向前滑动一步/多步
- ✦ 上限和下限之差为最大尺寸时，停止发送



接收窗口

✍ 概念

- ✱ 每个接收端设置一个，用于保存期待接收帧的序号（即：对应接收方允许接收的数据帧序号，可以非顺序接收）。窗口尺寸有最大限制，但窗口大小 W_R 始终是固定的。

✍ 滑动方法

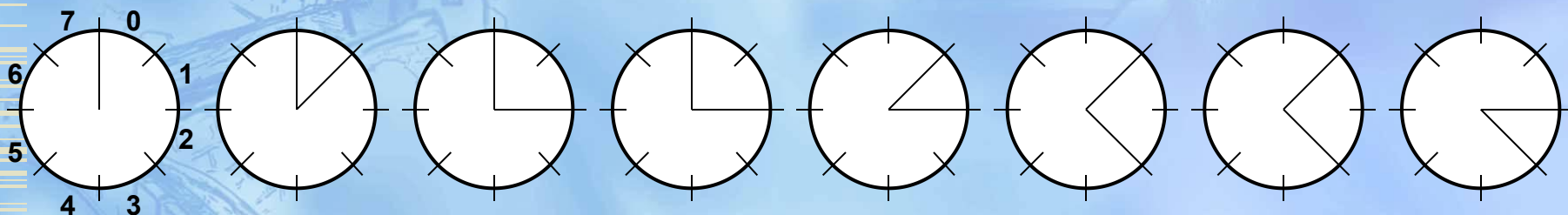
- ✱ 对接收到的数据帧，只有其序号落在接收窗口范围内才接收，否则丢弃；
- ✱ 对正确接收的数据帧，产生一个确认帧，接收窗口向前移动一个窗口大小的位置。



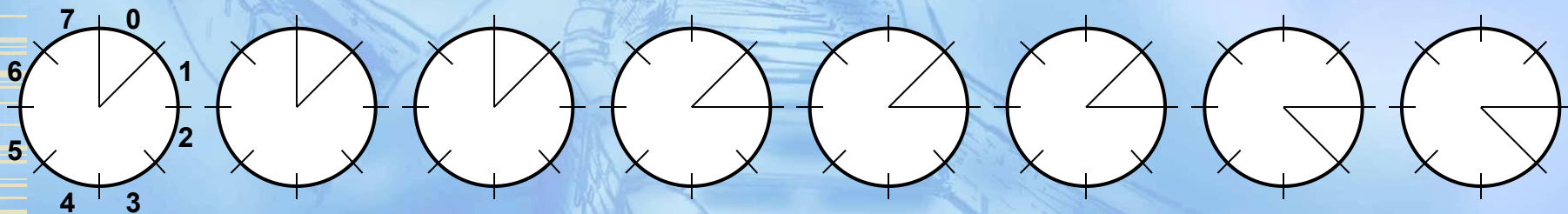
滑动窗口思想举例

✂ 帧序号位数**3bit**，发送窗口大小为**2**，接收窗口大小为**1**

发送端



接收端



- | | | | | | | | |
|----|-------|-------|---------|---------|-------|---------|---------|
| ① | ② | ③ | ④ | ⑤ | ⑥ | ⑦ | ⑧ |
| 初态 | 发送0号帧 | 发送1号帧 | 正确接收0号帧 | 接收确认0号帧 | 发送2号帧 | 正确接收1号帧 | 接收确认1号帧 |



1bit滑动窗口协议

(停-等协议)

✂ 帧序号位数**1bit**，发送窗口大小为**1**，接收窗口大小为**1**

发送端



接收端



①
初态

②
发送0号帧

③
正确接收0
号帧

④
接收确认0
号帧

⑤
发送1号帧



协议4

(sender + receiver)

```
typedef enum {frame_arrival, cksum_err, timeout} event_type;
void protocol4 (void)
{
    seq_nr next_frame_to_send;
    seq_nr frame_expected;
    frame r, s;
    packet buffer;
    event_type event;
    next_frame_to_send = 0;
    frame_expected = 0;
    from_network_layer(&buffer);
    s.info = buffer;
    s.seq = next_frame_to_send;
    s.ack = 1 - frame_expected;
    to_physical_layer(&s);
    start_timer(s.seq);
}
```



协议4

(sender + receiver)

```
while (true)
{
    wait_for_event(&event);
    if (event == frame_arrival)
    {
        from_physical_layer(&r);
        if (r.seq == frame_expected)
        {
            to_network_layer(&r.info);
            inc(frame_expected);
        }
        if (r.ack == next_frame_to_send)
        {
            stop_timer(r.ack);
            from_network_layer(&buffer);
            inc(next_frame_to_send);
        }
    }
    s.info = buffer;
    s.seq = next_frame_to_send;
    s.ack = 1 - frame_expected;
    to_physical_layer(&s);
    start_timer(s.seq);
}
```



协议4分析

✎ 主机A发送、主机B等待，运行正常。

A发送 (0, 1, A0)	Seq ack	A发送A0, seq=0
	B收到 (0, 1, A0) ▲	B收到A0
	B发送 (0, 0, B0)	B发送B0, 并确认A0
A收到 (0, 0, B0) ▲		A收到B0及对A0的确认
A发送 (1, 0, A1)		A发送A1及对B0的确认
	B收到 (1, 0, A1) ▲	B收到A1及对B0的确认
	B发送 (1, 1, B1)	B发送B1及对A1的确认
A收到 (1, 1, B1) ▲		A收到B1及对A1的确认
A发送 (0, 1, A2)		A发送A2及对B1的确认
	B收到 (0, 1, A2) ▲	B收到A2及对B1的确认
	B发送 (0, 0, B2)	B发送B2及对A2的确认
A收到 (0, 0, B2) ▲		A收到B2及对A2的确认
A发送 (1, 0, A3)		A发送A3及对B2的确认
	B收到 (1, 0, A3) ▲	B收到A3及对B2的确认
	B发送 (1, 1, B3)	



协议4分析

✂ 主机A和B同时发送，运行不正常，会出现数据帧多次发送，不会导致协议失败，但会引起效率的降低。

A发送 (0, 1, A0)	B发送 (0, 1, B0)	B也已从网络层得到B0
	B收到 (0, 1, A0) ▲	B以为B0分组A没有收到
	B发送 (0, 0, B0)	所以B又重复B0分组
A收到 (0, 1, B0) ▲		A以为A0分组B没有收到
A发送 (0, 0, A0)		所以A又重复A0分组
	B收到 (0, 0, A0) ▲	B收到A0及对B0的确认
	B发送 (1, 0, B1)	B发送B1及对A0的确认
A收到 (0, 0, B0) ▲		A收到B0及对A0的确认
A发送 (1, 0, A1)		A发送A1及对B0的确认
	B收到 (1, 0, A1) ▲	B以为B1分组A没有收到
	B发送 (1, 1, B1)	所以B又重复B1分组
A收到 (1, 0, B1) ▲		A以为A1分组B没有收到
A发送 (1, 1, A1)		所以A又重复A1分组
	B收到 (1, 1, A1) ▲	
	B发送 (0, 1, B2)	



引入

✂ 协议4存在的问题：链路利用率太低

✂ 问题的解决

✦ 管道化技术：发送方在发送窗口允许的范围内，不必等待确认帧的返回就连续发送多帧（即：发送窗口 >1 ）

✦ 发送窗口 $\geq 2BD+1$ 可使链路利用率达100%

✂ 如何恢复错误？

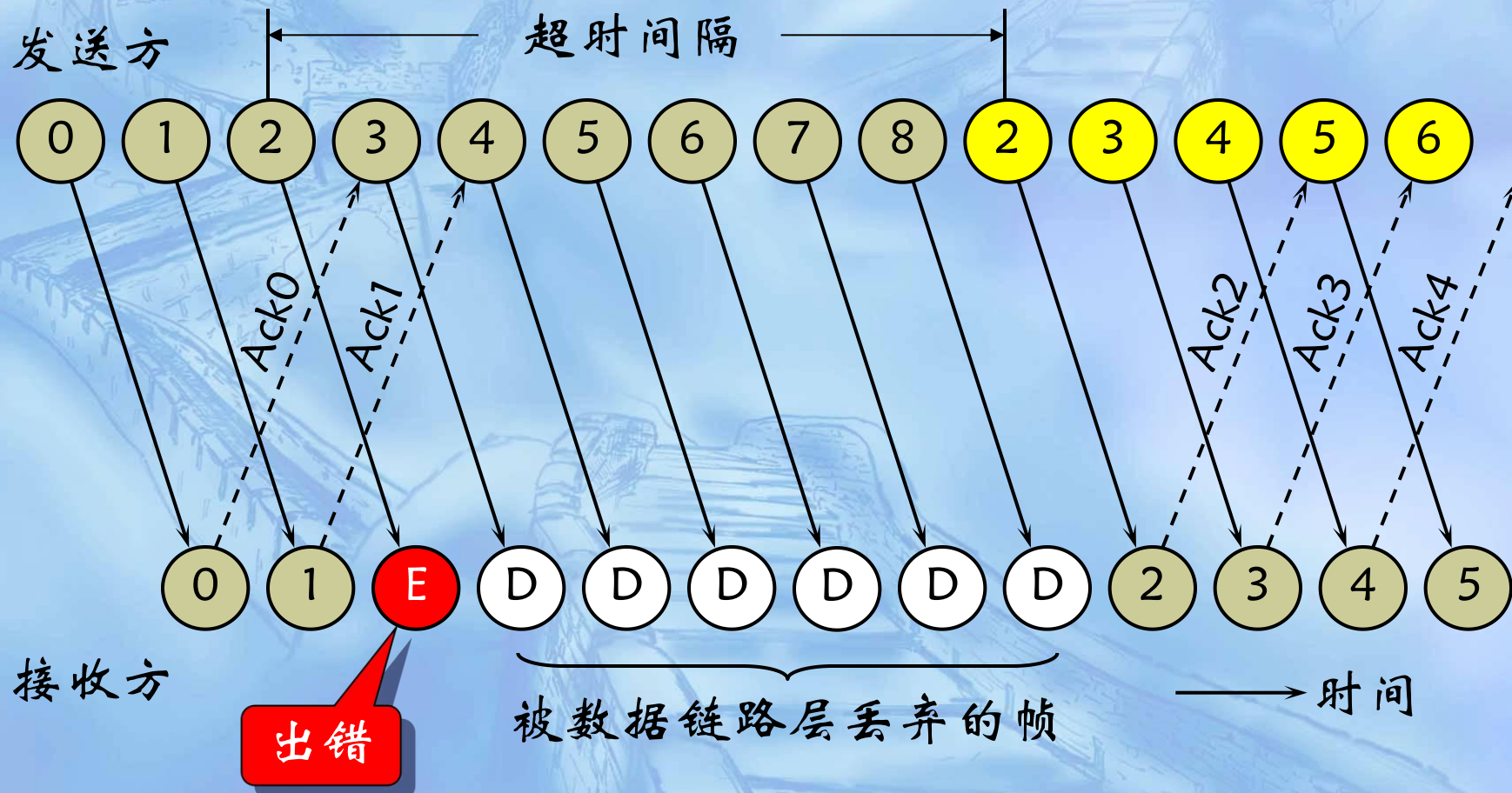
链路的带宽-延迟乘积，单位：帧。
2BD表示在链路上打个来回的时间
可以发送的帧数。

✦ 后退n帧（go back n）

✦ 选择重发（selective repeat）



后退n帧





使用后退n帧的滑动窗口协议 (后退n帧ARQ协议)

✍ 发送窗口 >1 ，接收窗口 $=1$

- 接收方仅接收正确且正期待的帧，并发确认；丢弃错帧或非期待帧及以后各帧。
- 发送方待数据帧定时器超时，重发所有待确认帧，而不仅仅是错帧。

✍ 窗口大小

- 对于帧序号位数为nbit，其发送窗口大小 W_T 必满足 $W_T \leq 2^n - 1$ 。



定理证明

✎ **定理**：对于帧序号位数为 n ，其发送窗口大小 W_T 必满足
 $W_T \leq 2^n - 1$ 。

✎ **证明**：设接收窗口大小为 W_R ，由后退 n 帧协议可知， $W_R = 1$ 。

∵ 帧序号为 n ∴ 帧序号空间是 $\text{mod } 2^n$

显然，在 W_T 范围内序号不应重叠，否则产生二义性。

又∵ 当且仅当对发送窗口上限帧序号的确认帧正在返回途中时，总存在 $W_R + W_T$ 覆盖范围最大，且 $W_R + W_T$ 范围内帧序号不重叠。

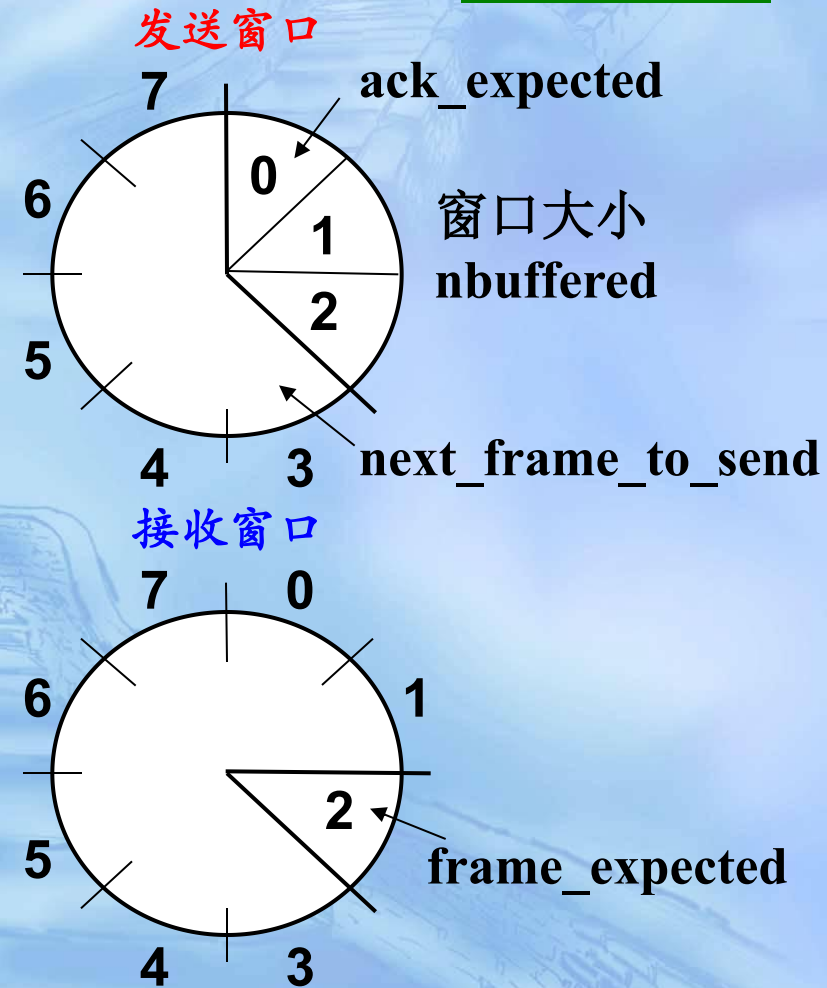
于是 $W_R + W_T \leq 2^n$ ，当 $W_R = 1$ 时，有 $W_T \leq 2^n - 1$ 成立。

<证毕>



协议5 (sender + receiver)

```
void protocol5(void)
{ .....
  enable_network_layer();
  ack_expected = 0;
  next_frame_to_send = 0;
  frame_expected = 0;
  nbuffered = 0;
```





协议5

(续)

```
while (true)
{
    wait_for_event(&event);
    switch(event)
    {
        case network_layer_ready:
            network_layer_ready处理
        case frame_arrival: frame_arrival处理
        case cksum_err: break;
        case timeout: timeout处理
    }
    if (nbuffered < MAX_SEQ)
        enable_network_layer();
    else
        disable_network_layer(); } }
```



network_layer_ready 处理

```
from_network_layer(&buffer[next_frame_to_send]);  
nbuffered = nbuffered + 1;  
send_data(next_frame_to_send,  
           frame_expected, buffer);  
inc(next_frame_to_send);  
break;
```

装配一个数据帧并发送，发送缓冲区数+1，准备发送下一数据帧。



frame_arrival处理

```
from_physical_layer(&r);  
if (r.seq == frame_expected)  
{ to_network_layer(&r.info);  
  inc(frame_expected); }  
while (between(ack_expected, r.ack,  
              next_frame_to_send))  
{ nbuffered = nbuffered - 1;  
  stop_timer(ack_expected);  
  inc(ack_expected); }  
break;
```

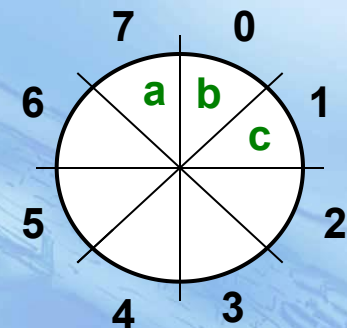
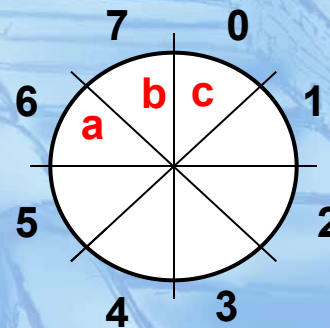
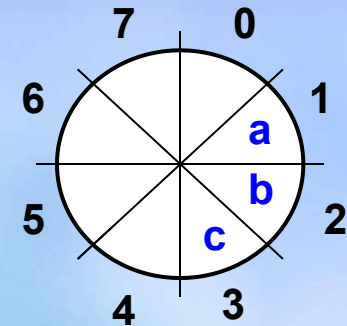
如收到一个数据帧
则交网络层，并期待
接收下一数据帧

如收到一个确认帧
则释放一个缓冲区，
定时器复位，并期待
接收下一确认帧



between函数

```
static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
    /* Return true if a <= b < c circularly */
    if (((a <= b) && (b < c)) ||
        ((c < a) && (a <= b)) ||
        ((b < c) && (c < a)))
        return(true);
    else
        return(false);
}
```





timeout处理

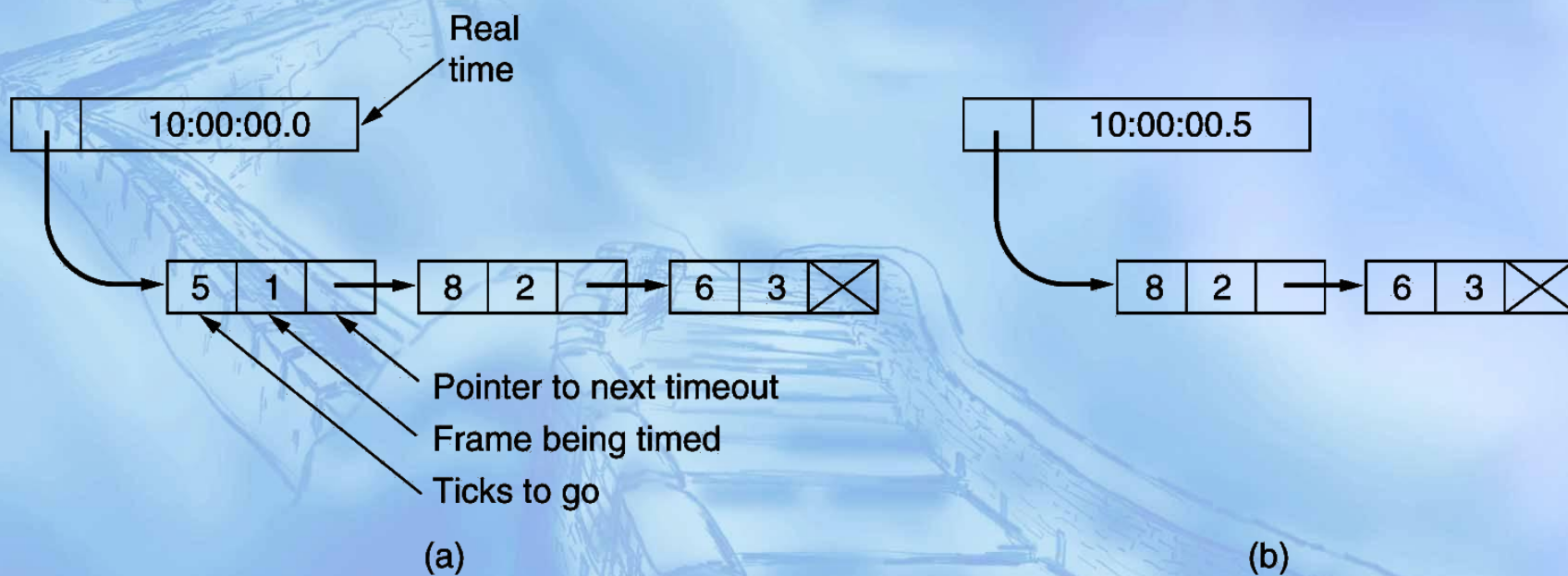
从等待确认的
帧开始全部重发

```
next_frame_to_send = ack_expected;  
for (i = 1; i <= nbuffered; i++)  
{ send_data(next_frame_to_send,  
            frame_expected, buffer);  
  inc(next_frame_to_send); }
```



定时器的实现

用软件来模拟多个定时器

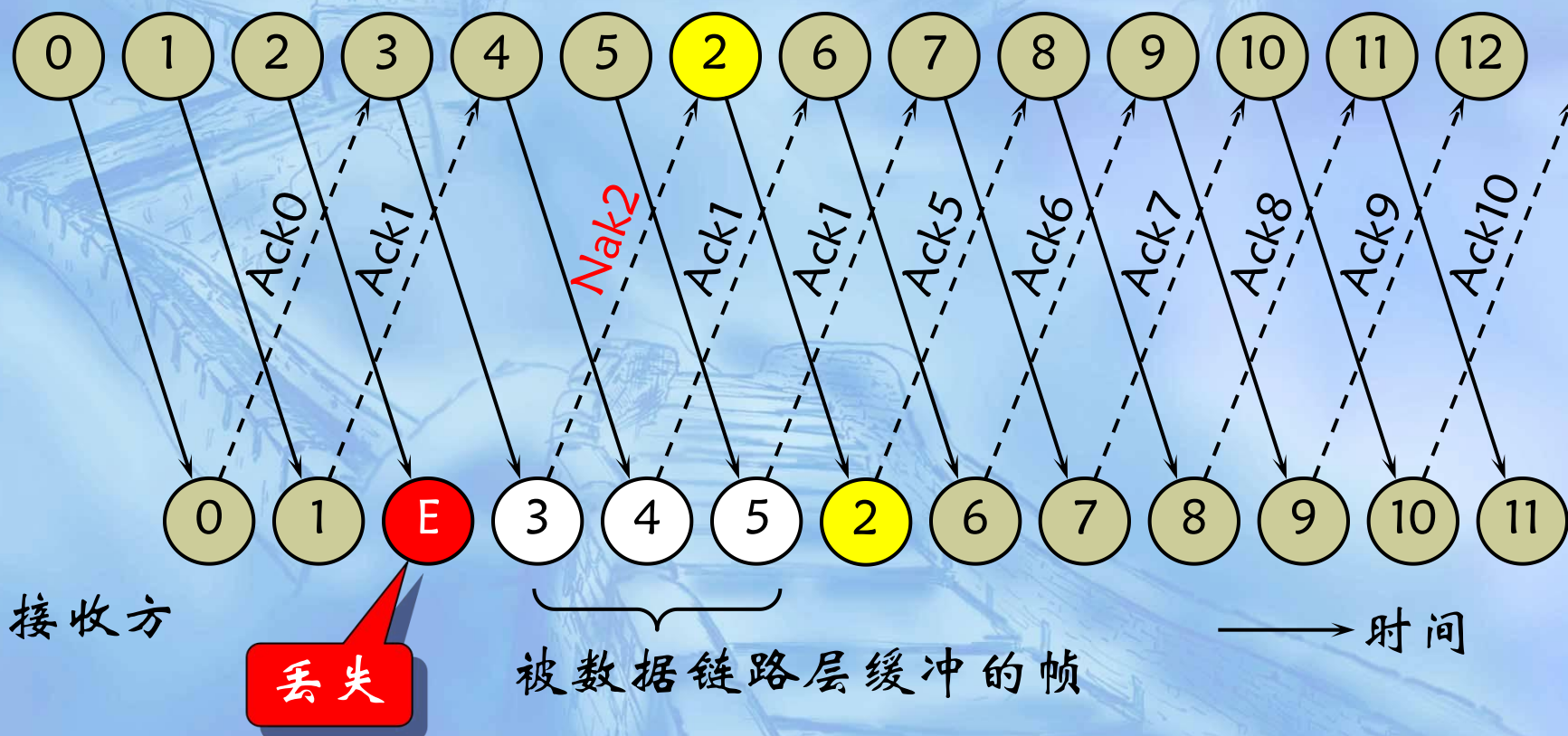




选择重发

本协议引入否定确认帧NAK以提高传输效率

发送方



接收方

丢失

被数据链路层缓冲的帧

时间



使用选择重发的滑动窗口协议

(选择重发ARQ协议)

✎ 发送窗口 >1 ，接收窗口 >1

- ✦ 接收方接收到正确且正期待的帧，上交并发肯定确认帧；接收到出错帧，丢弃并发否定确认帧；接收到正确且非期待的帧，缓存该帧并发确认帧；。
- ✦ 发送方收到否定确认帧或待数据帧定时器超时，只重发错帧。

✎ 窗口大小

- ✦ 对于帧序号位数为 n ，其发送窗口大小 W_T 和接收窗口大小 W_R 必满足 $W_R \leq W_T \leq 2^{n-1}$ ，通常取 $W_R = W_T = 2^{n-1}$ 。



定理证明

✎ **定理**：对于帧序号位数为 n ，其发送窗口大小 W_T 和接收窗口大小 W_R 必满足 $W_R \leq W_T \leq 2^{n-1}$ 。

✎ **证明**： $\because$ 帧序号为 n $\therefore$ 帧序号空间是 $\text{mod } 2^n$

显然，在 W_T 范围内序号不应重叠，否则产生二义性。

又 $\because$ 当且仅当对发送窗口上限帧序号的确认帧正在返回途中时，总存在 $W_R + W_T$ 覆盖范围最大，且 $W_R + W_T$ 范围内帧序号不重叠。

于是 $W_R + W_T \leq 2^n$ ，又 $\because$ $W_R > W_T$ 无意义

$\therefore$ 至多 $W_R = W_T$ ，因此有 $W_R \leq W_T \leq 2^{n-1}$ 成立。

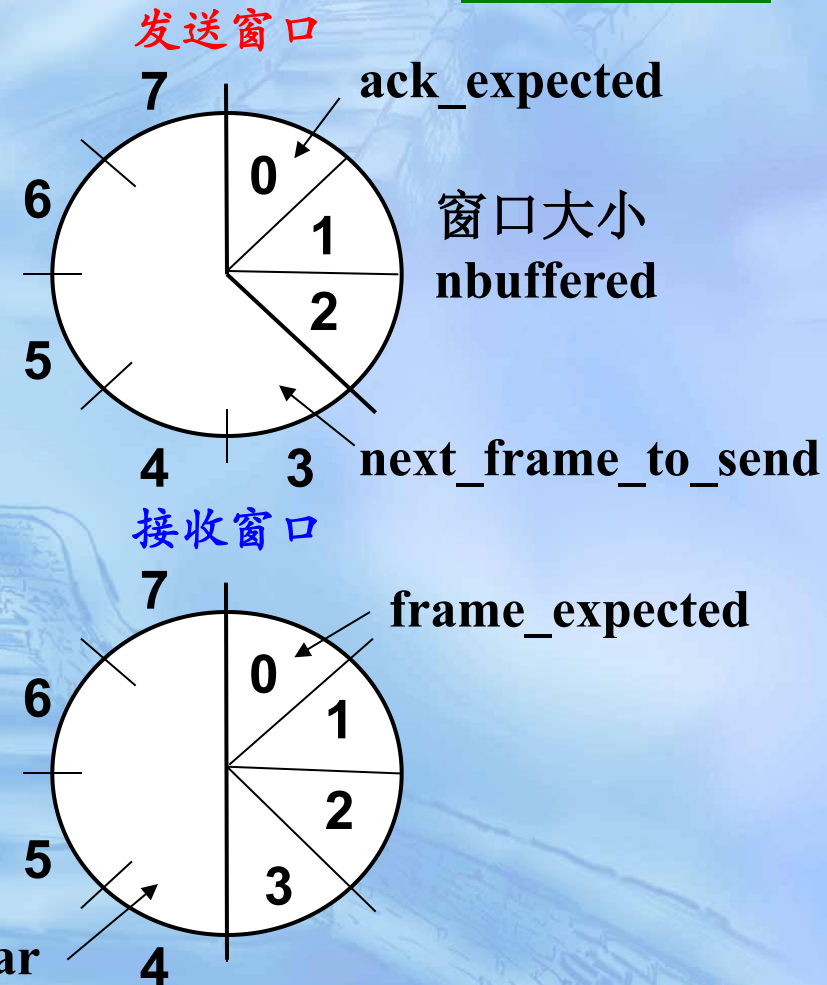
<证毕>



协议6

(sender + receiver)

```
void protocol6(void)
{ .....
  enable_network_layer();
  ack_expected = 0;
  next_frame_to_send = 0;
  frame_expected = 0;
  too_far = NR_BUFS;
  nbuffered = 0;
  for (i=0;i<NR_BUFS;i++)
    arrived[i] = false;
```





协议6

(续)

```
while (true)
{
    wait_for_event(&event);
    switch(event)
    {
        case network_layer_ready: 发送数据处理
        case frame_arrival: 帧到达处理
        case cksum_err: 收到一个坏帧且没有发过nak, 则发nak
            if (no_nak) send_frame(nak, 0, frame_expected, out_buf);
        case timeout: 数据帧超时重发
            send_frame(data, oldest_frame, frame_expected, out_buf);
        case ack_timeout: 确认定时器超时, 发一单独的ack
            send_frame(ack, 0, frame_expected, out_buf);
    }
    if (nbuffered < NR_BUFS)
        enable_network_layer();
    else
        disable_network_layer(); } }
```



network_layer_ready

```
nbuffered = nbuffered + 1;          /* 扩展发送窗口 */  
/* 从网络层取新的分组 */  
from_network_layer(&out_buf  
    [next_frame_to_send % NR_BUFS]);  
/* 发送数据帧 */  
send_frame(data, next_frame_to_send,  
    frame_expected, out_buf);  
inc(next_frame_to_send);          /* 发送窗口的上限+1 */  
break;
```



frame_arrival

```
from_physical_layer(&r);
if (r.kind == data)    处理数据帧
{ if ((r.seq != frame_expected) && no_nak)
    send_frame(nak, 0, frame_expected, out_buf);
  else start_ack_timer(); 如序号错则发nak, 否则启动确认定时器
  if (between(frame_expected, r.seq, too_far) &&
      (arrived[r.seq%NR_BUFS] == false)) 如序号落在接收窗口内则接收
  { arrived[r.seq % NR_BUFS] = true;
    in_buf[r.seq % NR_BUFS] = r.info;
    while (arrived[frame_expected % NR_BUFS])
      如顺序正确, 则交网络层, 调整参数, 启动确认定时器
    { to_network_layer(&in_buf[frame_expected % NR_BUFS]);
      no_nak = true;
      arrived[frame_expected % NR_BUFS] = false;
      inc(frame_expected);
      inc(too_far);
      start_ack_timer(); } } }
```



frame_arrival

(续)

处理nak帧

```
if ((r.kind==nak) && between(ack_expected,  
    (r.ack+1)%(MAX_SEQ+1),next_frame_to_send))  
    send_frame(data, (r.ack+1) % (MAX_SEQ + 1),  
        frame_expected, out_buf);
```

处理ack帧

```
while (between(ack_expected, r.ack,  
    next_frame_to_send))  
{    nbuffered = nbuffered - 1;  
    stop_timer(ack_expected % NR_BUFS);  
    inc(ack_expected);  
}
```



协议6更具有实用性

✎ 协议6中：

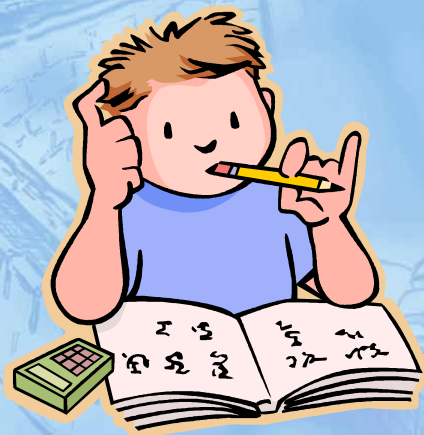
发送窗口=接收窗口= $(MAX_SEQ+1) / 2$

✎ 协议6中：增加了**否定确认nak**，当收到一个错帧，或收到一个非期望的帧，则发送nak帧。

✎ 协议6中：增加了一个**确认定时器**，当收到一个正确的帧，而反向没有可捎带确认的数据帧，则启动确认定时器等待；如果在等待时间内来了数据帧则捎带确认；如果等待超时了还没有等到数据帧，则立即发送一个单独的ack帧。



协议效率分析



- ✍ 画出协议4、5和6的程序流程图；
- ✍ 分析协议4、5和6的效率。

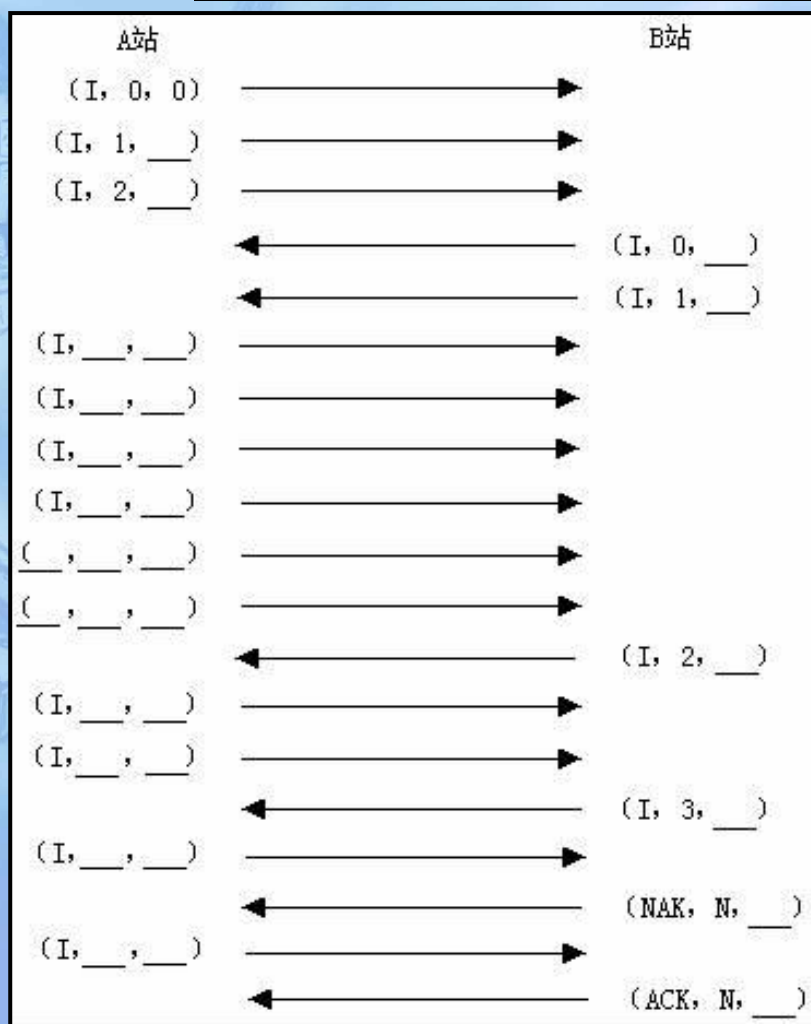


课堂练习

假设**A**站和**B**站之间的半双工数据帧传输使用滑动窗口进行流量控制和差错控制，帧序号位数为3，设**A**站有10个数据帧要发送，**B**站有4个数据帧要发送，使用选择重传**ARQ**协议，帧的确认尽量使用捎带确认和累计确认，若没有数据帧，可用**ACK**进行单独确认，用**NAK**进行单独否认。假定没有超时和帧丢失，发送窗口和接收窗口均从序号0开始。帧的格式为：（帧类型，发送序号，确认序号）。发送序号或确认序号如果没有意义，可用**N**表示；确认序号指出下一个希望接收的数据帧序号。请在后图所示的情况中填写带下划线的空白处（若没有帧，则帧类型为**NONE**）。



课堂练习





协议描述和验证

目的

- ✦ 由于实际使用的协议非常复杂，需有形式化的和数学的方法来验证协议的正确性。

协议描述

- ✦ 用描述语言描述协议实体的信息交互规则、格式和相关定义。

协议验证

- ✦ 用数学方法证实协议描述不存在逻辑错误（例如，死锁）。



两种基本方法

 有限状态机 (FSM)

 Petri网



引 入

现象

- 自然界中我们认为连续或模拟的东西，实际上，大多是一些单独或离散的事件的集合，例如：电影。

引出

- 一个协议（算法）可看成一系列的状态，每个状态全部/部分地由那个时刻程序变量的值来定义。



FSM构成的要素

协议机

- ✦ 通信的一方称为协议机
- ✦ 协议机任何时刻总处在某一特定状态

状态

- ✦ 协议机状态
- ✦ 信道状态

状态变迁

- ✦ 变迁是引起一种状态变迁到另一种状态的事件
- ✦ 系统存在一个初始状态



协议的四元组模型

FSM可将协议表示成形式化的四元组模型：

<S, M, I, T>

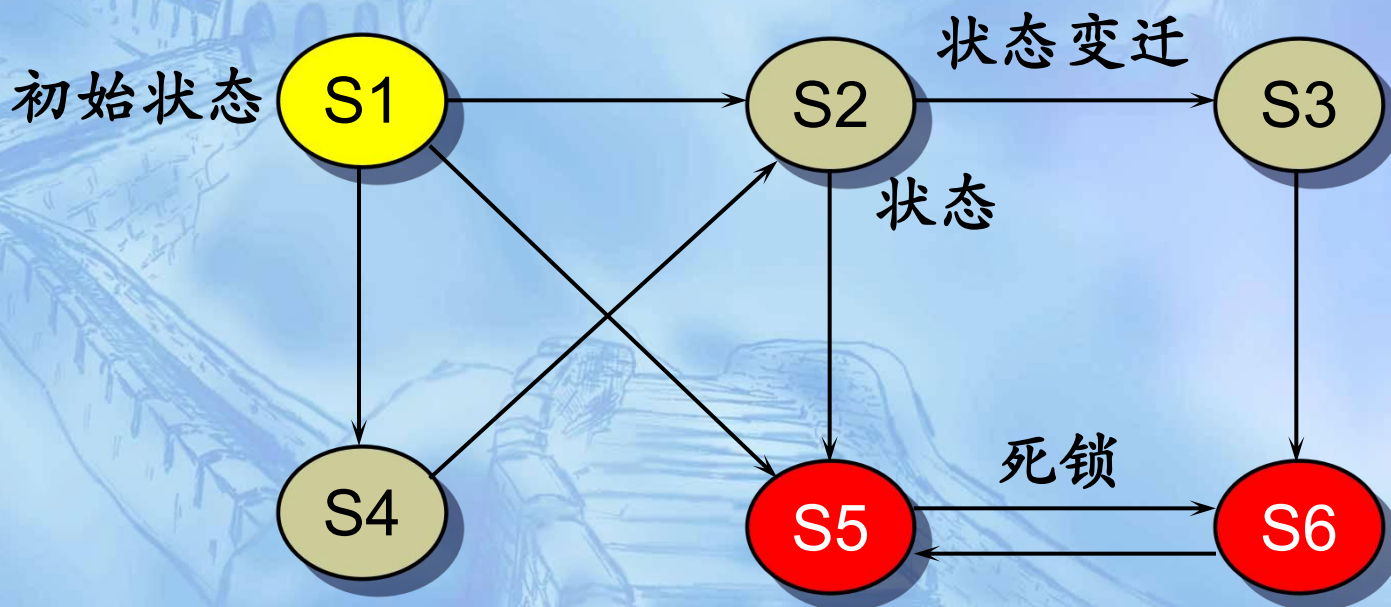
其中：

-  **S**：是进程和信道可能进入状态的集合
-  **M**：是能在信道上进行交换的帧的集合
-  **I**：是进程初始状态的集合
-  **T**：是状态之间进行变迁的集合



有限状态机的描述和验证

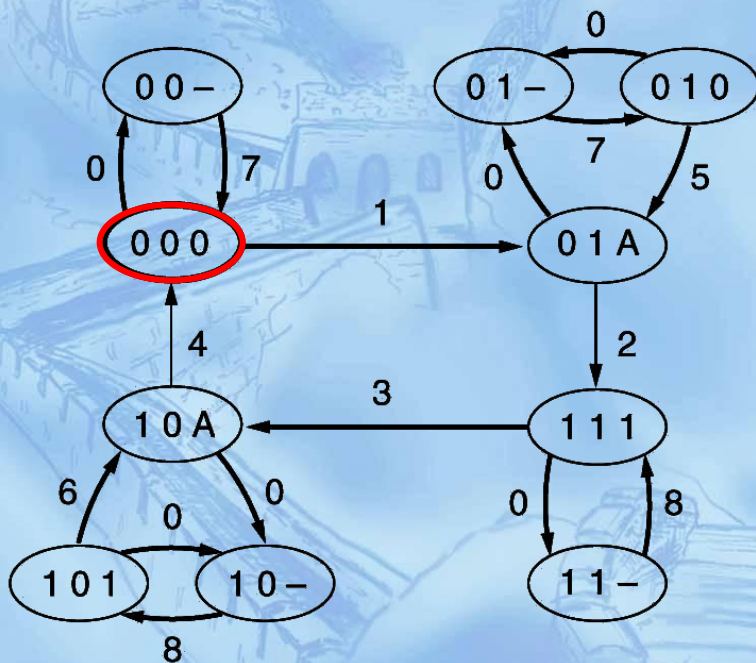
✎ **描述**：状态图法、矩阵法和列表法。



✎ **验证**：可达性分析（利用图论的可传递闭包计算，发现协议中可能的潜在错误，如状态遗漏、死锁等）



例：半双工停-等协议分析



(a)

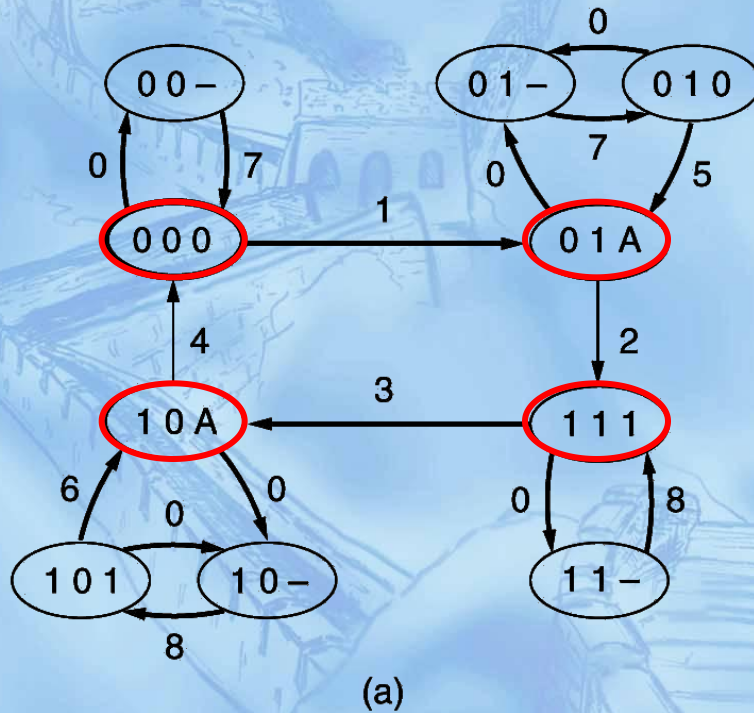
Transition	Who runs?	Frame accepted	Frame emitted	To network layer
0	—	(frame lost)		—
1	R	0	A	Yes
2	S	A	1	—
3	R	1	A	Yes
4	S	A	0	—
5	R	0	A	No
6	R	1	A	No
7	S	(timeout)	0	—
8	S	(timeout)	1	—

(b)

✎ **三元组状态变量**：发送方状态（发送的帧号）、接收方状态（希望接收的帧号）、信道状态；其中，发送方和接收方状态都有两种状态：帧0、帧1；信道有四种状态：帧0、帧1、Ack帧、空信道。



例：半双工停-等协议分析



Transition	Who runs?	Frame accepted	Frame emitted	To network layer
0	—	(frame lost)		—
1	R	0	A	Yes
2	S	A	1	—
3	R	1	A	Yes
4	S	A	0	—
5	R	0	A	No
6	R	1	A	No
7	S	(timeout)	0	—
8	S	(timeout)	1	—

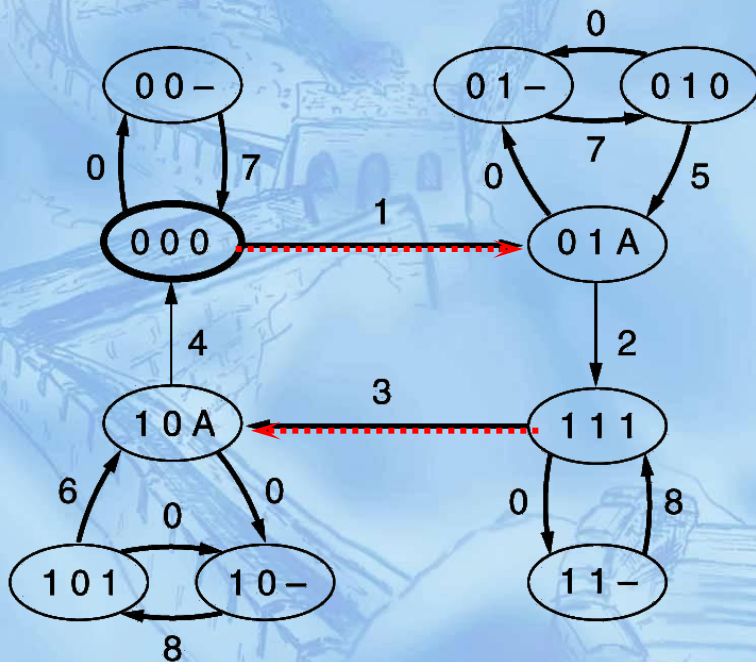
(b)

✍ **主变迁路径：** 000→01A→111→10A→000→...

✍ **无死锁**



例：半双工停-等协议分析



(a)

Transition	Who runs?	Frame accepted	Frame emitted	To network layer
0	—	(frame lost)		—
1	R	0	A	Yes
2	S	A	1	—
3	R	1	A	Yes
4	S	A	0	—
5	R	0	A	No
6	R	1	A	No
7	S	(timeout)	0	—
8	S	(timeout)	1	—

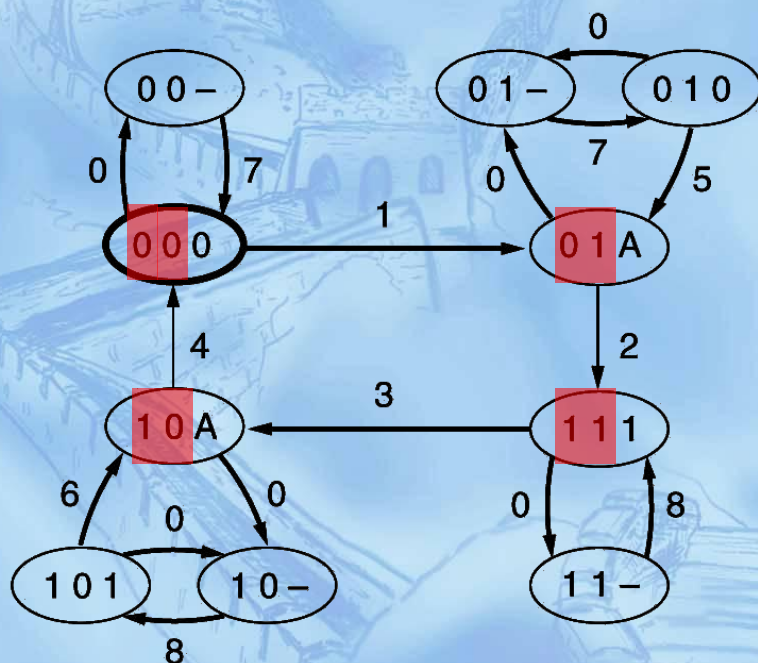
(b)

✍ **转换1（收帧0，发ACK0）和转换3（收帧1，发ACK1）交替进行**

即，无论什么事件发生，接收方都不可能交付两个奇数序列号（1）的帧，而中间没有偶数序列号的帧（0）；反之亦然。



例：半双工停-等协议分析



(a)

Transition	Who runs?	Frame accepted	Frame emitted	To network layer
0	—	(frame lost)		—
1	R	0	A	Yes
2	S	A	1	—
3	R	1	A	Yes
4	S	A	0	—
5	R	0	A	No
6	R	1	A	No
7	S	(timeout)	0	—
8	S	(timeout)	1	—

(b)

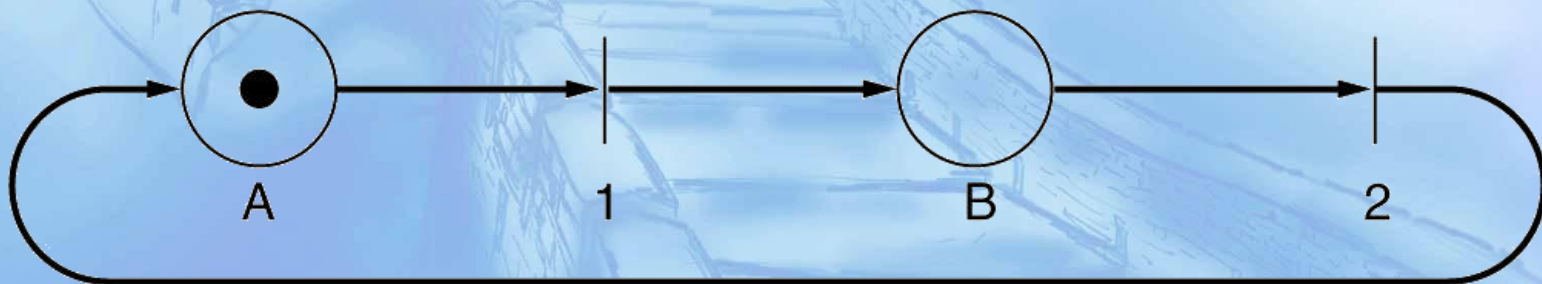
✍ 发送方主状态与接收方主状态交替进行

即，不会连续丢失两个数据帧。



Petri网的基本元素

- ✎ **位置**：用圆圈表示，表示系统可进入的状态。
- ✎ **标记**：用位置中的小黑点表示，表示系统当前所处状态。
- ✎ **变迁**：用水平线或垂直线表示，表示引起系统从一个位置变化到另一个的事件。
- ✎ **弧**：在变迁的两侧，连接着一个位置和变迁，可分为输入弧和输出弧。





状态变迁

✎ 状态变迁的条件

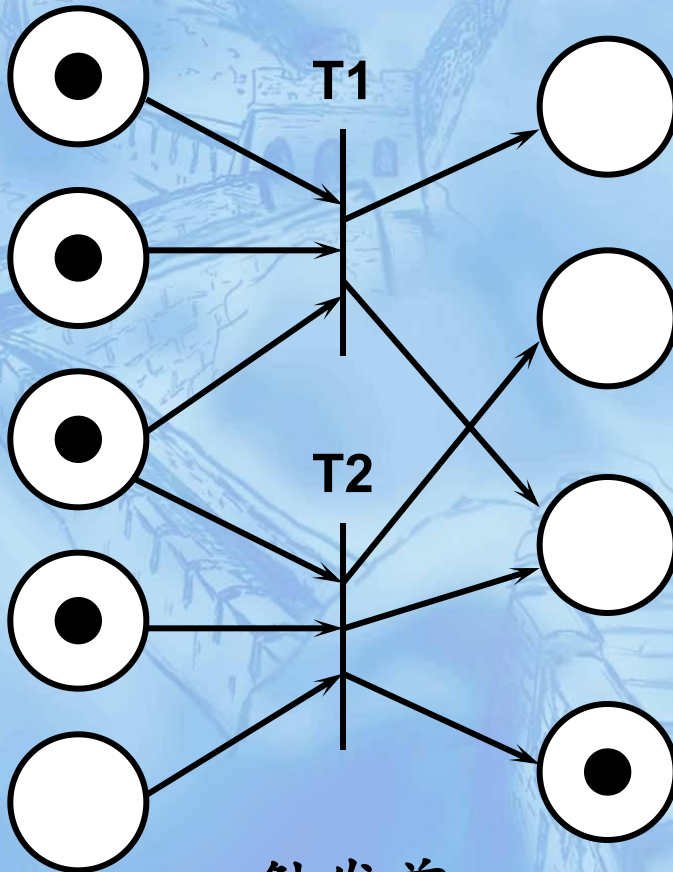
- ✦ **变迁必须就绪**：就绪指某变迁的所有输入位置中都有标记，且当输入位置有多条弧指向该变迁时，该输入位置至少应有与弧线条数相等的标记数。
- ✦ **必须发生触发**：触发指使就绪变迁引起变迁。对于多个就绪变迁，其中任何一个都可能发生触发。这种不确定性正是**Petri网**得以应用的缘由。

✎ 标记移动规则

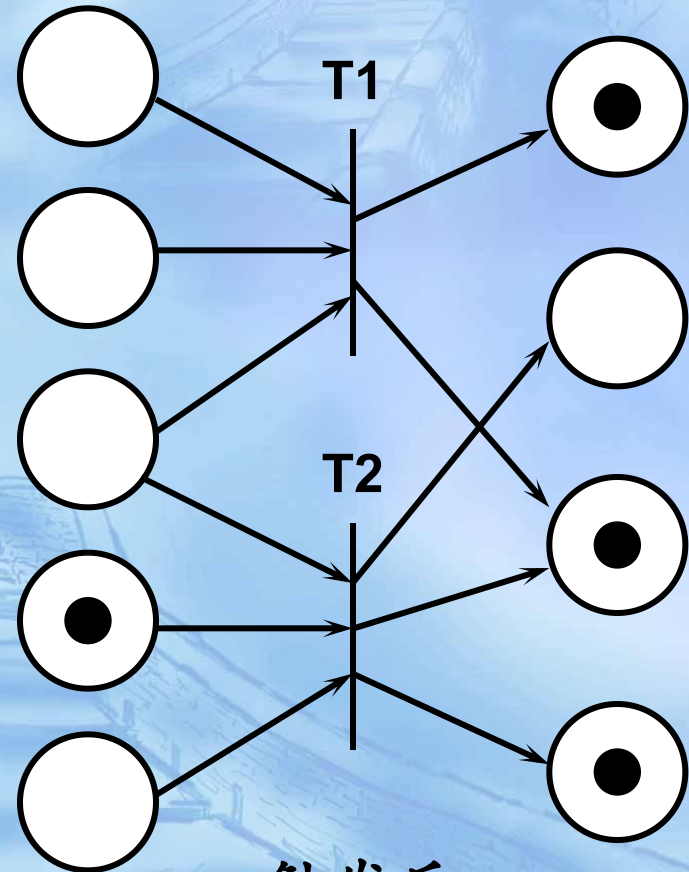
- ✦ 从触发变迁每条输入弧的输入位置取出一个标志，放入该变迁每条输出弧的输出位置中。注意：**状态变迁过程中标记数的不守恒**。



状态变迁举例



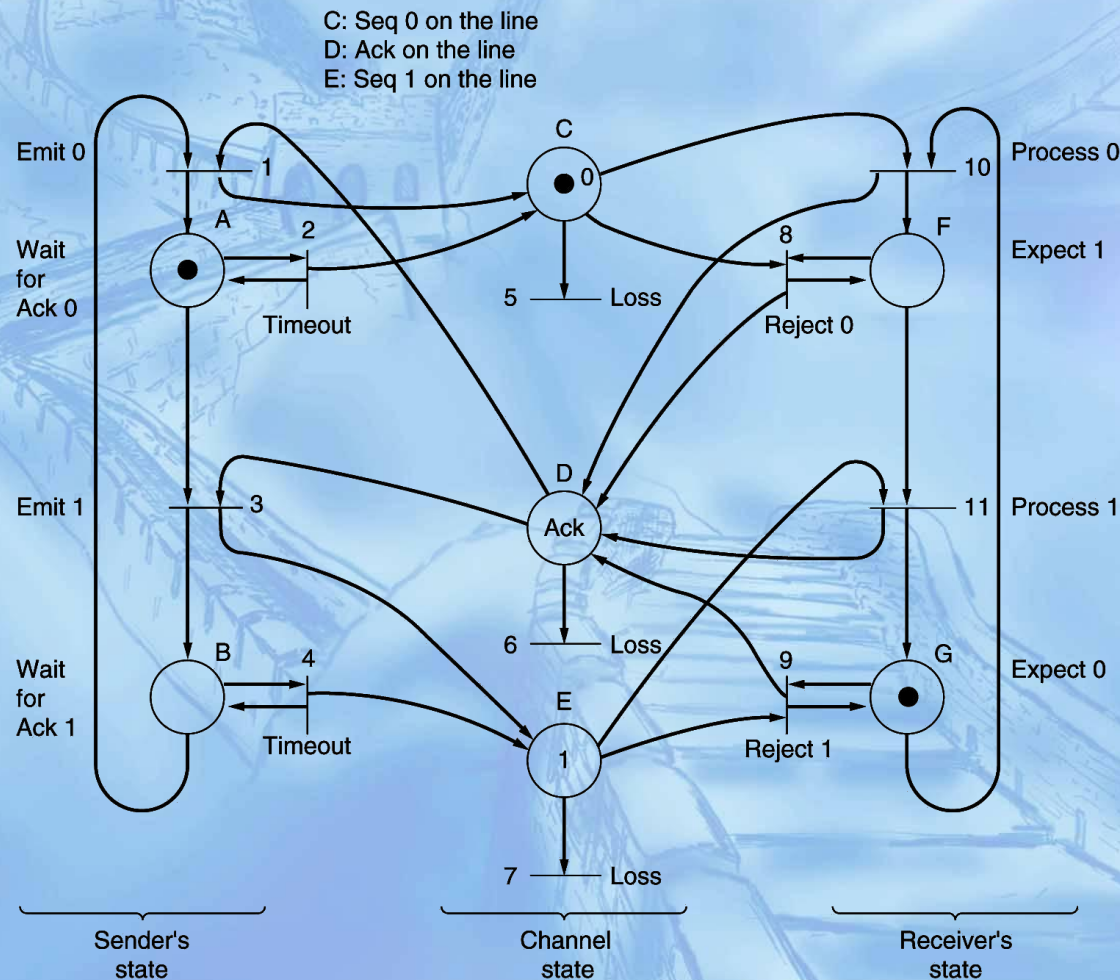
触发前



触发后



例：半双工停-等协议分析



- ✓ 在有限状态机中，用一个点表示一个确定的状态；而在Petri网中，需用几个位置才能表示一个完整的状态。
- ✓ 协议验证方法类似有限状态机。



Petri网表示法

图形表示法

 见上例

代数表示法

 上例有11个转换对应11条规则：

 1: $BD \rightarrow AC$

 7: $E \rightarrow$

 2: $A \rightarrow AC$

 8: $CF \rightarrow DF$

 3: $AD \rightarrow BE$

 9: $EG \rightarrow DG$

 4: $B \rightarrow BE$

 10: $CG \rightarrow DF$

 5: $C \rightarrow$

 11: $EF \rightarrow DG$

 6: $D \rightarrow$



数据链路层协议举例

~~✍~~ HDLC

~~✍~~ PPP



HDLC

(High-level Data Link Control)

- ✎ 最早由IBM提出**SDLC** (Synchronous Data Link Control)。
- ✎ ISO根据SDLC提出了**HDLC** (High-level Data Link Control)，作为国际标准**ISO 3309**。
- ✎ CCITT将HDLC修改后称为**LAP(Link Access Procedure)**。HDLC 的新版本又把 LAP 修改为 **LAPB**，“B”表示平衡型(Balanced)。
- ✎ 我国的相应国家标准是**GB 7496**。
- ✎ **协议特点**：实现可靠传输（现已很少使用）。



HDLC

 基本概念

 数据帧

 工作过程



基本概念

✍ 站点类型

✍ 链路类型

✍ 传输模式



站点类型

主站

- 负责链路控制，包括系统初启、组织传输数据、流量控制、差错控制等。

从站

- 在主站的控制下进行操作。主站通过命令帧控制从站，而从站通过响应帧接收控制。

组合站

- 兼具主站和从站的功能（既可发送命令帧，又可发送响应帧）。



链路类型

✎ **非平衡组合**：由一个主站或一个/多个从站组成，支持全双工/半双工。



a. Point-to-point



b. Multipoint

✎ **平衡组合**：由两个组合站组成，支持全双工和半双工。





传输模式

✍ 正常响应方式 (NRM) : 用于非平衡组合

- ✦ 由主站发送命令对从站的数据传送进行初始化, 从站发送数据作为对该命令的响应。

✍ 异步响应方式 (ARM) : 用于非平衡组合

- ✦ 从站不必得到主站的允许即可进行初始化传输, 但主站仍然负责链路的初始化、纠错和断开逻辑链路等操作。ARM用的最少。

✍ 异步平衡方式 (ABM) : 用于平衡组合

- ✦ 允许任何组合站启动传输。ABM最常用。



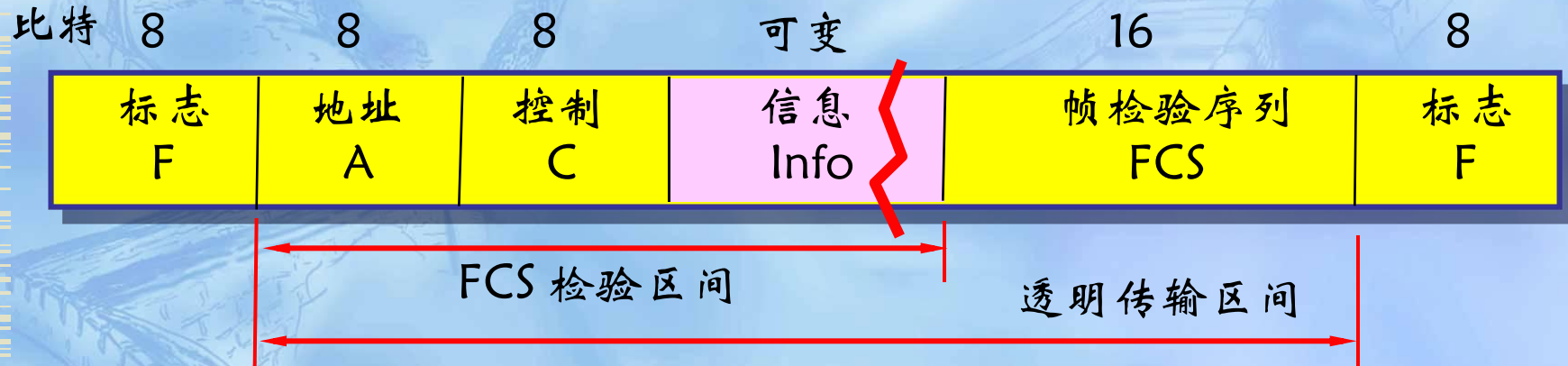
数据帧

✍ 帧格式

✍ 帧类型



HDLC的帧结构



✎ **标志字段 (F) : 1B (01111110)**

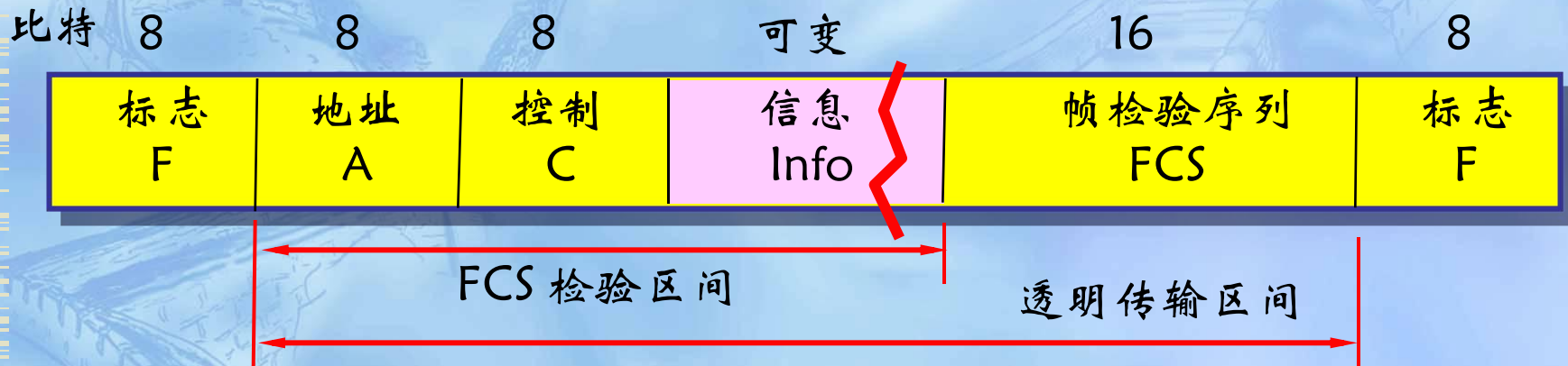
- 作用：标志一帧的开始和结束；帧间的同步信号。为实现数据透明性传输，采用“**0比特插入/删除法**”。

✎ **地址字段 (A) : 1B**

- 用于**指明从站地址**（命令帧：接收帧的从站地址；响应帧：发出帧的从站地址）。



HDLC的帧结构



✎ 控制字段 (C) : 1B

- ✎ 用于HDLC帧的分类：信息帧 (I帧)、监控帧 (S帧) 和无编号帧 (U帧)。不同类型的帧其控制字段格式不同，后面介绍。

✎ 信息字段 (I)

- ✎ 用于存放用户数据，可有可无，长度未作限制（下限为0，上限由FCS字段或缓冲器容量决定）。

✎ 帧校验序列 (FCS) : 2B，生成多项式采用CRC-CCITT



帧类型

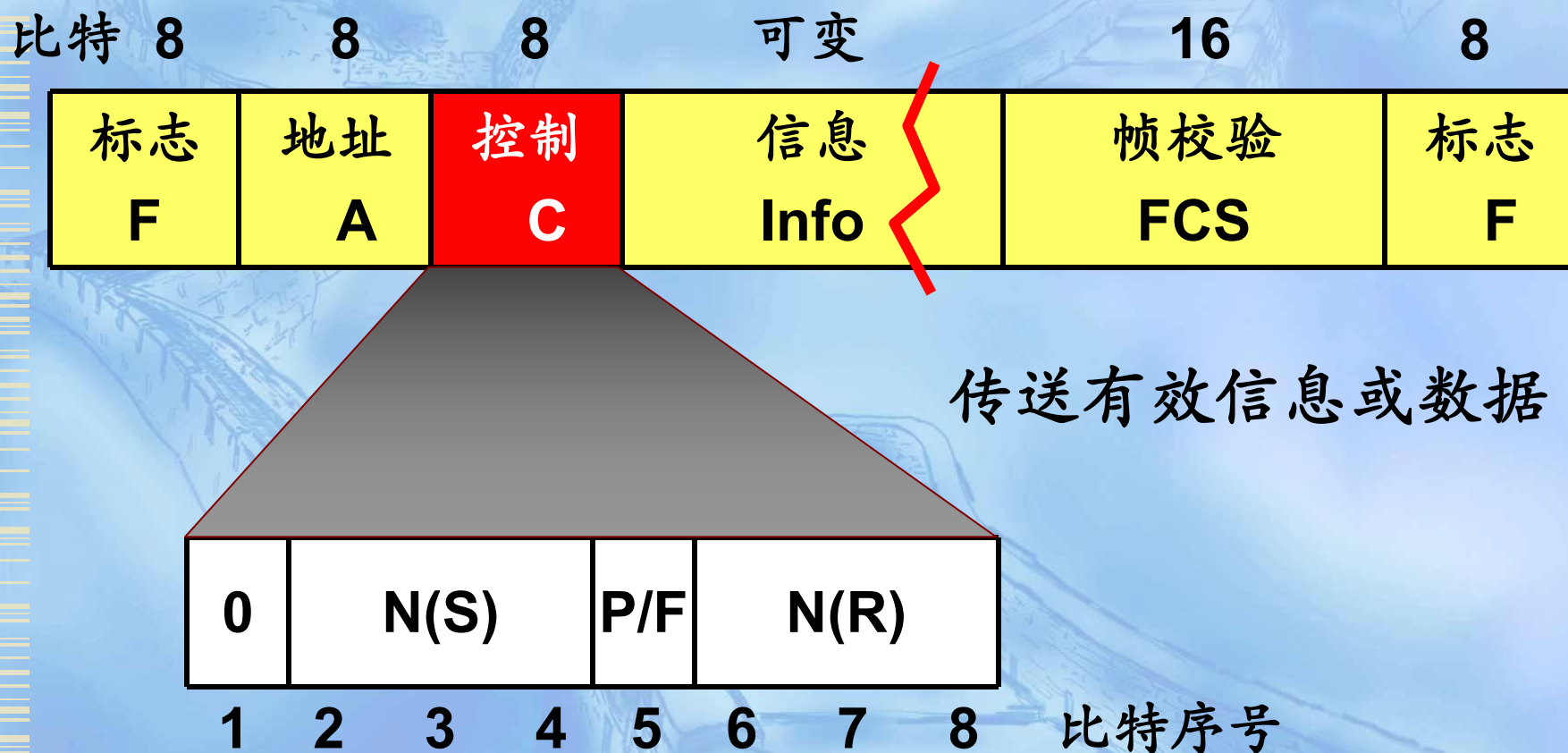
 信息帧 (I帧)

 监控帧 (S帧)

 无编号帧 (U帧)



信息帧 (I帧)





控制字段的格式

N(S)

- ✦ 发送方当前发送的帧编号。

N(R)

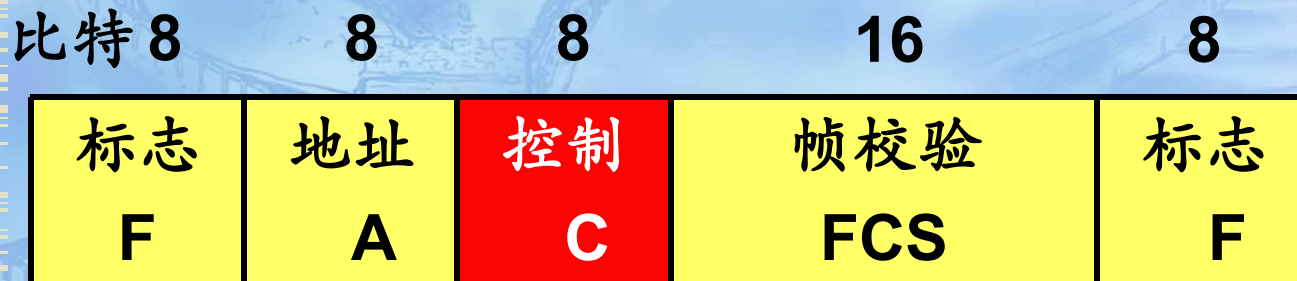
- ✦ 接收方希望接收的帧编号（捎带确认）。

P/F (Poll/Final)

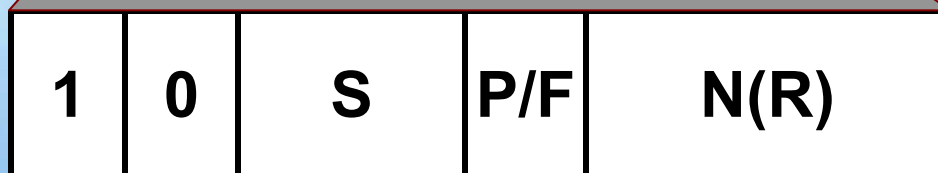
- ✦ 主站发出的命令帧中若将P比特置为1则表示要求对方立即发送响应。
- ✦ 从站发出的响应帧中若将F比特置为1则表示要发送的数据已经发送完毕。



监控帧 (S帧)



用于差错控制和流量控制



1 2 3 4 5 6 7 8 比特序号



控制字段的格式

✍ **S**

- ✦ 类型，用于对各类监控的区分。

✍ **N(R)**

- ✦ 接收方希望接收的帧编号（捎带确认）。

✍ **P/F (Poll/Final)**

- ✦ 主站发出的命令帧中若将P比特置为1则表示要求对方立即发送响应。
- ✦ 从站发出的响应帧中若将F比特置为1则表示要发送的数据已经发送完毕。

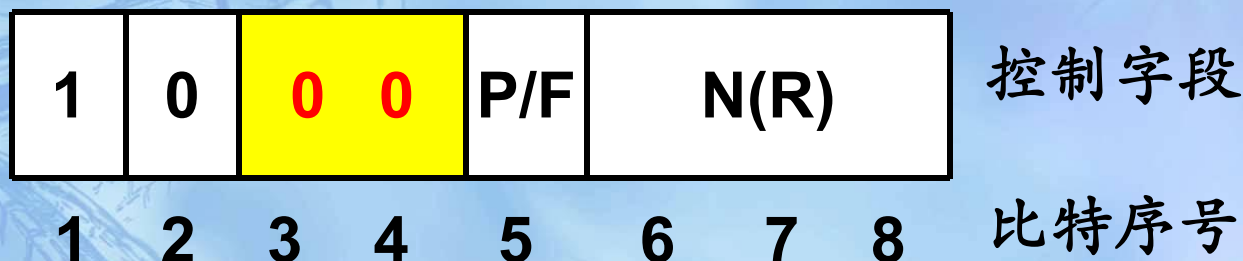


监控帧的类型

流量控制

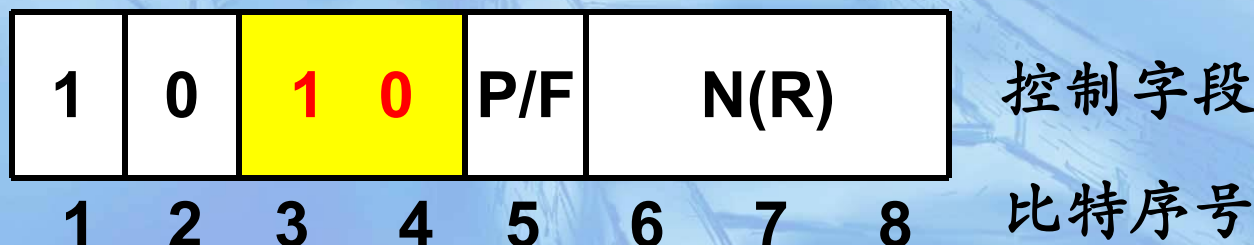
✎ 接收准备好 (RR: Receive Ready)

✦ 准备收N(R)帧，并确认序号为N(R)-1及其前的各帧。



✎ 接收未准备好 (RNR: Receive Not Ready)

✦ 暂停接收N(R)帧，并确认序号为N(R)-1及其以前的各帧。



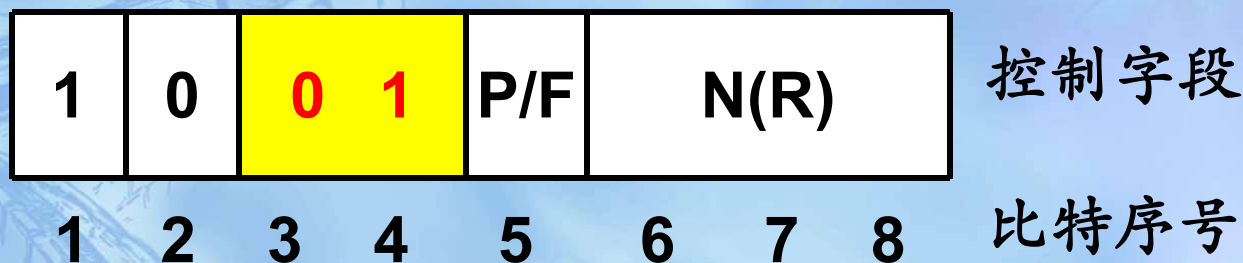


监控帧的类型

差错控制

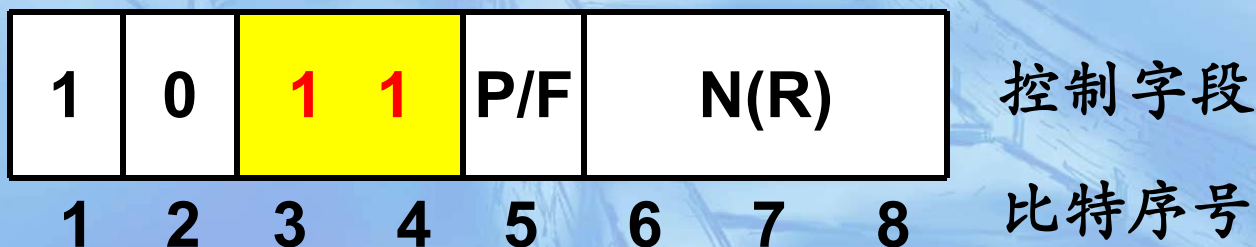
✎ 拒绝 (REJ: REject)

✦ 要求重发N(R)开始的所有帧，属退后n帧策略。



✎ 选择拒绝 (SREJ: Selective Reject)

✦ 仅要求重发N(R)的单独帧，属选择重传策略。

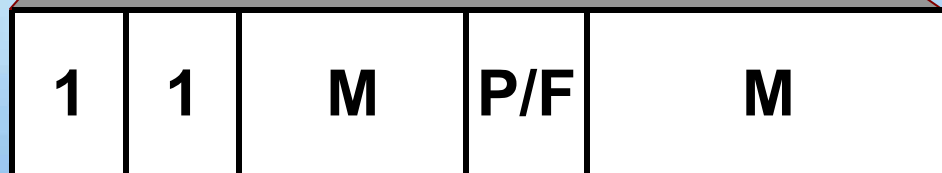




无编号帧 (U帧)



提供对链路的建立、拆除以及多种控制功能。



1 2 3 4 5 6 7 8 比特序号



控制字段的格式

P/F (Poll/Final)

- 主站发出的命令帧中若将P比特置为1则表示要求对方立即发送响应。
- 从站发出的响应帧中若将F比特置为1则表示要发送的数据已经发送完毕。

M

- 类型，用于对各类无编号帧的区分，具体见后表。

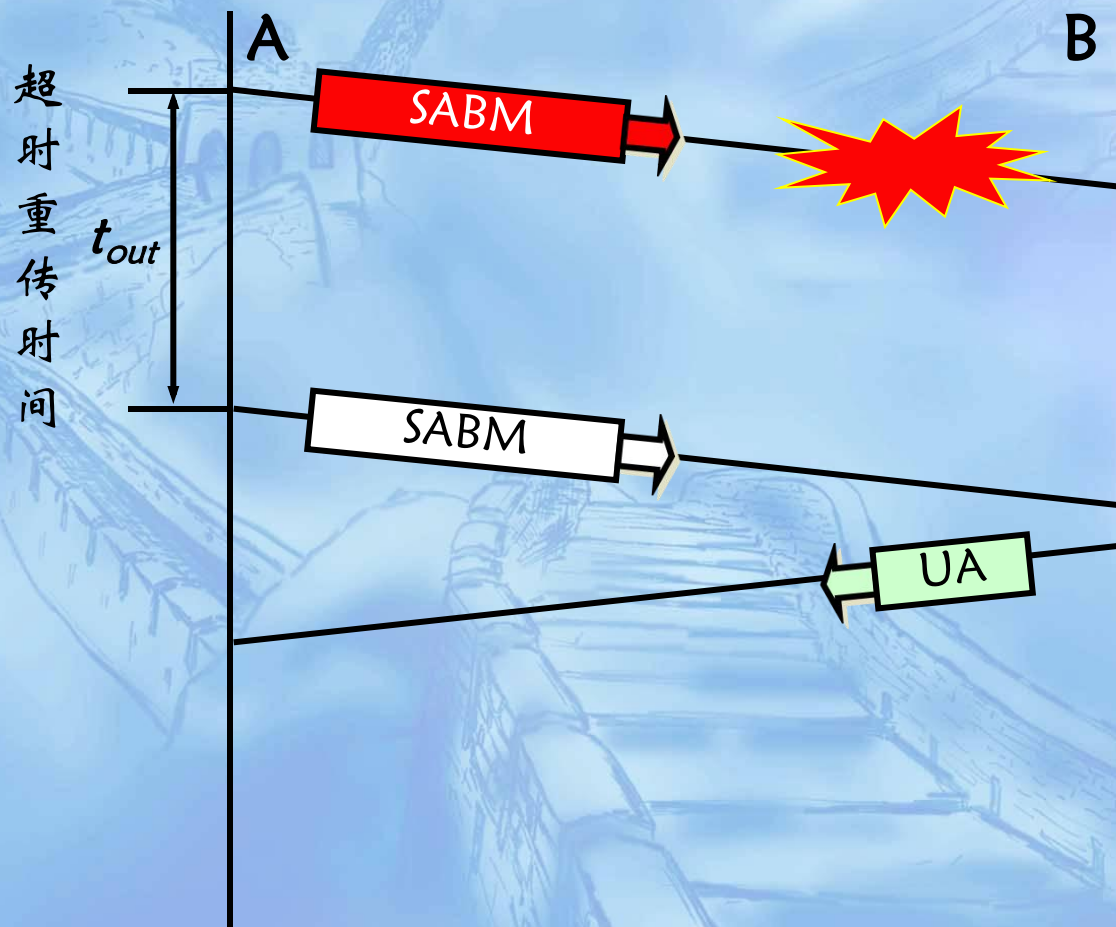


U帧的类型

Command/response	Meaning
SNRM	Set normal response mode
SNRME	Set normal response mode (extended)
SABM	Set asynchronous balanced mode
SABME	Set asynchronous balanced mode (extended)
UP	Unnumbered poll
UI	Unnumbered information
UA	Unnumbered acknowledgment
RD	Request disconnect
DISC	Disconnect
DM	Disconnect mode
RIM	Request information mode
SIM	Set initialization mode
RSET	Reset
XID	Exchange ID
FRMR	Frame reject

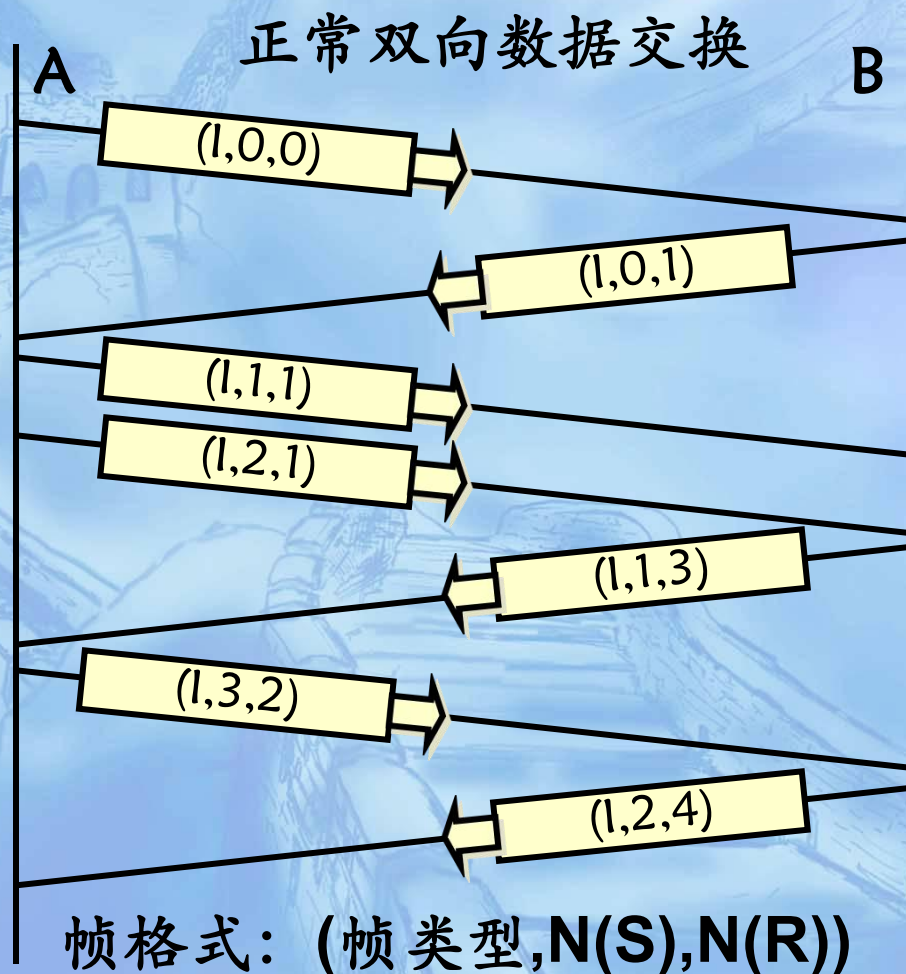


步骤一：初始化



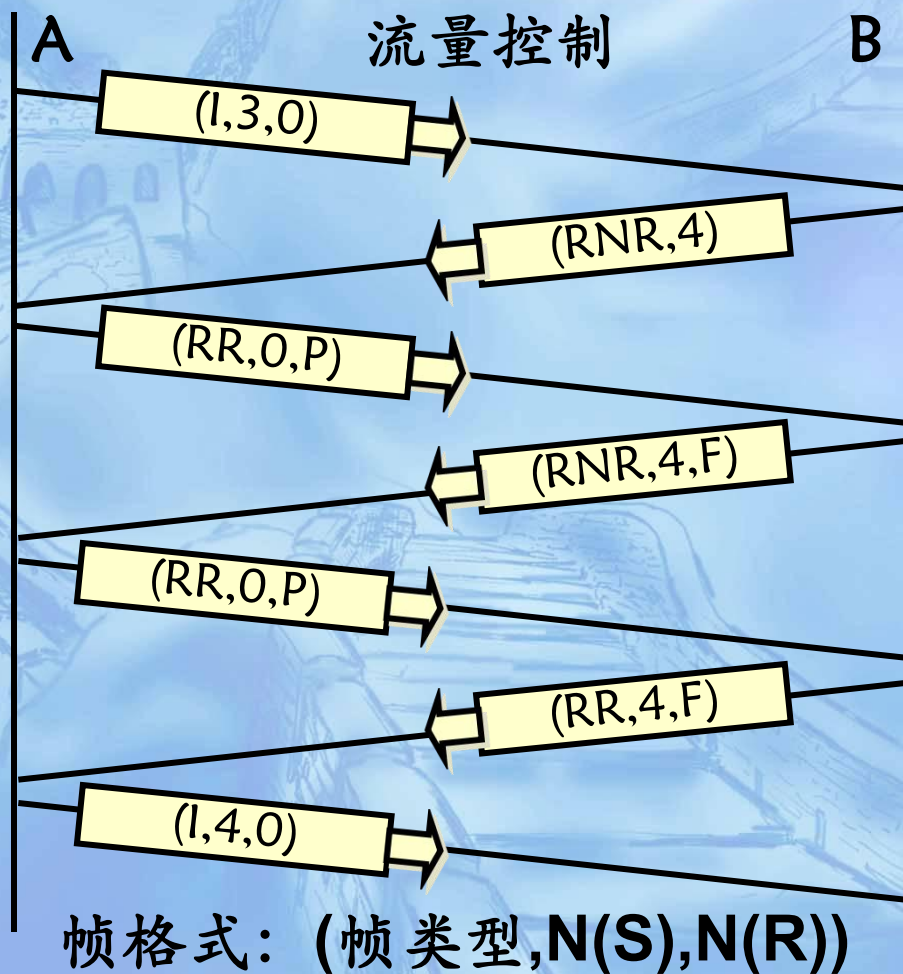


步骤二：数据传输 (半双工)



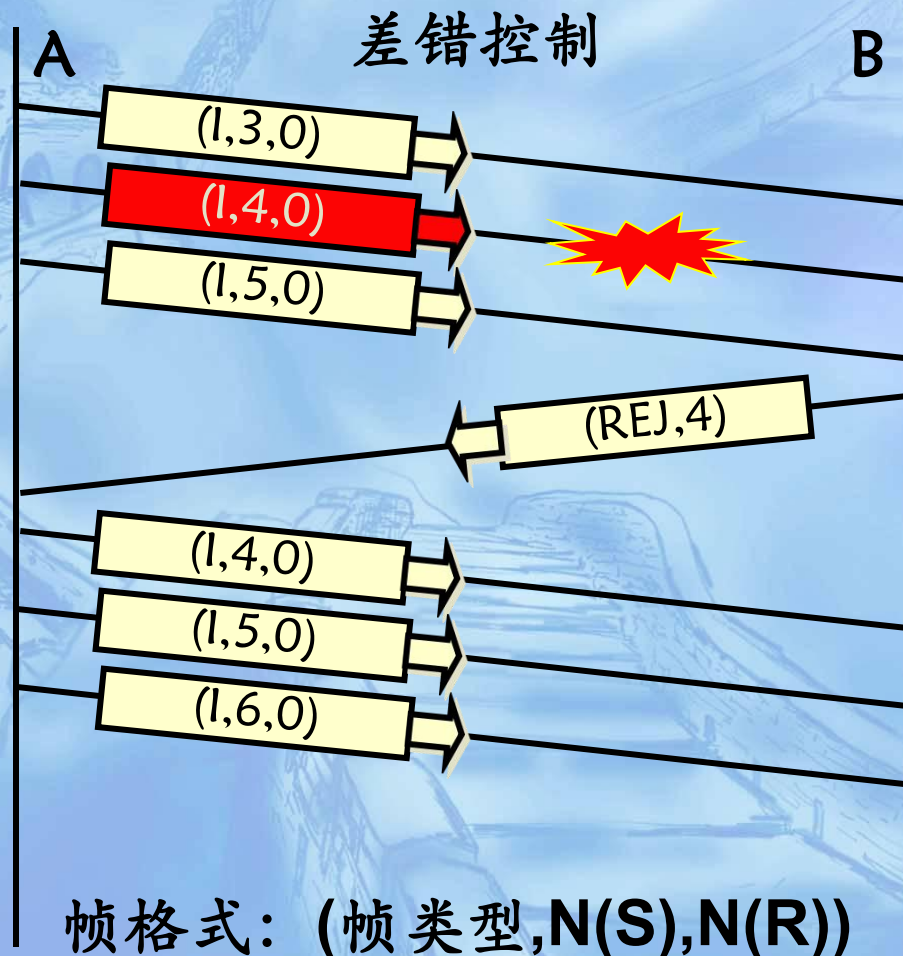


步骤二：数据传输 (半双工)



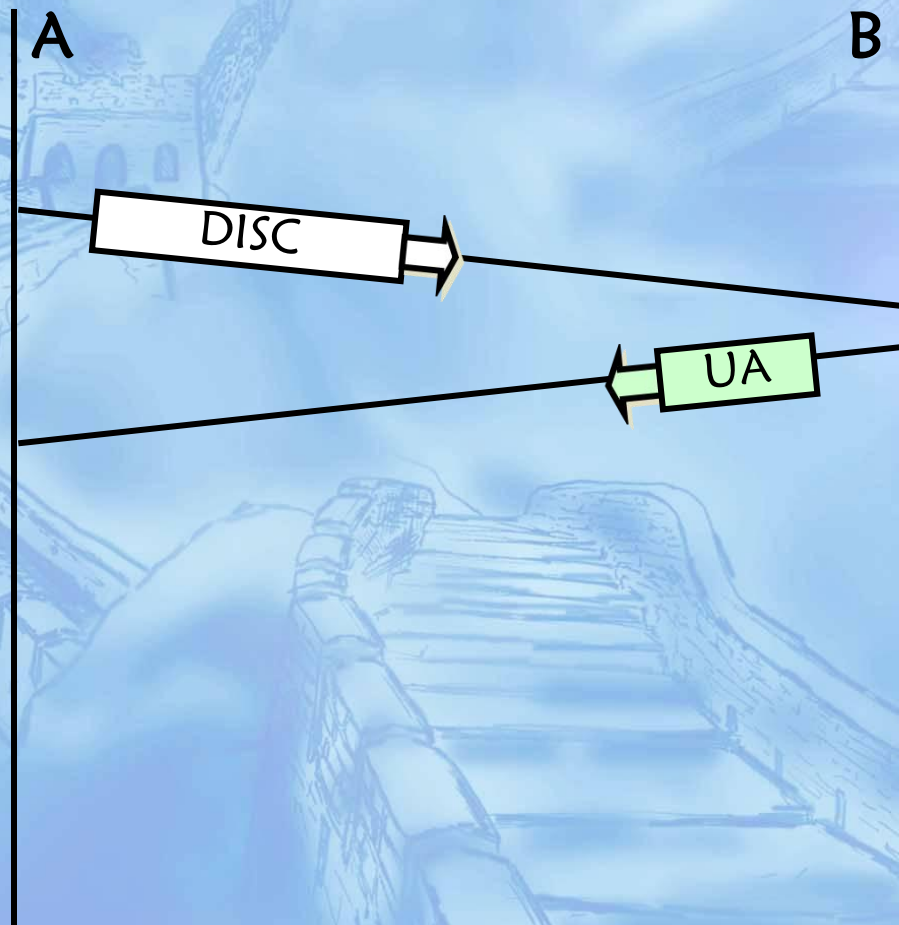


步骤二：数据传输 (半双工)





步骤三：断开连接





PPP

(Point-to-Point Protocol)

✍ 基本概念

✍ 帧格式

✍ LCP

✍ 认证协议

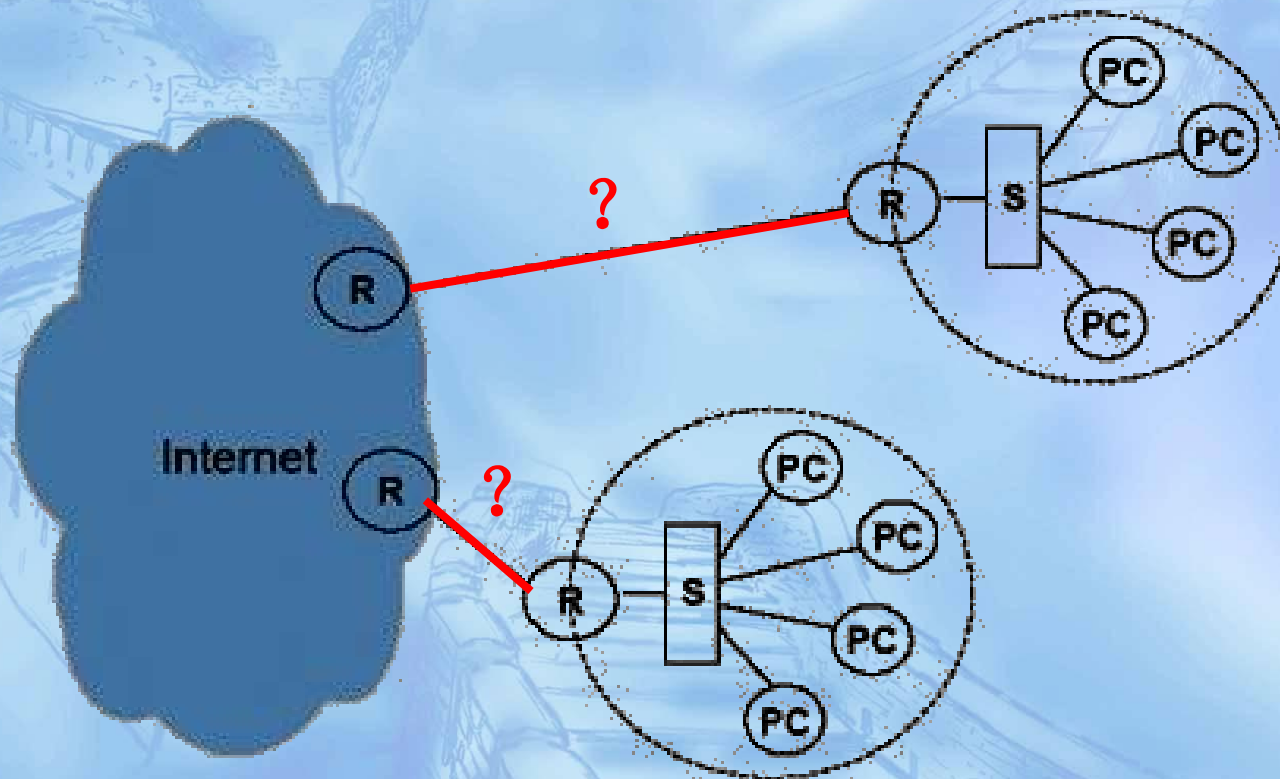
✍ NCP

✍ 工作过程



Internet接入方式

(路由器-路由器接入)

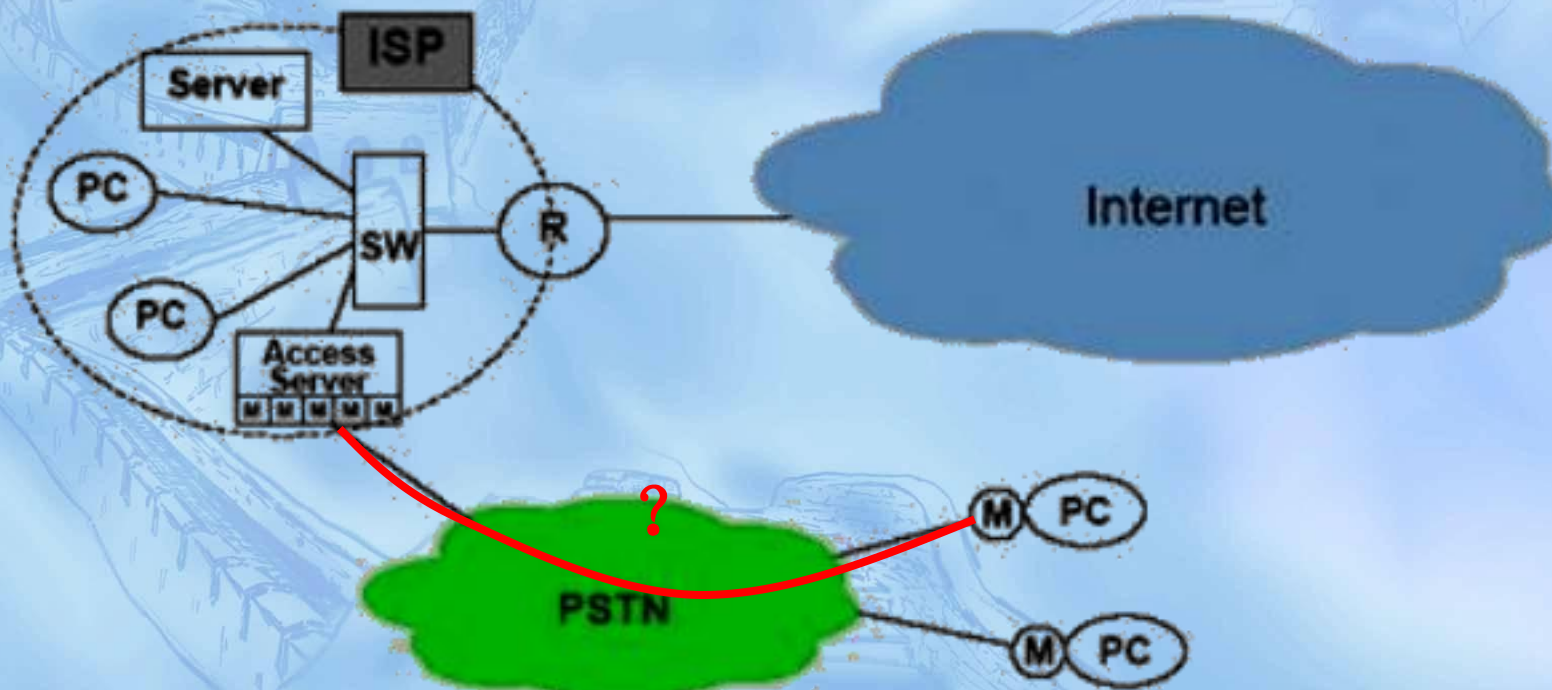


✎ 用于一个机构的一个或多个**LAN**的接入。其中路由器-路由器之间是通过**点-点连接**来实现。



Internet接入方式

(拨号主机-路由器接入)



✎ 用于个人的接入。远程家庭用户通过拨号与ISP连接后采用点-点连接。



PPP

- ✗ **PPP (RFC1661)** 是Internet上广泛使用的数据链路层协议，用于点对点的链路。例如：通过电话线拨号接入Internet的线路，路由器与路由器连接的线路。
- ✗ **PPP**在宽带接入中也衍生出新的协议，例如：**ADSL**接入中的**PPPoE** (PPP over Ethernet)、**PPPoA** (PPP over ATM)。
- ✗ **协议特点：**不可靠的数据传输（目前应用较多）



PPP协议的组成

✎ 串行链路上封装IP数据报的方法（数据成帧）

- ✦ 既支持异步链路（以字符单位进行传输），也支持面向比特的同步链路（例如：**SONET/SDH**）。

✎ LCP（Link Control Protocol）

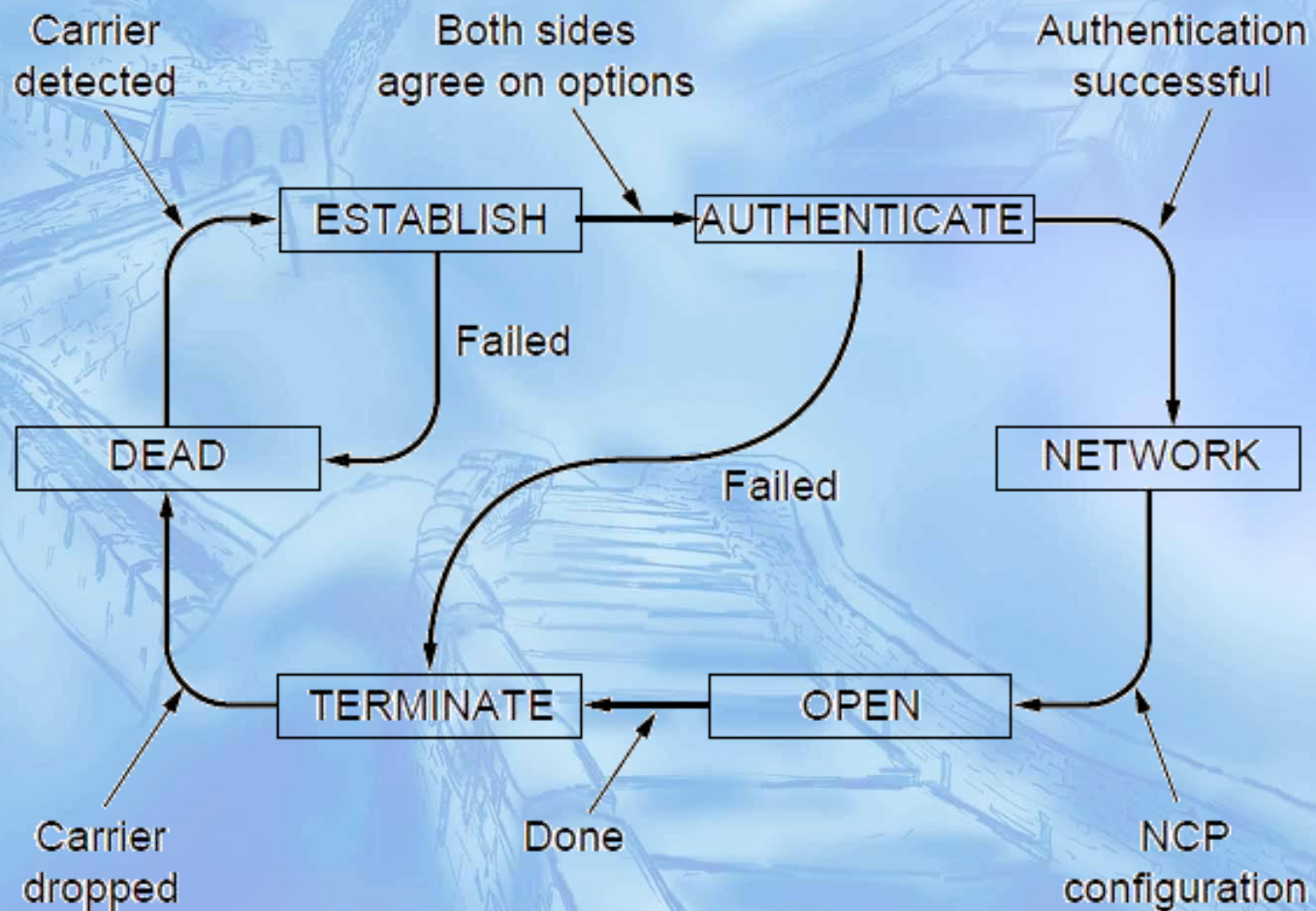
- ✦ 用来建立、维持和拆除数据链路，还有测试线路及协商选项等功能。

✎ NCP（Network Control Protocol）

- ✦ **NCP**是多个协议的总称，以与网络层无关的方法协商网络层选项，不同的网络层协议有不同的**NCP**，例如：适用于**IP**的**IPCP**、适用于**IPX**的**IPXCP**等。

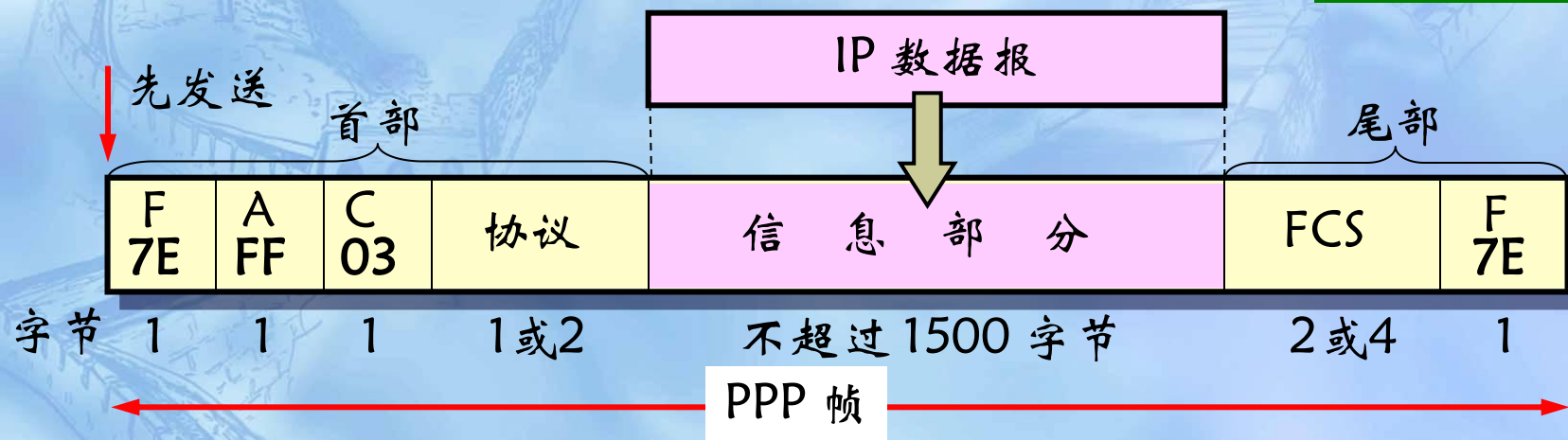


PPP协议的状态图





PPP协议的帧格式



✎ **标志字段 (F) : 1B (0x7E)**

- ✎ 标志一帧的开始和结束, 为实现数据透明性传输, 采用“字符插入/删除”技术。

✎ **地址字段 (A) : 1B (0xFF)**

- ✎ 广播地址。



透明性传输

✍ 同步传输链路

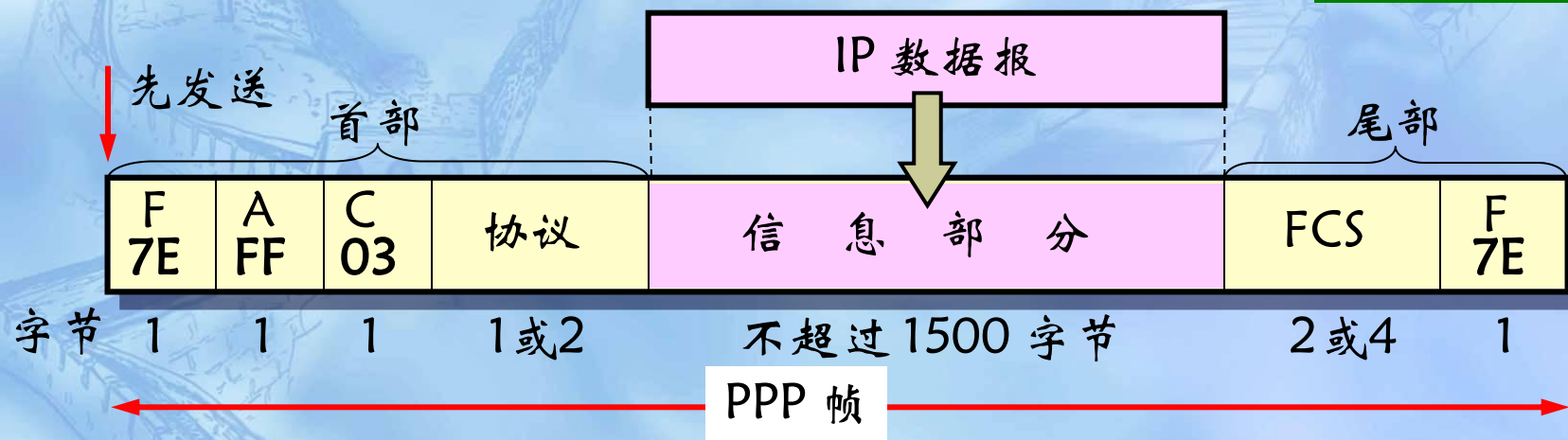
- ✦ 协议规定采用硬件来完成比特填充（和HDLC的做法一样）。

✍ 异步传输链路

- ✦ 使用一种特殊的字符填充法：将信息字段中出现的每一个**0x7E**字节转变成成为2字节序列(**0x7D, 0x5E**)；若信息字段中出现一个**0x7D**的字节，则将其转变成成为2字节序列(**0x7D, 0x5D**)；若信息字段中出现ASCII码的**控制字符**（即数值小于**0x20**的字符），则在该字符前面要加入一个**0x7D**字节，同时将该字符的编码加以改变（与**0x20**进行**XOR**操作）。



PPP协议的帧格式



✂ **控制字段 (C) : 1B**

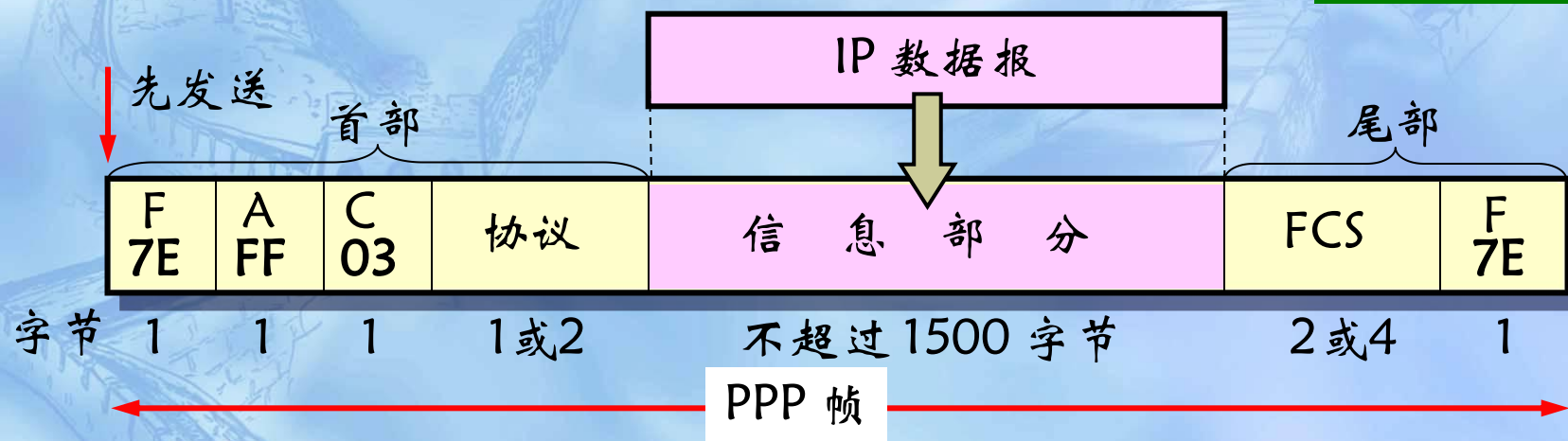
✚ 缺省为0x03, 表示采用**无序帧**方式进行传输 (没有序列号和确认)。

【注意】

A、C字段经LCP协商可以去掉以实现头压缩。



PPP协议的帧格式

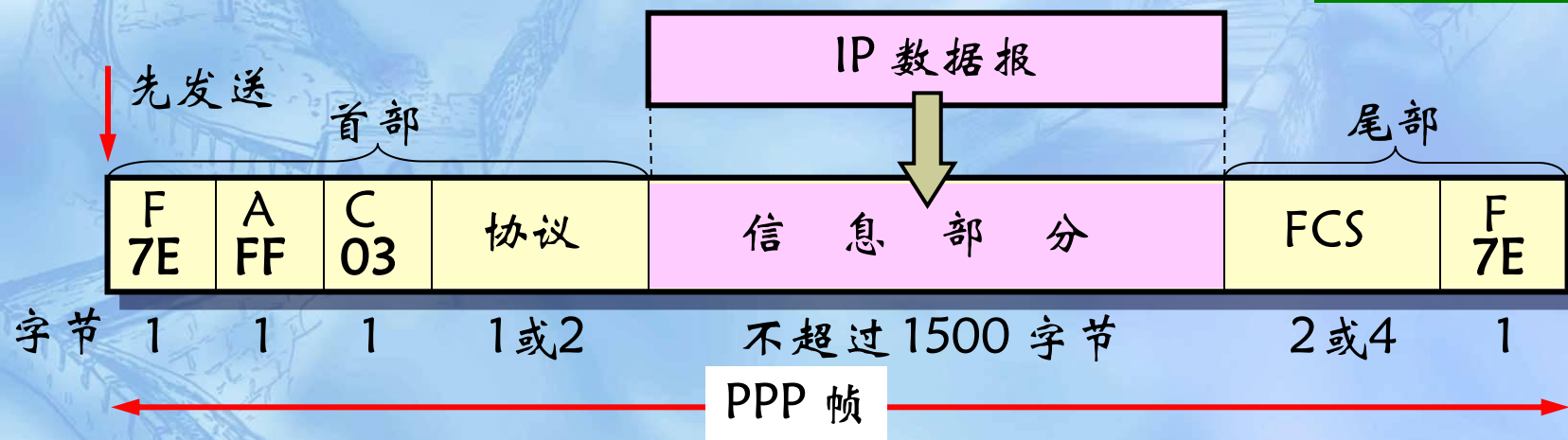


✎ 协议字段 (P) : 1B或2B

- ✎ 用于标识封装在帧信息字段中数据所属协议。例如：“0”开头表示网络层协议（例如：IP、IPX），“1”开头表示本协议簇中其它协议（例如：LCP、NCP）。
经LCP协商可压缩为1B。



PPP协议的帧格式



✎ 信息字段 (I)

✎ 用于存放用户数据，变长，缺省为1500B。

✎ 帧校验序列 (FCS) : 2B或4B

✎ 错误检测 (CRC)。默认为2B，可协商成4B。



问题：不使用序号和确认？

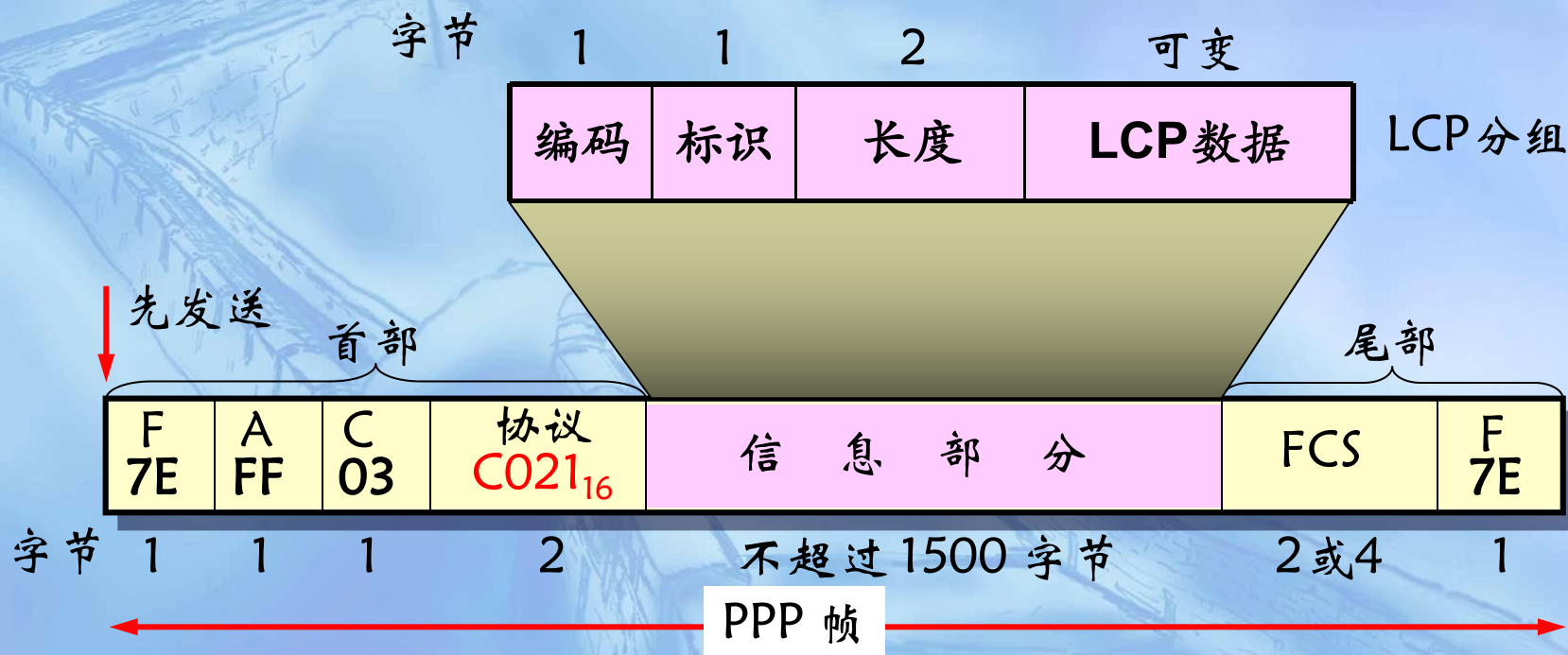
- ✍ **理由1**：在数据链路层出现差错的**概率不大**时，使用比较简单的**PPP**协议较为合理。
- ✍ **理由2**：在Internet环境下，**PPP**的信息字段放入的数据是**IP数据报**。数据链路层的可靠传输并不能够保证网络层的传输也是可靠的。
- ✍ **理由3**：**帧校验序列FCS**字段可保证无差错接收。



LCP

(Link Control Protocol)

作用：用来建立、维持和拆除数据链路，还有测试线路及协商选项等功能。





LCP分组

Code	Packet Type	Description
01 ₁₆	Configure-request	Contains the list of proposed options and their values
02 ₁₆	Configure-ack	Accepts all options proposed
03 ₁₆	Configure-nak	Announces that some options are not acceptable
04 ₁₆	Configure-reject	Announces that some options are not recognized
05 ₁₆	Terminate-request	Requests to shut down the line
06 ₁₆	Terminate-ack	Accepts the shut down request
07 ₁₆	Code-reject	Announces an unknown code
08 ₁₆	Protocol-reject	Announces an unknown protocol
09 ₁₆	Echo-request	A type of hello message to check if the other end is alive
0A ₁₆	Echo-reply	The response to the echo-request message
0B ₁₆	Discard-request	A request to discard the packet



通常的选项

Option	Default
Maximum receive unit	1500
Authentication protocol	None
Protocol field compression	Off
Address and control field compression	Off



认证协议

~~✍~~ PAP

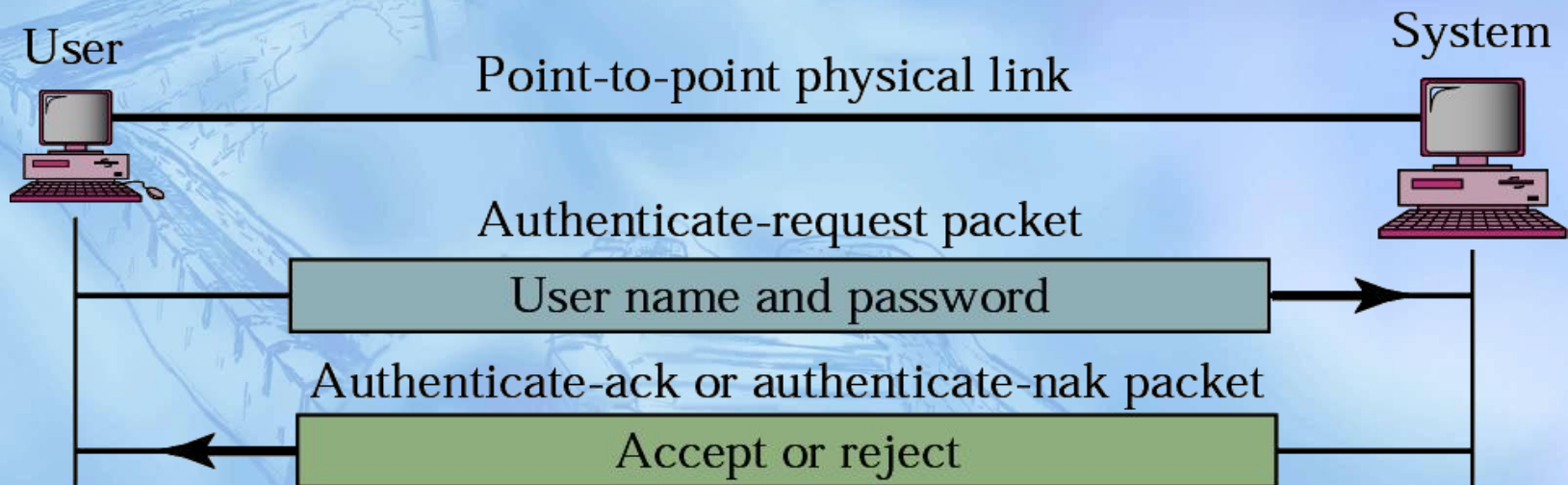
~~✍~~ CHAP



PAP

(Password Authentication Protocol)

✍ 工作过程:

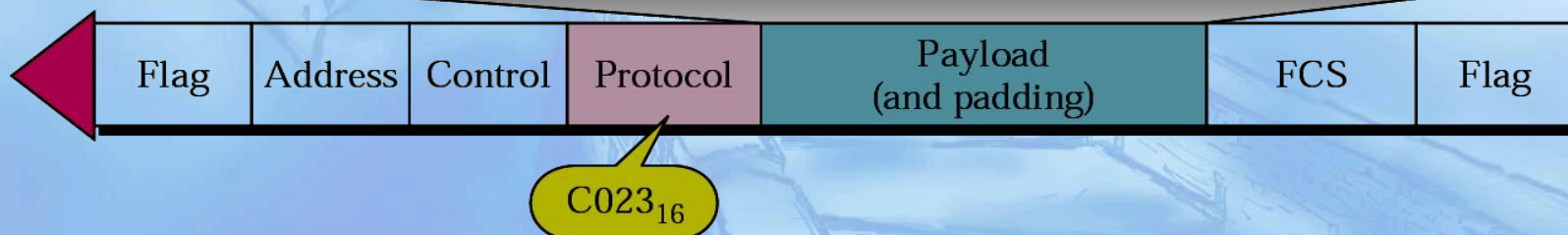




PAP分组

PAP packets

	1 byte	1 byte	2 bytes	1 byte	Variable	1 byte	Variable
Authenticate-request	Code: 1	ID	Length	User name length	User name	Password length	Password
	1 byte	1 byte	2 bytes	1 byte	Variable		
Authenticate-ack	Code: 2	ID	Length	Message length	User name		
	1 byte	1 byte	2 bytes	1 byte	Variable		
Authenticate-nak	Code: 3	ID	Length	Message length	User name		

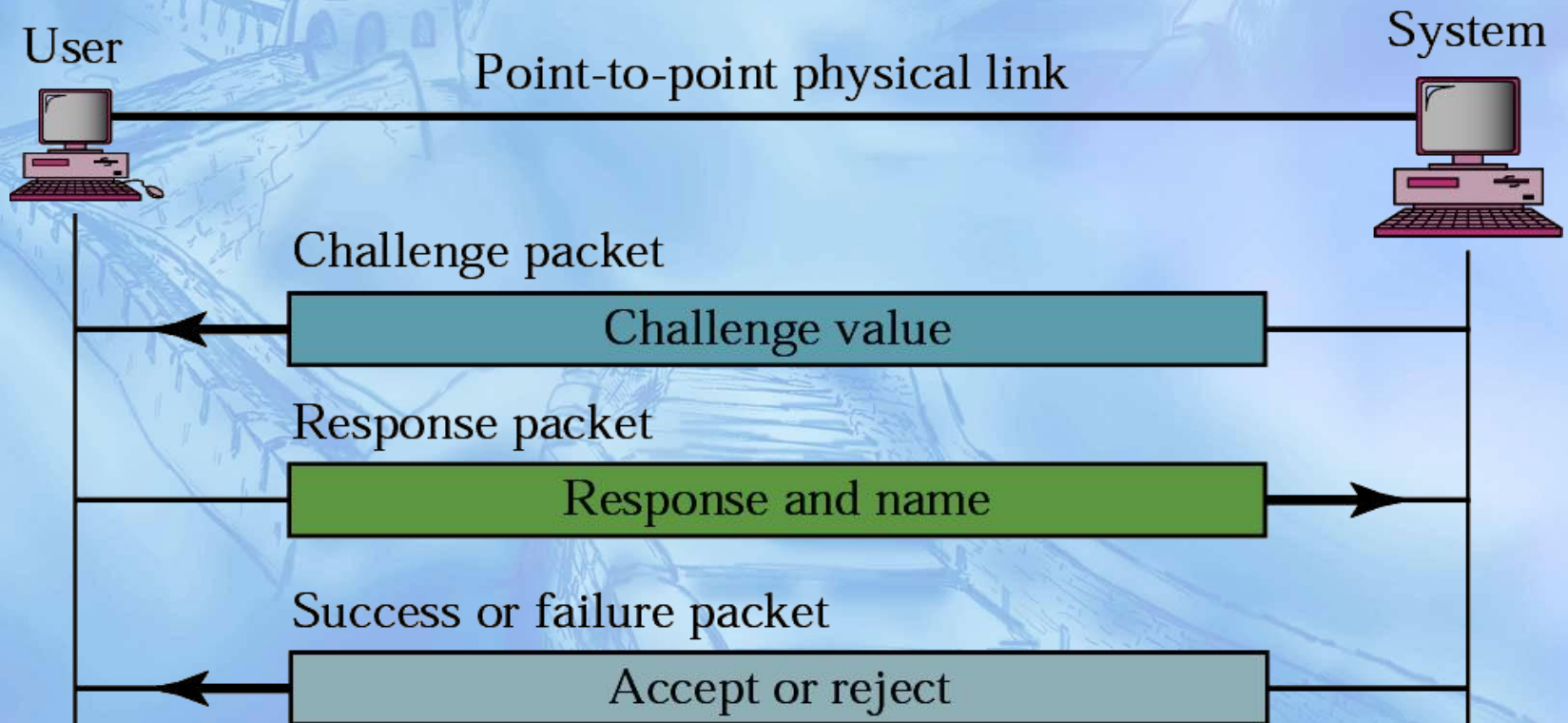




CHAP

(Challenge-Handshake Authentication Protocol)

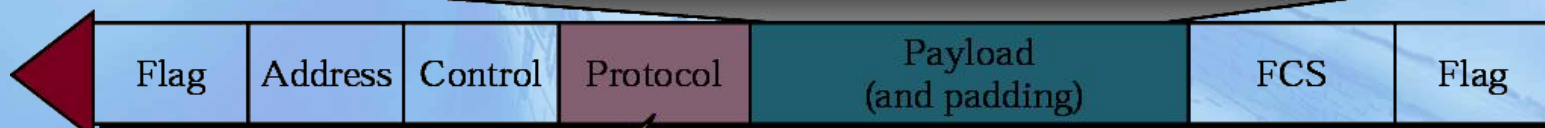
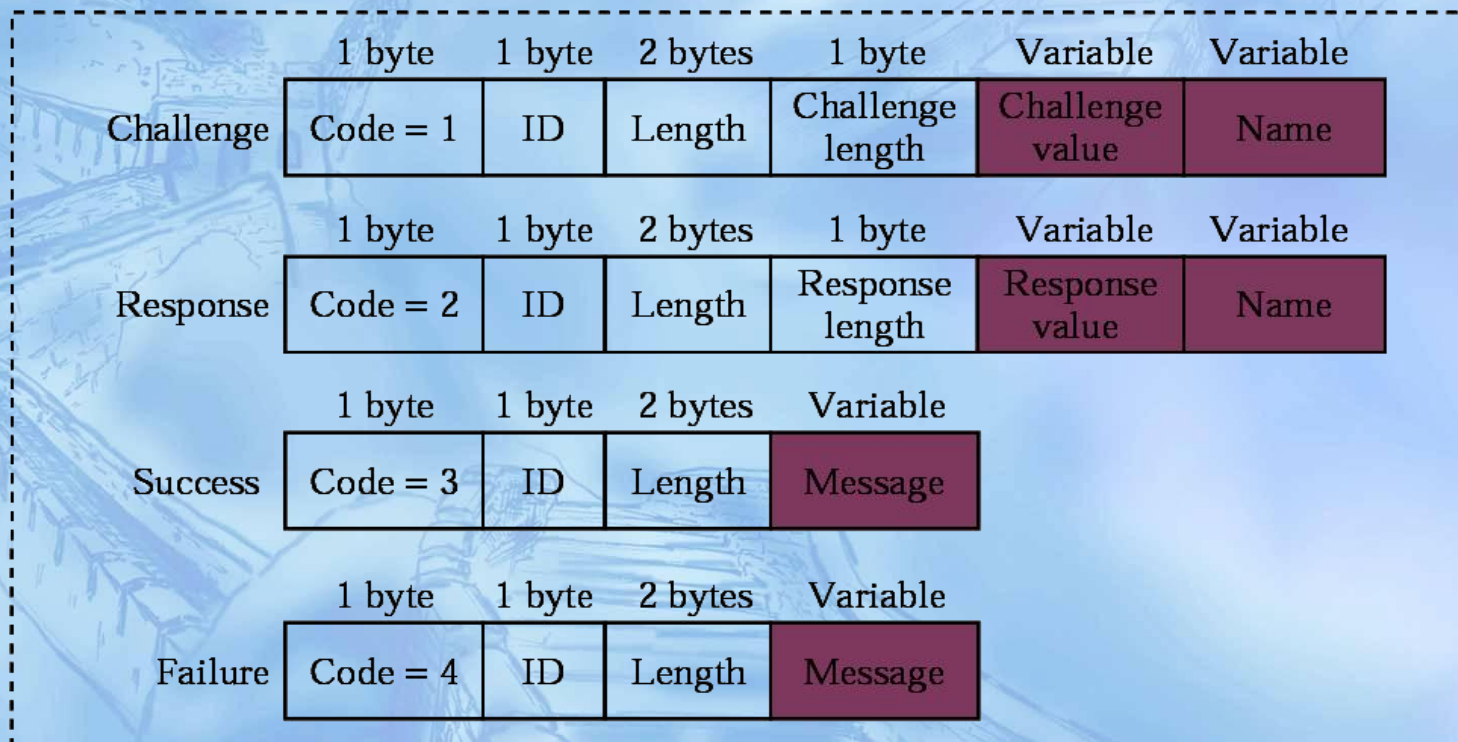
✍ **工作过程:**





CHAP分组

CHAP packets



C223₁₆



NCP

(Network Control Protocol)

作用

- ✚ 用来建立两端网络层的连接，协商网络层的选项和传递网络层数据。**NCP**不是一个协议，而是一组协议，包括**IPCP**、**IPXCP**等。

例子

- ✚ **IPCP (Internet Protocol Control Protocol)**
主要用来协商网络层选项，设置用户的**IP**地址。

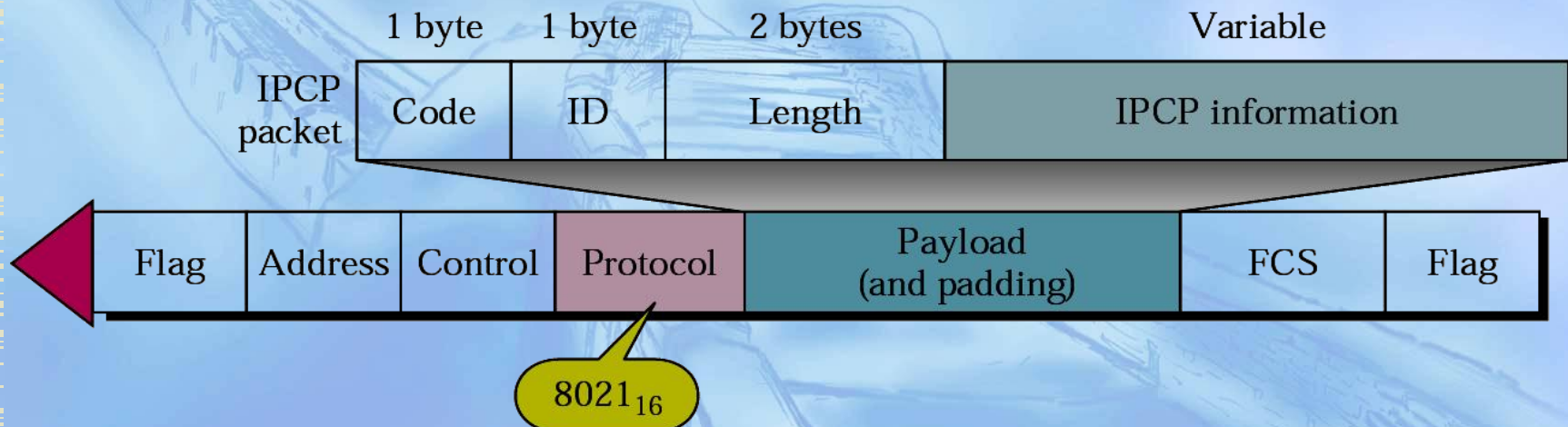


IPCP

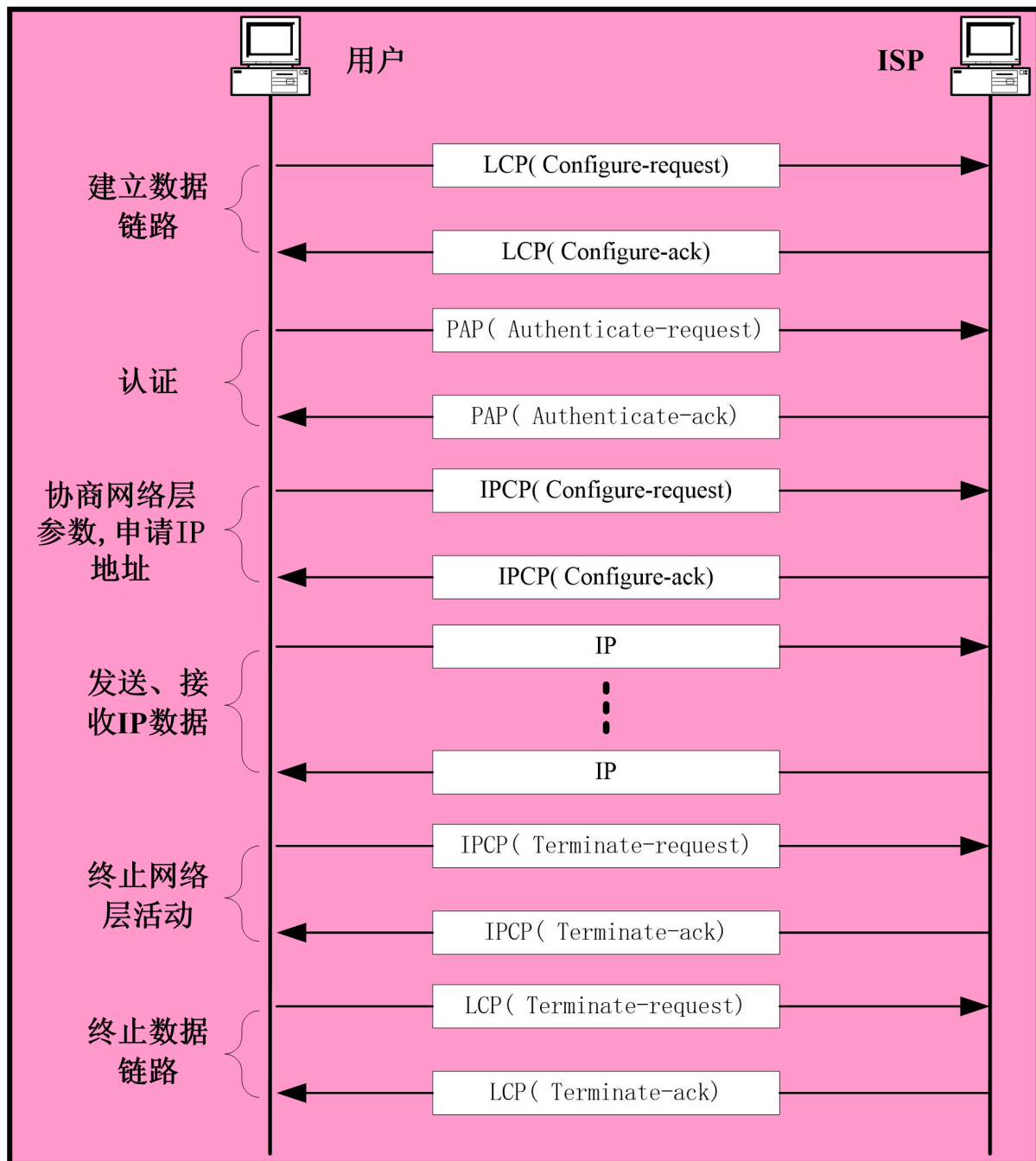
(Internet Protocol Control Protocol)

Code	IPCP Packet
01	Configure-request
02	Configure-ack
03	Configure-nak
04	Configure-reject

Code	IPCP Packet
05	Terminate-request
06	Terminate-ack
07	Code-reject



PPP协议的工作过程





作业

P201 2、24、30、33、35、
44、49

